Introduction to Artificial Intelligence and Machine Learning



Samadhi Chathuranga Rathnayake

MSc, MBA, PGDip, BSc (1st Class)

Introduction to Module

This is a compulsory unit for the 3rd year students who follow BSc (Hons) in IT specializing in the Data Science program.

The objective of this unit is to get an in-depth understanding of both the theory and practices in the field of Neural Networks and Optimization methods.

Introduction to Module

- In this unit, the students will learn:
  - Fundamentals of Neural Networks
  - Multilayer Neural Networks
  - Convolutional Neural Networks
  - Recurrent Networks
  - Reinforcement Learning
  - Support vector machines
  - Linear Programming
  - Genetic Algorithms

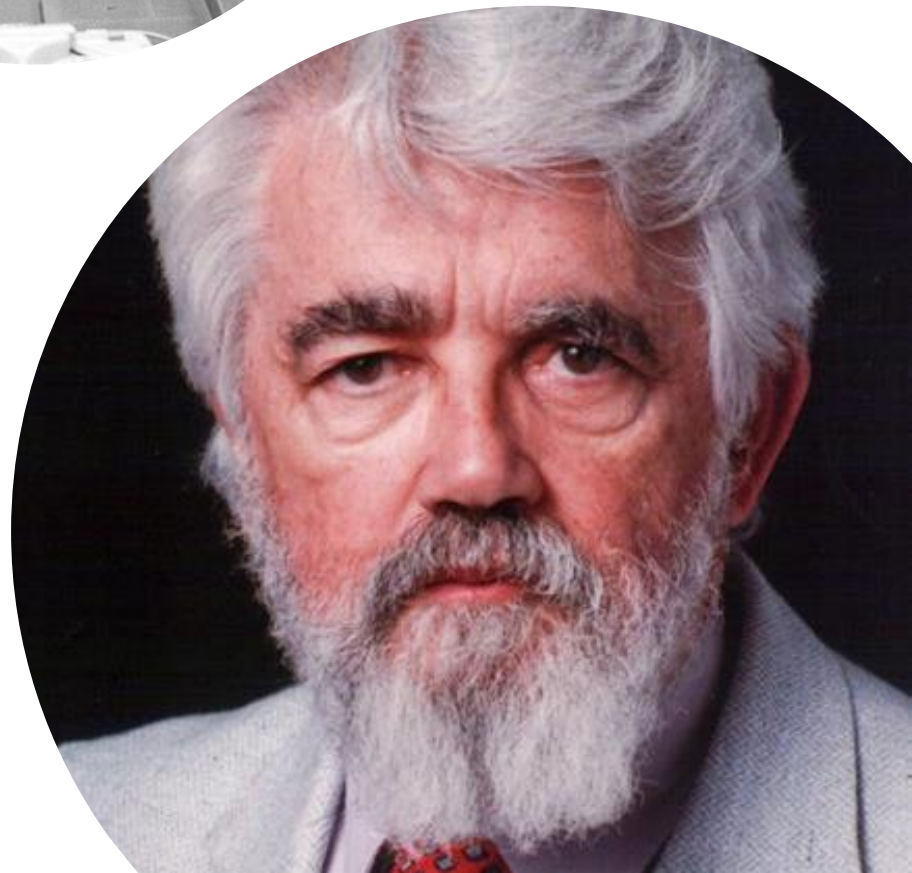
Assessments

- Assignment 1 (25%) – In Class Practical Assignment (25%)
- Assignment 2 (25%) – Group Assignment (25%)
- Final Exam (50%)

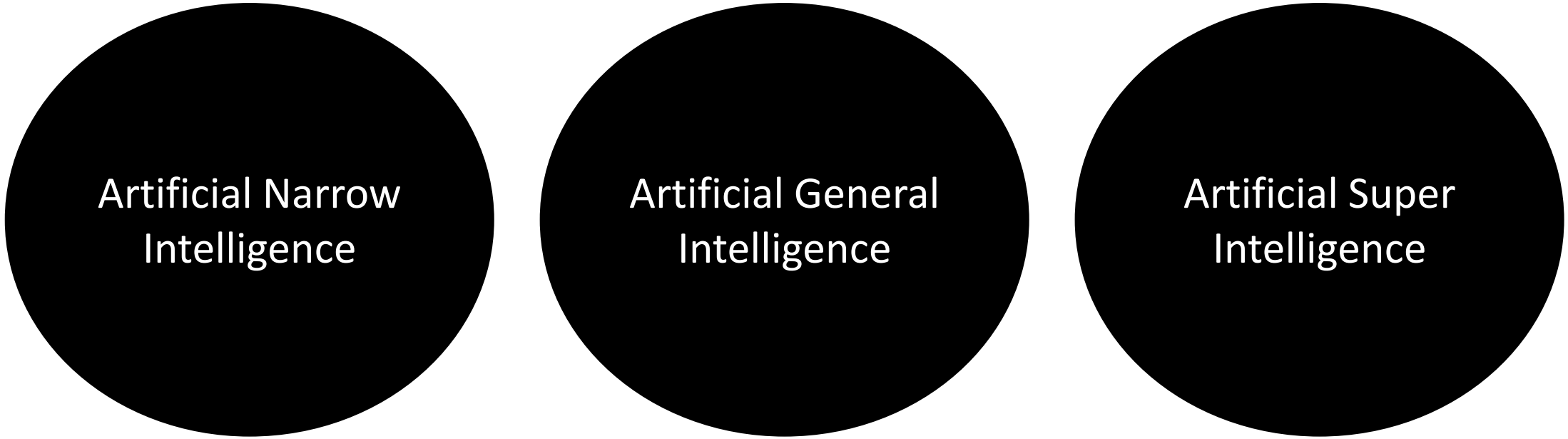
What is Artificial Intelligence?



“**The science and engineering of making intelligent machines, especially intelligent computer programs**” – John McCarthy



Main Types of AI



Artificial Narrow Intelligence (ANI)

- Also referred to as weak AI or narrow AI, is the only type of artificial intelligence we have successfully realized to date.
- Goal-oriented, designed to perform singular tasks - i.e. facial recognition, speech recognition/voice assistants, driving a car, or searching the internet - and is very intelligent at completing the specific task it is programmed to do.
- They operate under a narrow set of constraints and limitations, which is why this type is commonly referred to as weak AI.

Artificial General Intelligence (AGI)

- Artificial general intelligence (AGI), also referred to as strong AI or deep AI, is the concept of a machine with general intelligence that mimics human intelligence and/or behaviours, with the ability to learn and apply its intelligence to solve any problem.
- AGI can think, understand, and act in a way that is indistinguishable from that of a human in any given situation.
- AI researchers and scientists have not yet achieved strong AI.
- AGI focuses on **Theory of Mind (TOM)** AI framework.

Artificial Super Intelligence (ASI)

- Artificial superintelligence (ASI) is a form of AI that is capable of surpassing human intelligence by manifesting cognitive skills and developing thinking skills of its own.
- Also known as super AI, artificial superintelligence is considered the most advanced, powerful, and intelligent type of AI that transcends the intelligence of some of the brightest minds, such as Albert Einstein.
- Machines with superintelligence are self-aware and can think of abstractions and interpretations that humans cannot.

Agents

- An agent refers to any entity (software or hardware) that perceives its environment through sensors and acts upon it using actuators, with the goal of achieving specific objectives.
- Two types of agents
  - Software Agents
  - Physical Agents

Agents

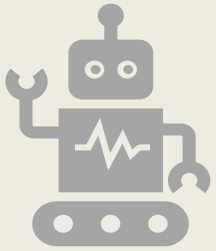


Software Agents

These agents exist solely in a digital environment.

They can interact with other software, systems, or the internet and are often used to automate tasks, solve problems, or make decisions.

Ex – Siri, Netflix Recommendation System



Physical Agents

These agents are embodied in the physical world and can interact with it through mechanical means.

They perceive the environment through sensors (like cameras or infrared) and act upon it using actuators (like motors or arms).

Ex- Robots, Autonomous Drones, Self Driving Cars

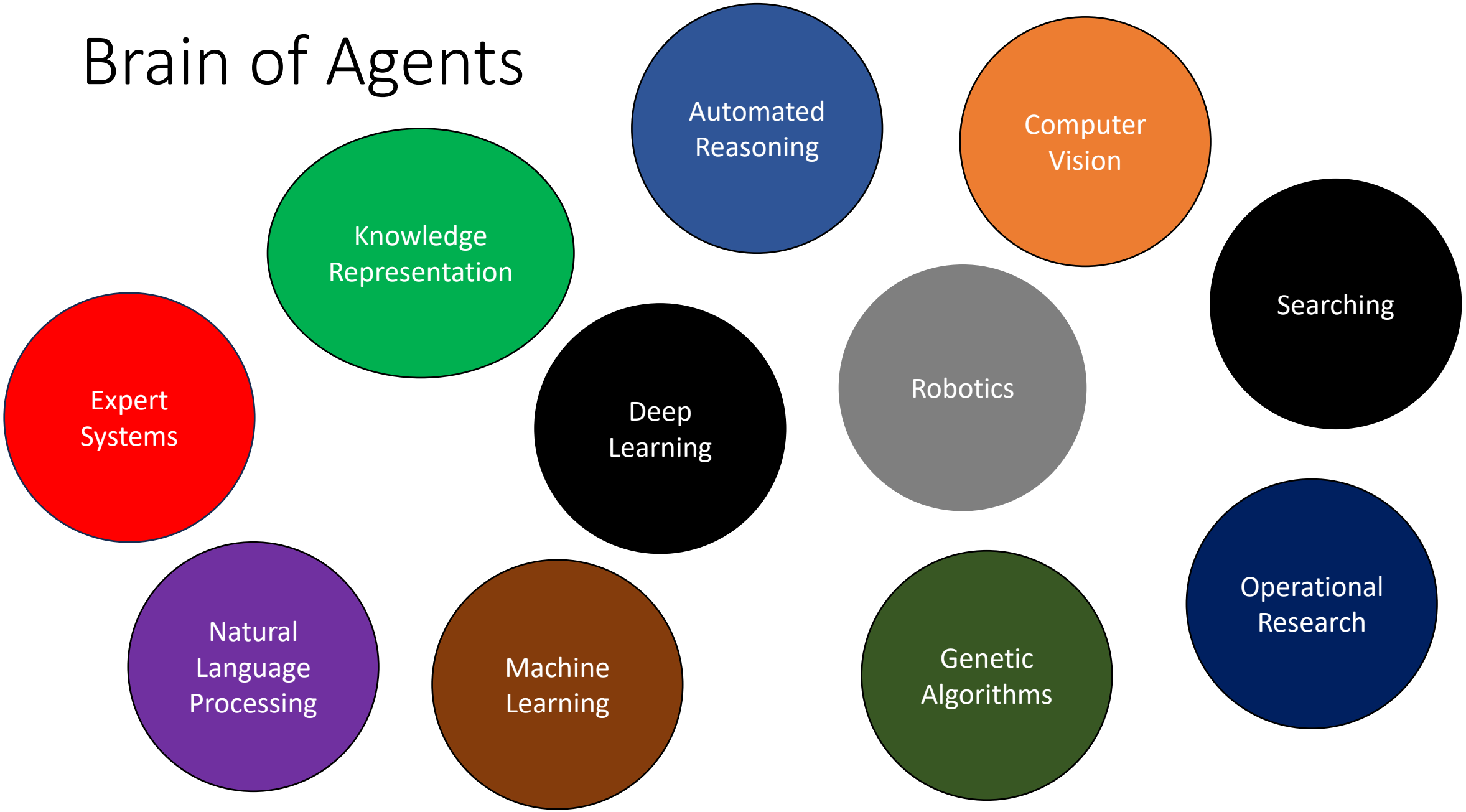
Single Agent Systems (SAS)

- A single-agent system is a system that consists of a single agent. An agent is an autonomous entity that can sense its environment, make decisions, and act to achieve its goals. Single-agent systems are often used to solve problems that are well-defined and have a single optimal solution.
- Examples of single-agent systems:
  - A chess-playing program
  - A robot that navigates a maze
  - A spam filter

Multi-Agent Systems (MAS)

- A multi-agent system (MAS) is a system that consists of multiple interacting agents. Agents in an MAS can cooperate or compete with each other in order to achieve their goals. Multi-agent systems are often used to solve problems that are complex and have multiple possible solutions.
- Examples of multi-agent systems:
  - A group of robots that collaborate to build a structure
  - A swarm of bees that collectively search for food
  - A team of players that cooperates to win a game

Brain of Agents



Hard Computing

- Hard computing refers to traditional, rule-based, deterministic computing where problems are solved using precise algorithms and logic.
  - Traditional Expert Systems
- Key Features:
  - Requires complete, precise input
  - Gives exact solutions
  - Examples: A banking system verifying if account balance ≥ transaction amount using precise logic.

Soft Computing

- Soft computing is a branch of computing that uses approximate reasoning and probabilistic models to handle uncertainty, imprecision, and partial truth similar to how humans make decisions..
- Key Features:
  - Inspired by biological systems and human reasoning
  - Works with incomplete, noisy, or imprecise data
  - Produces acceptable (but not always perfect) solutions

Soft Computing

- Components
  - Fuzzy Logic
  - Artificial Neural Networks (ANNs)
  - Evolutionary Algorithms (like Genetic Algorithms)
  - Probabilistic Reasoning (like Bayesian networks)
- Example: Predicting whether a customer will buy a product based on past behavior, even if the data is incomplete or noisy.

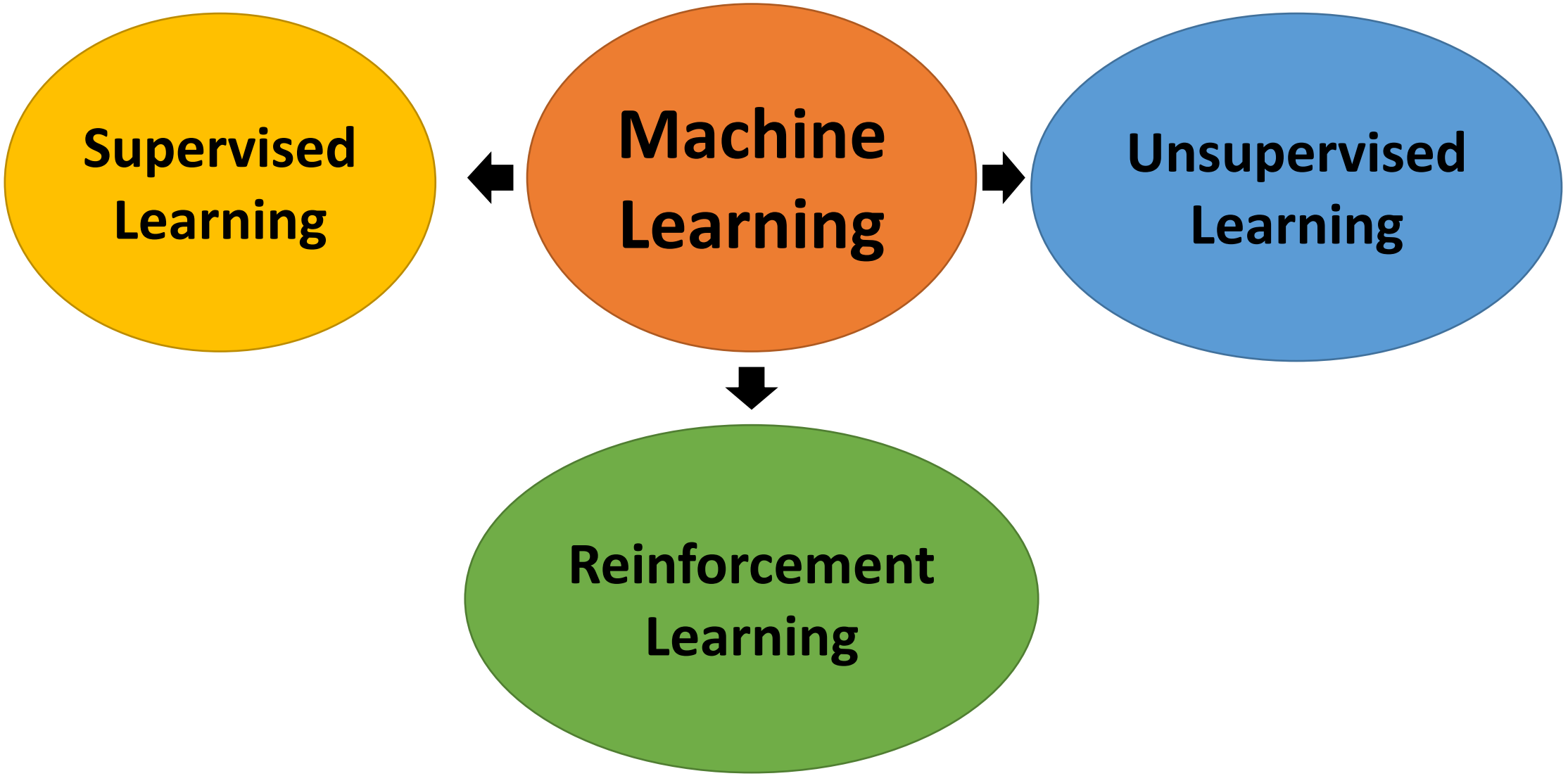
Hard Computing VS Soft Computing

Aspect	**Hard Computing**	**Soft Computing → ML**
Logic	Deterministic	Probabilistic / Approximate
Flexibility	Low	High
Handles Uncertainty	Poorly	Effectively
Rule Creation	Manual (rules coded)	Data-driven (learned from data)

Machine Learning

- ML algorithms learn patterns and relationships from data without being explicitly programmed.
- They improve their performance over time as they are exposed to more data.
- ML models represent knowledge implicitly in the form of statistical patterns learned from data. This knowledge is often stored in the parameters of the model.
- ML models (especially deep learning, probabilistic models, reinforcement learning, etc.) handle uncertainty, patterns, and generalization, key aspects of soft computing.

Machine Learning



Supervised Learning

- In Supervised Learning, teacher shows how to do
- Students learn by understanding the inputs & outputs
- The model will be trained with inputs and related outputs.
- After training it can make decisions based on predictions on unseen situations.



Underfitting and Overfitting

Overfitting refers to a model that models the training data too well. This means that the noise or random fluctuations in the training data is picked up and learned as concepts by the model. The problem is that these concepts do not apply to new data and negatively impact the models ability to generalize.

Underfitting refers to a model that can neither model the training data nor generalize to new data. An underfit machine learning model is not a suitable model and will be obvious as it will have poor performance on the training data.

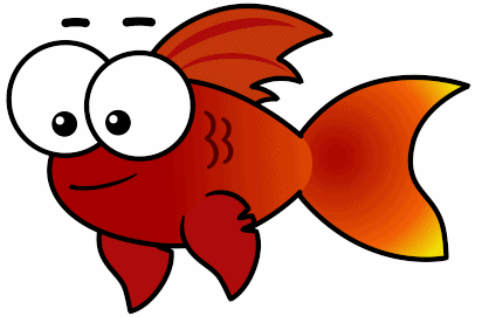
Unsupervised Learning

- Unsupervised learning, is where you only have input data (X) and no corresponding output variables.
- Derive some structure or pattern from the “unlabeled data” by just looking at the relationship between the data themselves.



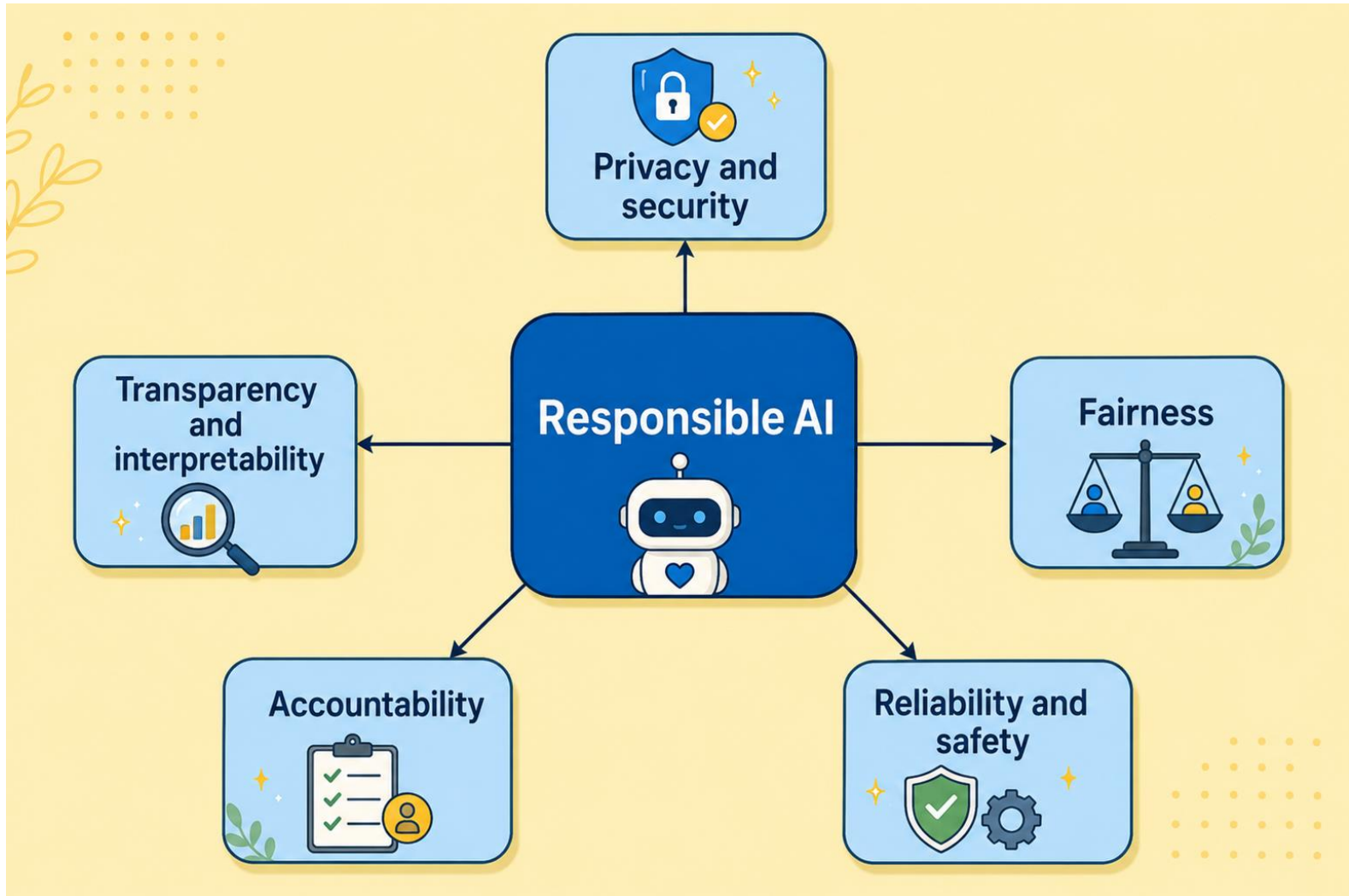






Reinforcement Learning

- It is about taking suitable action to maximize reward in a particular situation.
- It is employed by various software and machines to find the best possible behavior or path it should take in a specific situation.


Privacy and security
Transparency and interpretability
Responsible AI
Fairness
Accountability
Reliability and safety

Privacy and Security

- Responsible AI ensures that user data is protected from unauthorized access and misuse.
- Privacy refers to protecting individual identity, while security refers to protecting data systems and models from attacks.
  - Data Anonymization: Remove personal identifiers from datasets.
  - Differential Privacy: Add noise to data to prevent tracing individual records.
  - Federated Learning: Train models on user devices without centralizing the raw data.
  - Encryption: Protect data during storage and transfer.
- Ex- Apple's Face ID uses on-device machine learning, ensuring your face data never leaves the device, maintaining user privacy.

Fairness

- Fairness in AI is about preventing discrimination or bias based on gender, race, religion, age, or any protected attribute.
- It promotes equal treatment and opportunity for all users.
  - Data Bias: Biased datasets from real-world inequalities.
  - Algorithmic Bias: Model amplifies or introduces bias.
  - Labeling Bias: Subjectivity in how data was labeled.
- Ex- A hiring AI trained on historical resumes might prefer male applicants. Amazon discontinued such a system after it penalized resumes with the word “women’s”.

Reliability and Safety

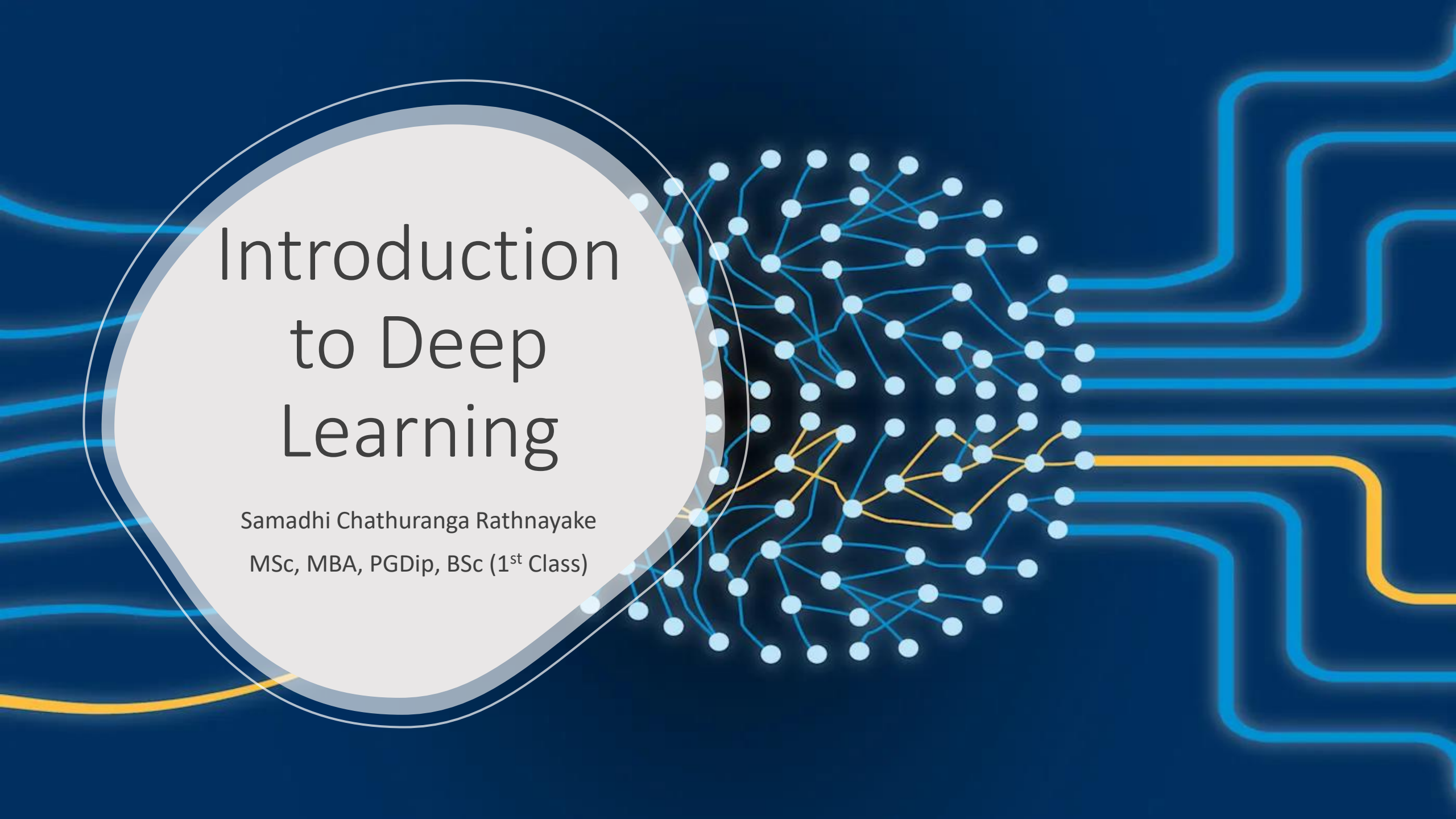
- AI systems should work consistently under varied conditions and fail safely if unexpected inputs are encountered.
- This is critical in real-world, high-risk settings.
  - Robust Training: Use adversarial training and diverse datasets.
  - Stress Testing: Simulate edge cases and real-world noise.
  - Fail-safe Mechanisms: Gracefully degrade the system when failure is imminent.
- Ex- Tesla’s Autopilot must detect obstacles, road signs, and pedestrians reliably—even in poor lighting or weather. A failure could cause fatal accidents.

Accountability

- Clear mechanisms must be in place to trace actions and assign responsibility when something goes wrong.
- This is critical for legal, ethical, and operational safety.
  - Audit Trails: Keep logs of data usage and model behavior.
  - Version Control: Maintain traceable versions of training data and models.
  - Defined Roles: Know who is responsible—data provider, developer, deployer, or operator.
- Ex- If an AI model denies a loan unfairly, a human officer or the company should be accountable, not the machine. Laws like the EU AI Act stress this point.

Transparency and Interpretability

- Stakeholders (developers, users, regulators) should be able to understand how an AI model works, especially when decisions affect people’s lives.
  - Explainable AI (XAI): Use methods like SHAP, LIME to visualize decision influences.
  - Model Cards: Document a model's purpose, performance, limitations.
  - Dataset Transparency: Document data sources, collection methods, and biases.
- Ex- A bank using AI for loan approvals must explain to rejected applicants why they were denied. Tools like SHAP can show that low income or credit score affected the decision most.



Introduction to Deep Learning

Samadhi Chathuranga Rathnayake

MSc, MBA, PGDip, BSc (1st Class)

Classical Machine Learning VS Deep Learning

- Classical Machine Learning relies on manual feature extraction and simpler algorithms to make predictions.
- Deep Learning automatically learns features from raw data using multiple layers of neural networks, making it more powerful for complex tasks like image and speech recognition.
- Classical Machine Learning works like traditional programming with human-crafted features, while Deep Learning mimics the human brain by using artificial neural networks that automatically learn patterns.
- This biological inspiration allows Deep Learning to handle complex data like images and sounds more effectively.

Classical Machine Learning VS Deep Learning

Aspect	Classical Machine Learning	Deep Learning
Feature Engineering	Requires manual feature selection and engineering	Learns features automatically from raw data
Data Requirement	Performs well with smaller datasets	Requires large amounts of labeled data
Model Complexity	Models are generally simpler (Ex- decision trees, SVM, logistic regression)	Models are complex with many layers (e.g., deep neural networks)
Interpretability	Easier to interpret and explain	Harder to interpret (often considered “black-box” models)

Classical Machine Learning VS Deep Learning

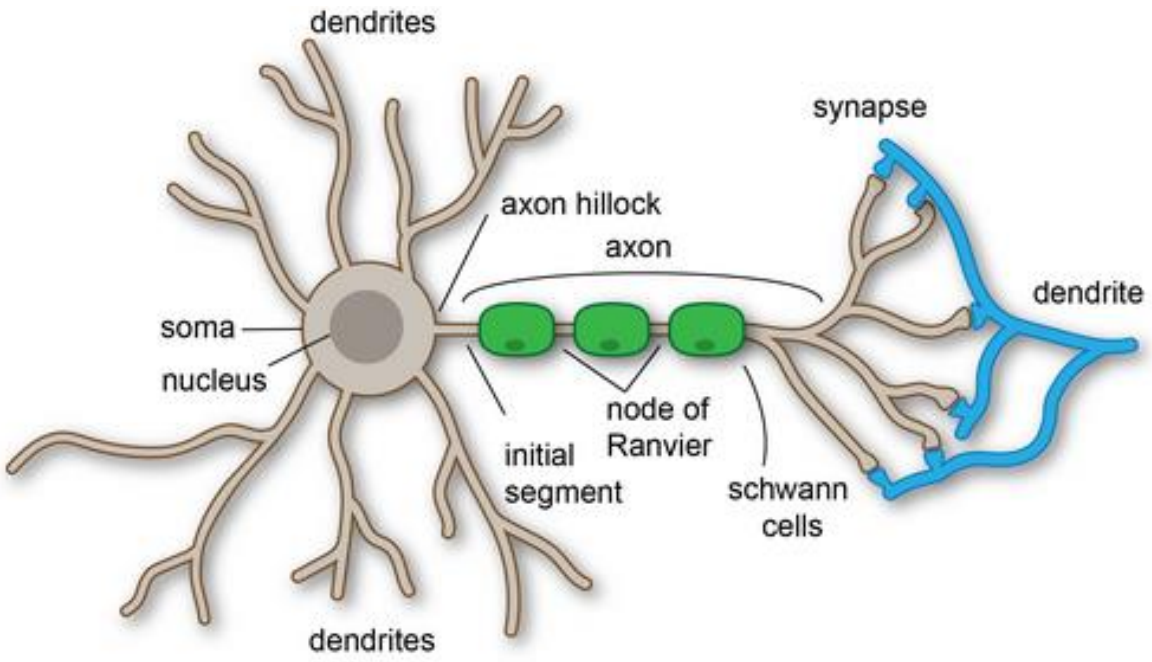
Aspect	Classical Machine Learning	Deep Learning
Computational Power	Works on standard CPUs	Requires GPUs/TPUs for efficient training
Training Time	Generally faster to train	Takes longer due to deeper architectures
Application Areas	Tabular data, structured data tasks	Image, audio, video, NLP, and other unstructured data tasks
Dependency on Domain Knowledge	High – domain expertise often needed for feature design	Low – learns features automatically from data

Deep Learning

- Brain inspired systems which are intended to replicate the way that we humans learn.
- Consist of input and output layers, as well as (in most cases) a hidden layer consisting of units that transform the input into something that the output layer can use.
- When a child is born, what does the child know? To our knowledge, the child knows only how to cry The child probably does not know its parents.
- When the child grows, the step by step learning process begins First, the child learns to drink milk
- Then the child learns to identify its parents Every time a child learns something, it is encoded into some portion of the brain.
- If we do not practice what we learned, we start to forget Consequently, by practice or training, we can hard code some selected things into our brains.

Deep Learning

- Neuroscientists believe that learning stimulates new dendrite connections between neurons.
- Greater usage of the brain through learning and stimulation creates greater dendrite connectivity thus, as we learn more and more, we become more intelligent.
- Wisdom is not created through genetics Wisdom and knowledge are based on how we learn and how we practice what we learned.



Perceptron

This the general process of decision making inside a biological neuron.

Perceptron

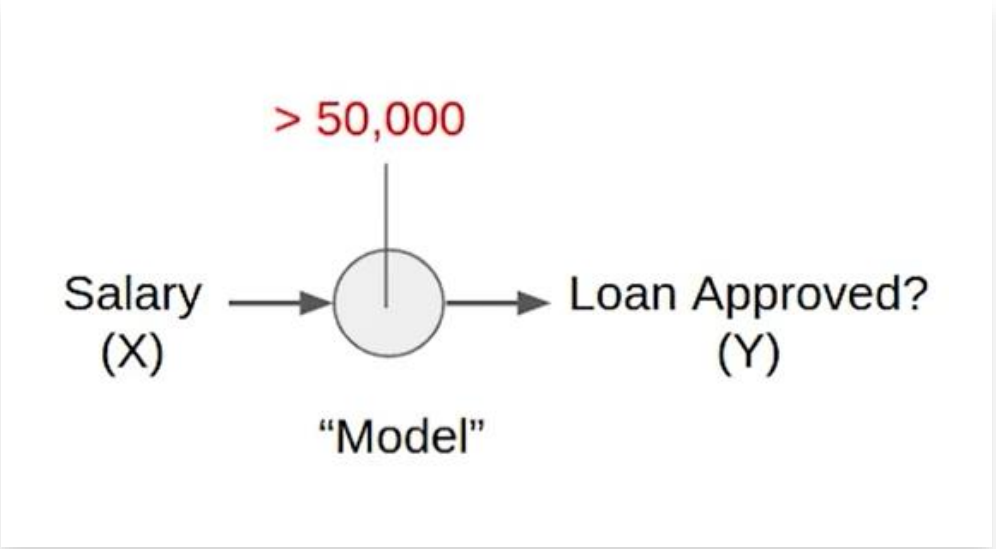
Think about following question. Here with the salary, it will check whether the loan is approved or not.



Perceptron

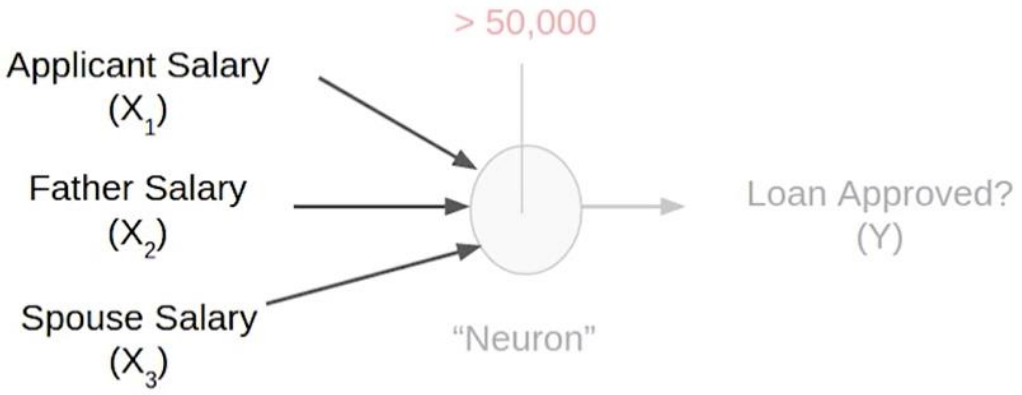


Think that the loan is going to approved when the salary is greater than 50000.



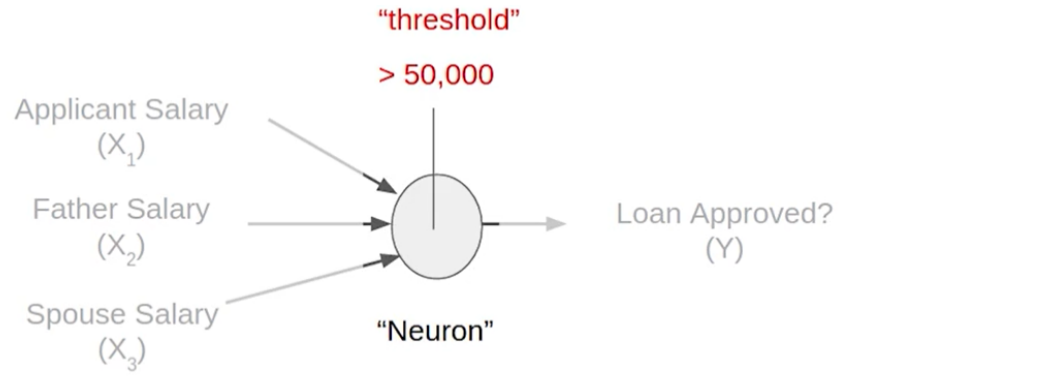
Perceptron

More than one input variables can be considered.



Perceptron

Consider that the addition of these input values is going to be compared with the threshold.



Total Income = Applicant Salary (X_1) + Father Salary (X_1) + Spouse Salary (X_3)

Total Income > threshold ?

Perceptron

Consider that the addition of these input values is going to be compared with the threshold.

$$X_1 + X_2 + X_3 > T$$

$$X_1 + X_2 + X_3 - T > 0$$

$$X_1 + X_2 + X_3 + Bias > 0$$

$X_1 + X_2 + X_3 + Bias > 0$ 1

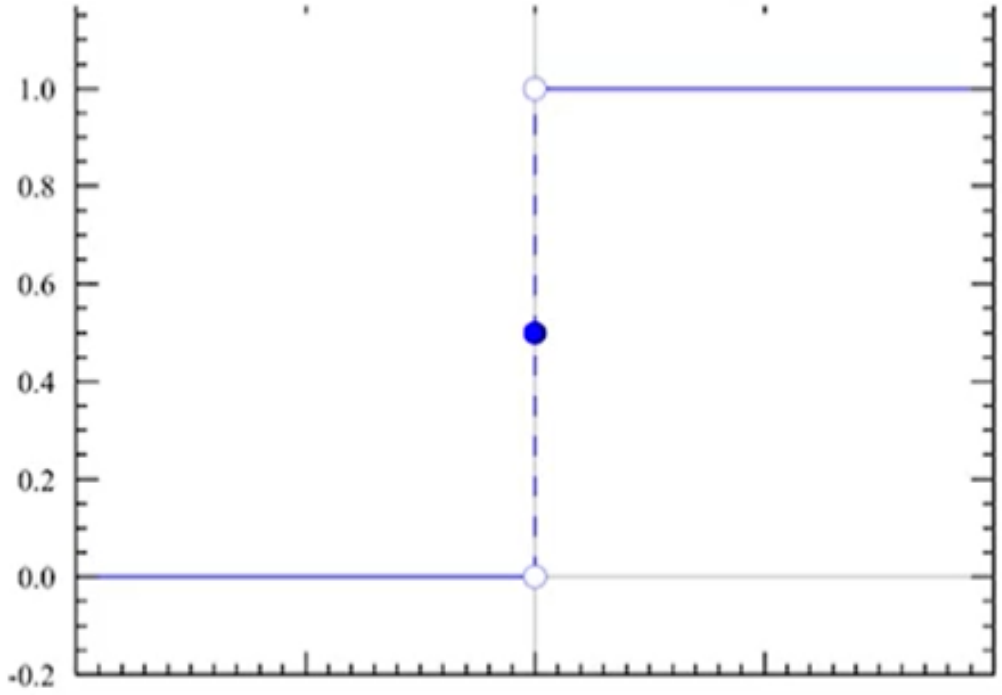
$X_1 + X_2 + X_3 + Bias < 0$ 0

Perceptron

This follows the step function.

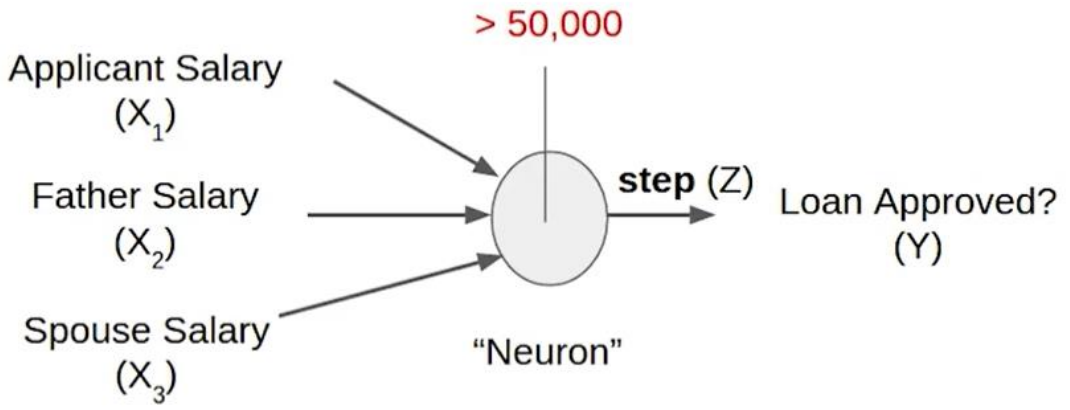
$$Z = X_1 + X_2 + X_3 + Bias$$

$$Step(Z) = Y = \begin{cases} 1 & Z > 0 \\ 0 & Z \leq 0 \end{cases}$$



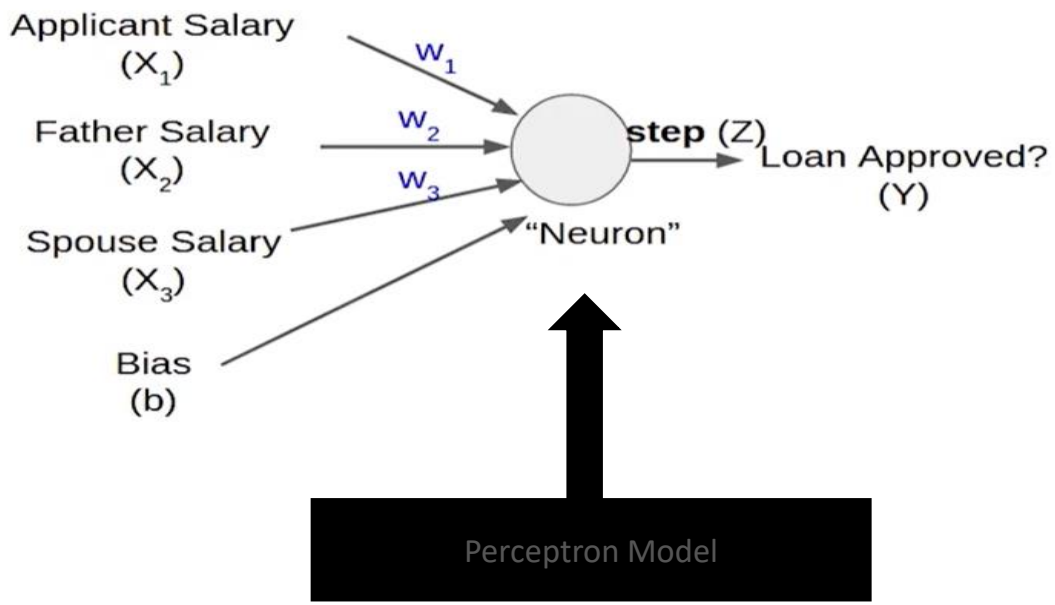
Perceptron

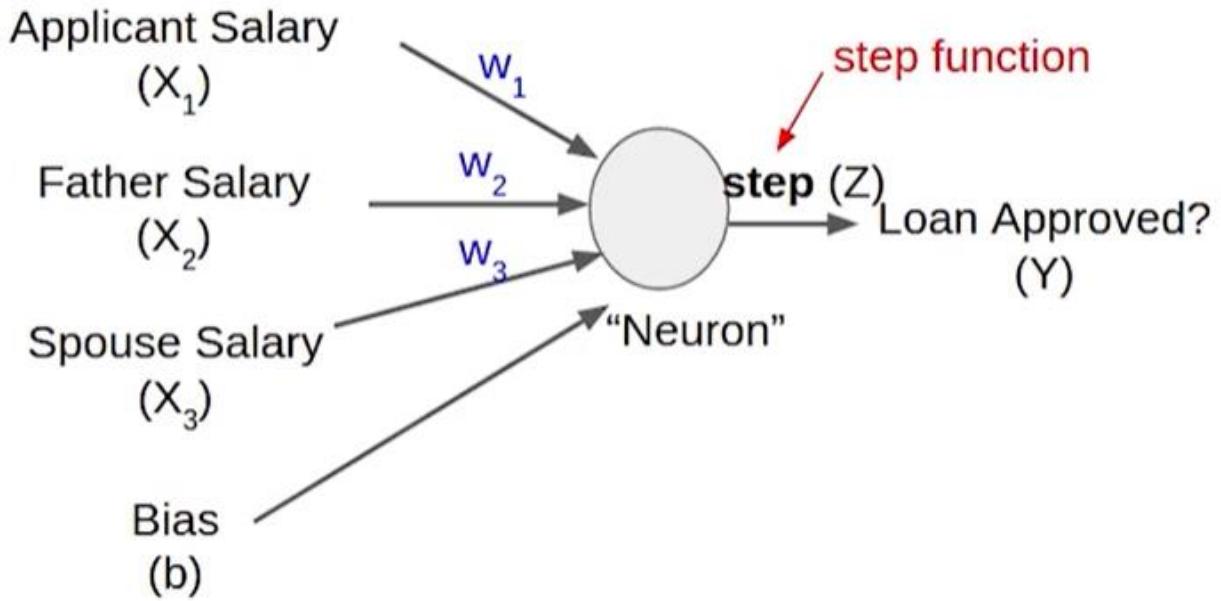
Finally, the system can be obtained as,



Perceptron

The weight can be changed for each input.





Sum of inputs = $X_1 * w_1 + X_2 * w_2 + X_3 * w_3$

$Z = X_1 * w_1 + X_2 * w_2 + X_3 * w_3 + b$ (bias)

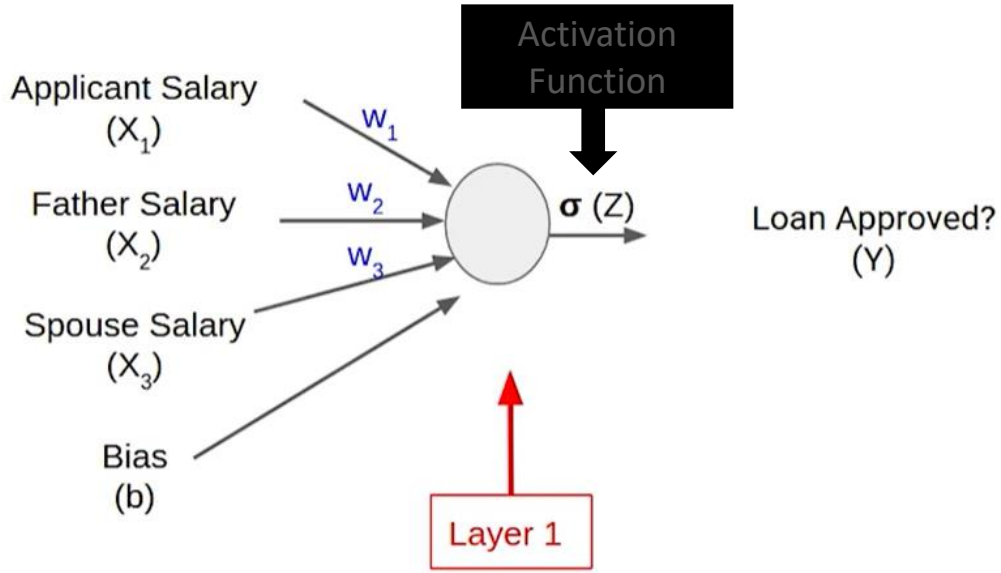
$\hat{Y}$ (output) = **step** (Z)

Perceptron

Weights and bias values can be changed and updated.

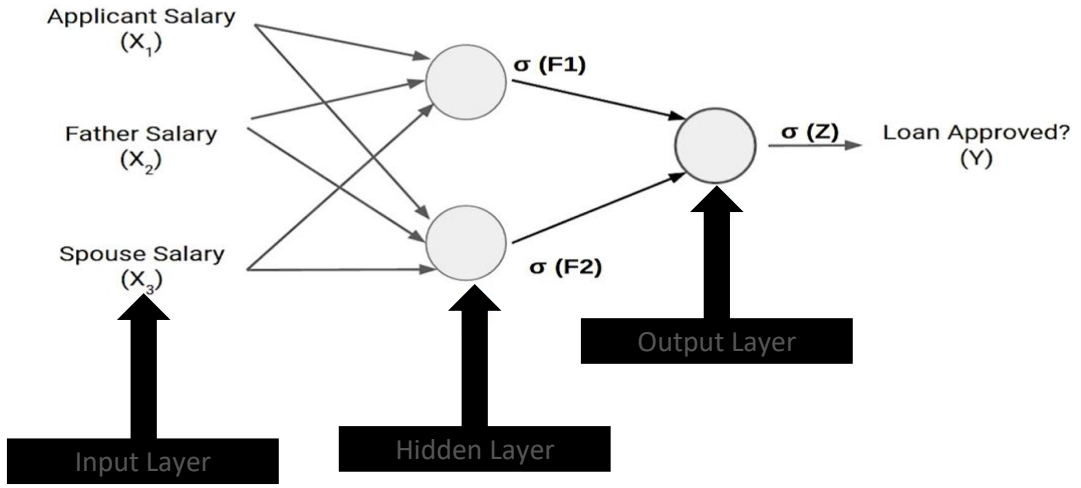
Single Layer Perceptron

Here we have only one neuron in the hidden layer.



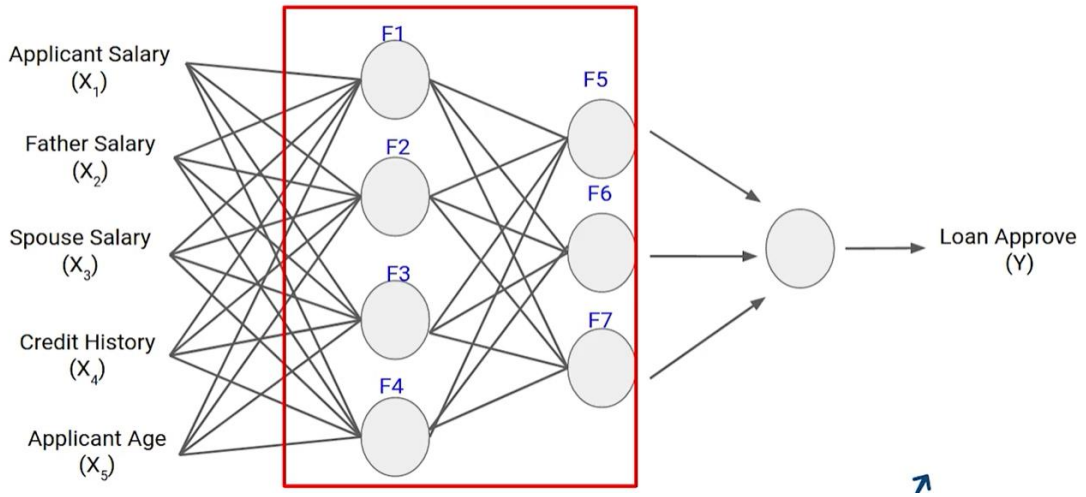
Multi Layer Perceptron

We can have more than one neuron in the hidden layer.

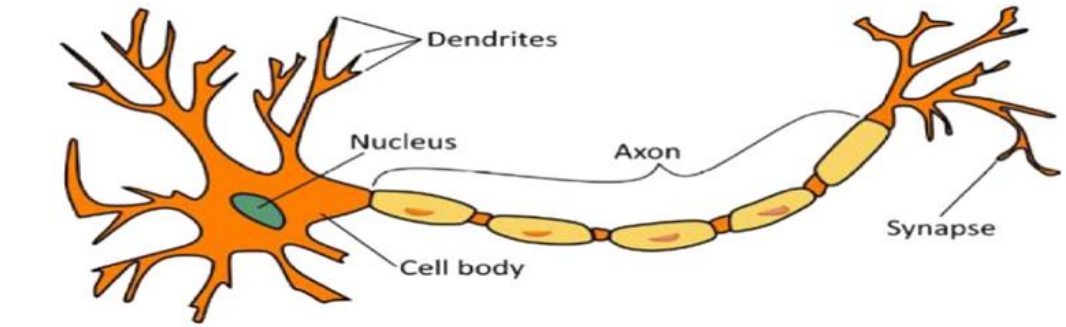


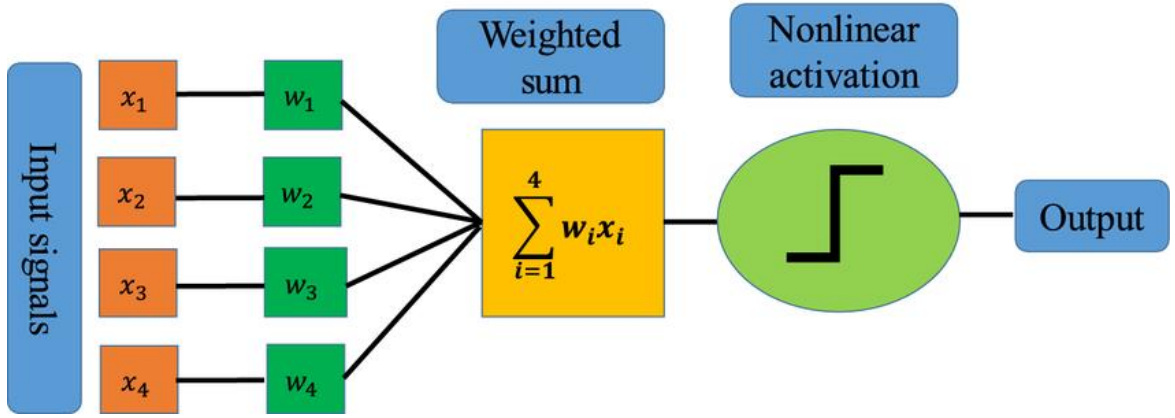
Multi Layer Perceptron

More than one layers can be added.



Biological Neuron vs Artificial Neuron





Biological Neuron vs Artificial Neuron

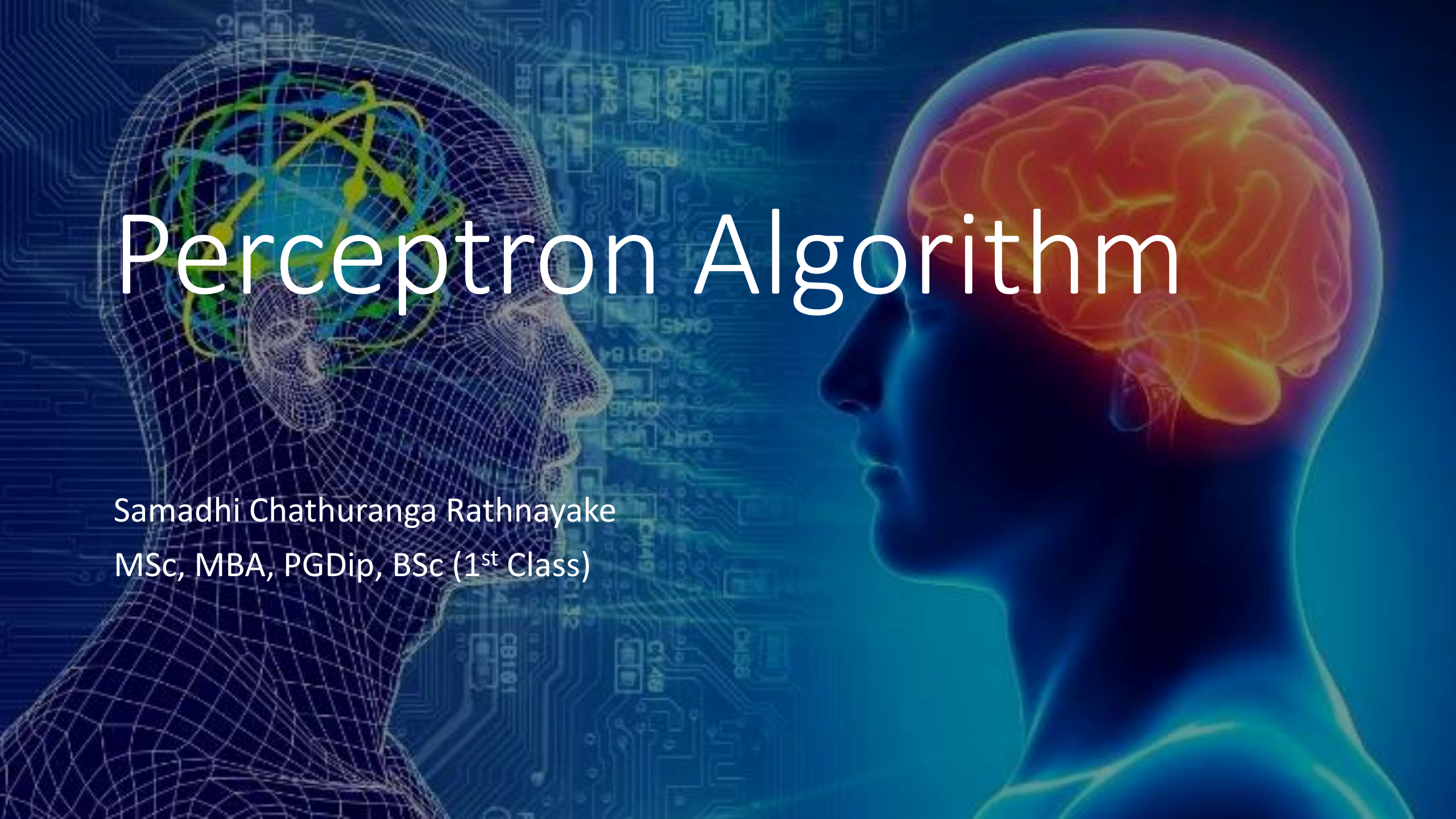
Biological Neuron	Artificial Neuron	Functionality
Dendrites	Input Features (x_1, x_2...)	Dendrites receive signals from other neurons, just like how an artificial neuron receives input values (features) from data.
Synaptic Weights	Weights (w_1, w_2...)	Synapses strengthen or weaken signals; similarly, each input in an artificial neuron is multiplied by a weight that adjusts the signal's importance.

Biological Neuron vs Artificial Neuron

Biological Neuron	Artificial Neuron	Functionality
Cell Body (Soma)	Summation Function	The soma integrates the signals, similar to how an artificial neuron computes the weighted sum: $z = \sum(w_i \cdot x_i) + b$.
Axon Hillock	Activation Function	The axon hillock decides whether to fire or not. In an artificial neuron, this is handled by an activation function (e.g., ReLU, Sigmoid, Step).

Biological Neuron vs Artificial Neuron

Biological Neuron	Artificial Neuron	Functionality
Axon	Output Value	The axon transmits the output signal. In the artificial neuron, the result of the activation function becomes the output to the next neuron/layer.
Synapse (at Axon End)	Connection to next layer	Just like a synapse sends signals to other neurons, the output of one artificial neuron becomes input to neurons in the next layer.



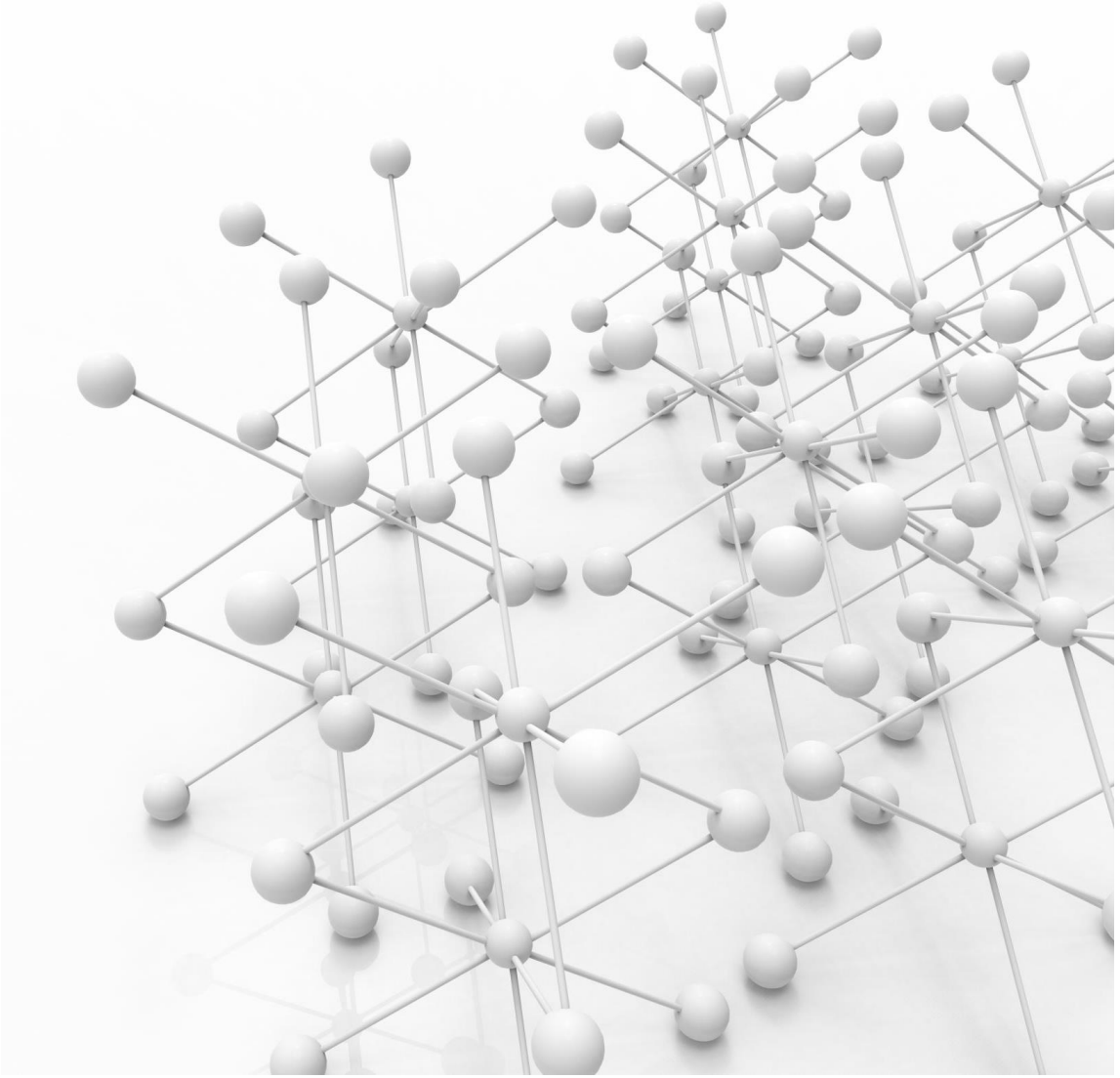
Perceptron Algorithm

Samadhi Chathuranga Rathnayake

MSc, MBA, PGDip, BSc (1st Class)

What is Perceptron?

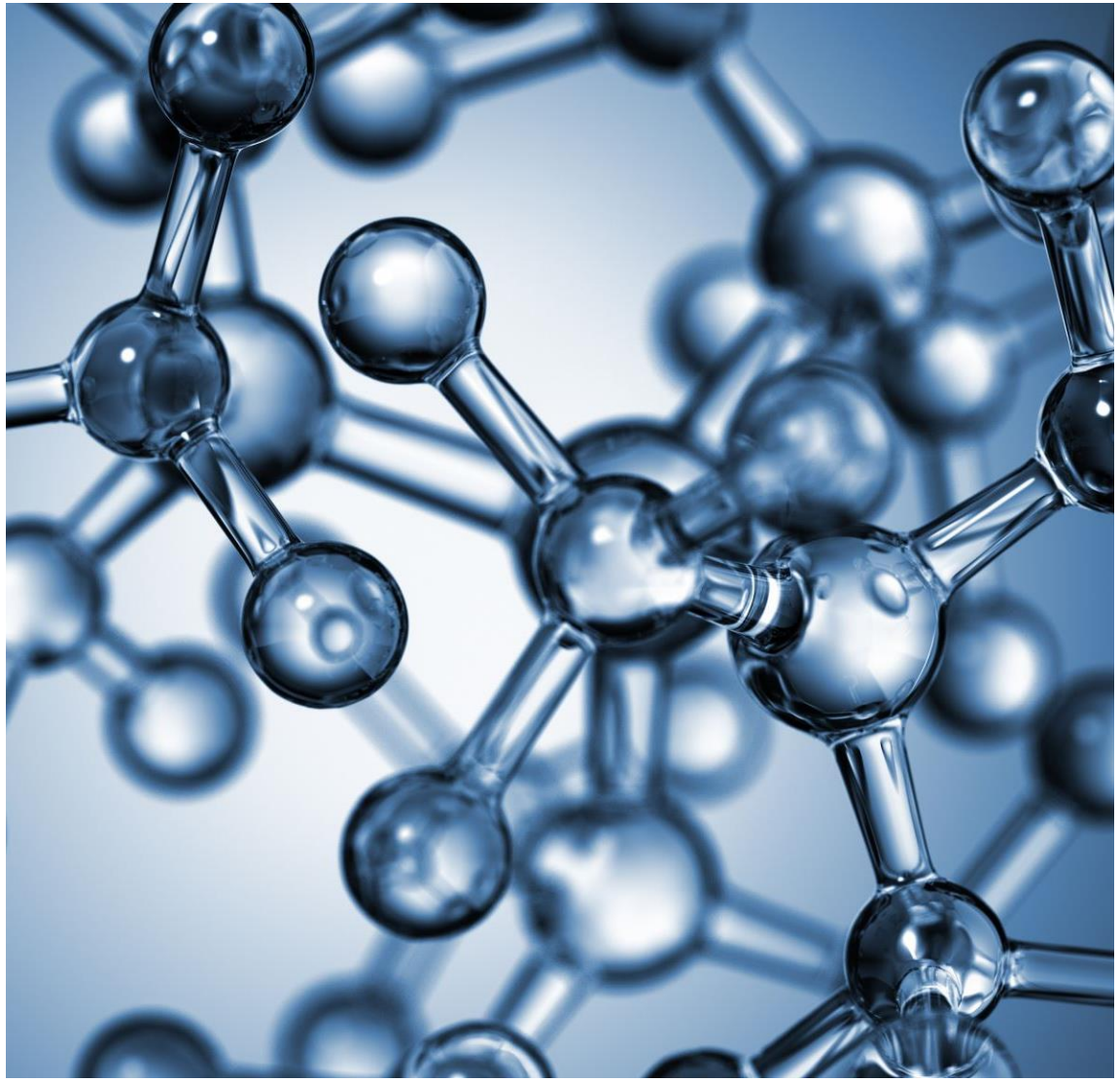
- Perceptron is one of the simplest Artificial neural network architectures. It was introduced by Frank Rosenblatt in 1957s.
- It is the simplest type of feedforward neural network, consisting of a single layer of input nodes that are fully connected to a layer of output nodes.
- It can learn the linearly separable patterns.
- It uses slightly different types of artificial neurons known as threshold logic units (TLU).
- It was first introduced by McCulloch and Walter Pitts in the 1943.



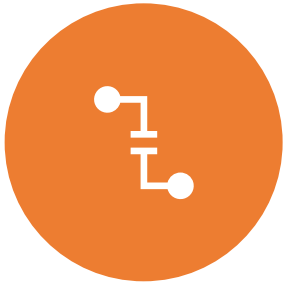
Types of Perceptron

Single-Layer Perceptron: This type of perceptron is limited to learning linearly separable patterns. effective for tasks where the data can be divided into distinct categories through a straight line.

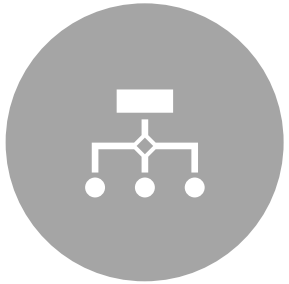
Multilayer Perceptron: Multilayer perceptrons possess enhanced processing capabilities as they consist of two or more layers, adept at handling more complex patterns and relationships within the data.



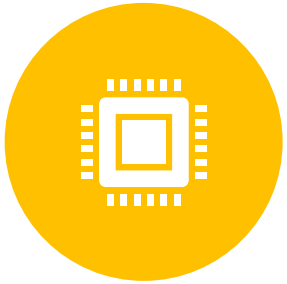
Basic Components of Perceptron



A perceptron, the basic unit of a neural network, comprises essential components that collaborate in information processing.



Input Features: The perceptron takes multiple input features, each input feature represents a characteristic or attribute of the input data.



Weights: Each input feature is associated with a weight, determining the significance of each input feature in influencing the perceptron's output. During training, these weights are adjusted to learn the optimal values.



Summation Function: The perceptron calculates the weighted sum of its inputs using the summation function. The summation function combines the inputs with their respective weights to produce a weighted sum.

Basic Components of Perceptron



Activation Function: The weighted sum is then passed through an activation function. Perceptron uses Heaviside step function functions. which take the summed values as input and compare with the threshold and provide the output as 0 or 1.



Output: The final output of the perceptron, is determined by the activation function's result. For example, in binary classification problems, the output might represent a predicted class (0 or 1).



Bias: A bias term is often included in the perceptron model. The bias allows the model to make adjustments that are independent of the input. It is an additional parameter that is learned during training.



Learning Algorithm (Weight Update Rule): During training, the perceptron learns by adjusting its weights and bias based on a learning algorithm. A common approach is the perceptron learning algorithm, which updates weights based on the difference between the predicted output and the true output.

Basic Components of Perceptron

These components work together to enable a perceptron to learn and make predictions.

While a single perceptron can perform binary classification, more complex tasks require the use of multiple perceptrons organized into layers, forming a neural network.

How does Perceptron work?

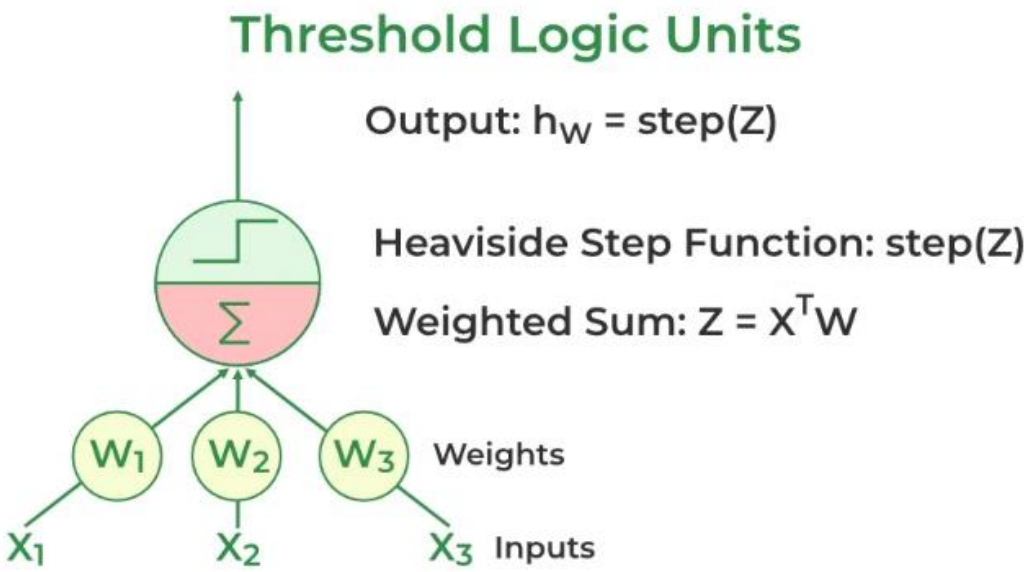
- A weight is assigned to each input node of a perceptron, indicating the significance of that input to the output.
- The perceptron's output is a weighted sum of the inputs that have been run through an activation function to decide whether or not the perceptron will fire.
- It computes the weighted sum of its inputs as:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n = \mathbf{X}^\top \mathbf{W}$$

How does Perceptron work?

$$h(z) = \begin{cases} 0 & \text{if } z < \text{Threshold} \\ 1 & \text{if } z \geq \text{Threshold} \end{cases}$$

- The step function compares this weighted sum to the threshold, which outputs 1 if the input is larger than a threshold value and 0 otherwise, is the activation function that perceptrons utilize the most frequently.
- The most common step function used in perceptron is the Heaviside step function.



How does Perceptron work?

A perceptron has a single layer of threshold logic units with each TLU connected to all inputs.

How does Perceptron work?

- When all the neurons in a layer are connected to every neuron of the previous layer, it is known as a fully connected layer or dense layer.
- The output of the fully connected layer can be:

$$f_{W,b}(X) = h(X^\top W + b)$$

- Where X is the input W is the weight for each inputs neurons and b is the bias and h is the step function.

How does Perceptron work?

- During training, both the weights and bias of the perceptron are updated using a learning rule that aims to reduce classification errors by minimizing the difference between the predicted and actual outputs.

Weight update:

$$w_i := w_i + \eta \cdot (y - \hat{y}) \cdot x_i$$

Bias update:

$$b := b + \eta \cdot (y - \hat{y})$$

- This is called as **Perceptron Learning Rule**.
- The perceptron uses a rule-based update

Example

X1	X2	X3	Y
1	0	1	1
0	1	1	0
1	1	0	1

Consider this dataset

Example

- Learning rate $\eta=0.1$
- Initial weights $w=[0,0,0]$
- Initial bias $b=0$
- Activation- Step function (output is 1 if net ≥ 0, otherwise 0)

Record 1

$$z = (1)(0) + (0)(0) + (1)(0) + 0 = 0$$

$$\hat{y} = \text{step}(z) = \text{step}(0) = 1$$

$$y = \hat{y} = 1 \Rightarrow \text{No update}$$

Record 2

$$z = (0)(0) + (1)(0) + (1)(0) + 0 = 0$$

$$\hat{y} = \text{step}(0) = 1$$

$$\Delta w_i = \eta(y - \hat{y})x_i = 0.1(-1)(x_i)$$

$$w_1 = 0 + 0.1(-1)(0) = 0$$

$$w_2 = 0 + 0.1(-1)(1) = -0.1$$

$$w_3 = 0 + 0.1(-1)(1) = -0.1$$

$$b = 0 + 0.1(-1) = -0.1$$

Record 3

$$z = (1)(0) + (1)(-0.1) + (0)(-0.1) + (-0.1) = -0.1 - 0.1 = -0.2$$

$$\hat{y} = \text{step}(-0.2) = 0$$

$$\Delta w_i = 0.1(1)(x_i)$$

$$w_1 = 0 + 0.1(1)(1) = 0.1$$

$$w_2 = -0.1 + 0.1(1)(1) = 0$$

$$w_3 = -0.1 + 0.1(1)(0) = -0.1$$

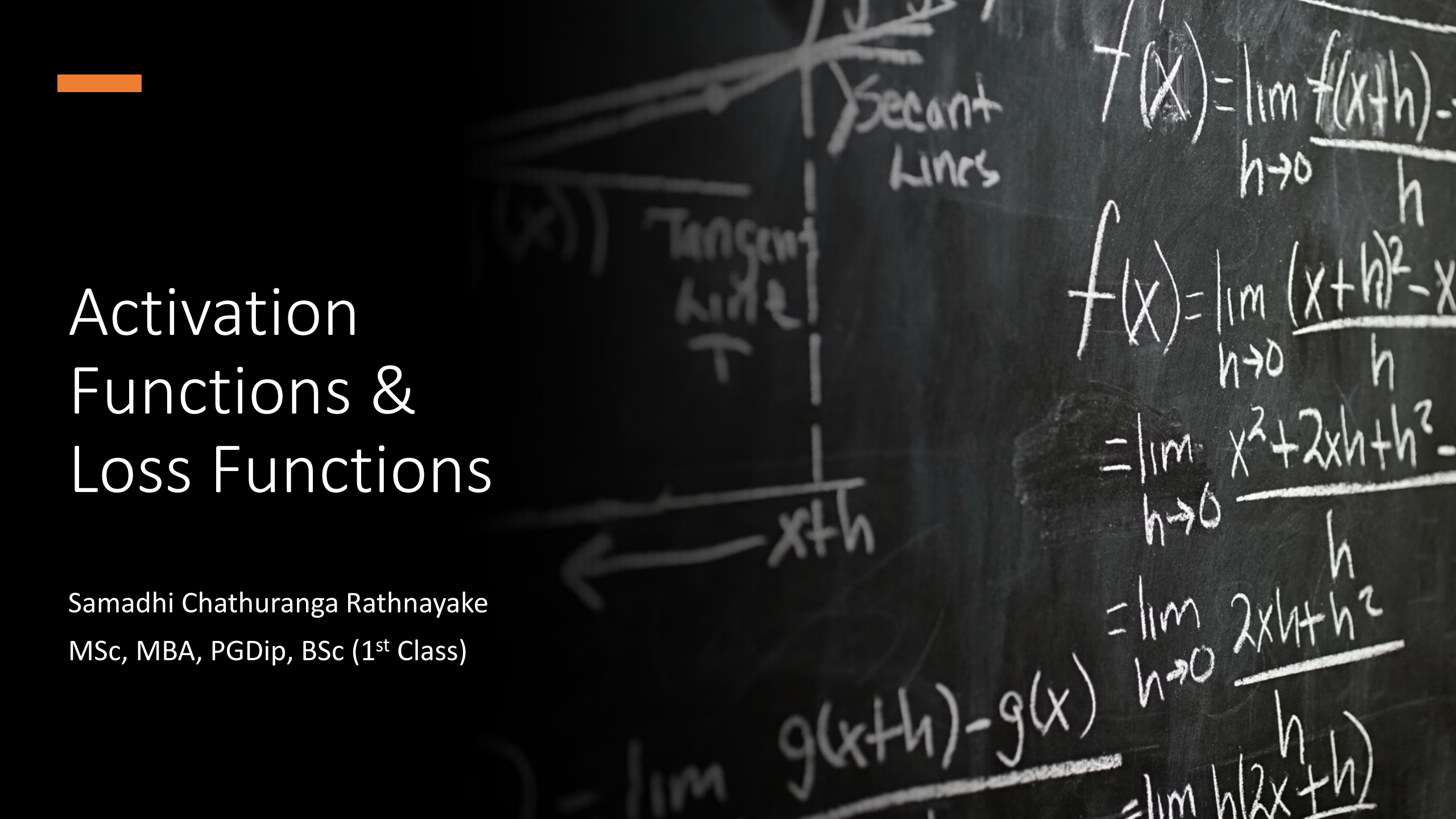
$$b = -0.1 + 0.1 = 0$$

Summary

Step	Weights	Bias
Init	[0.0, 0.0, 0.0]	0.0
R1	[0.0, 0.0, 0.0]	0.0
R2	[0.0, -0.1, -0.1]	-0.1
R3	[0.1, 0.0, -0.1]	0.0

Activation Functions & Loss Functions

Samadhi Chathuranga Rathnayake
MSc, MBA, PGDip, BSc (1st Class)

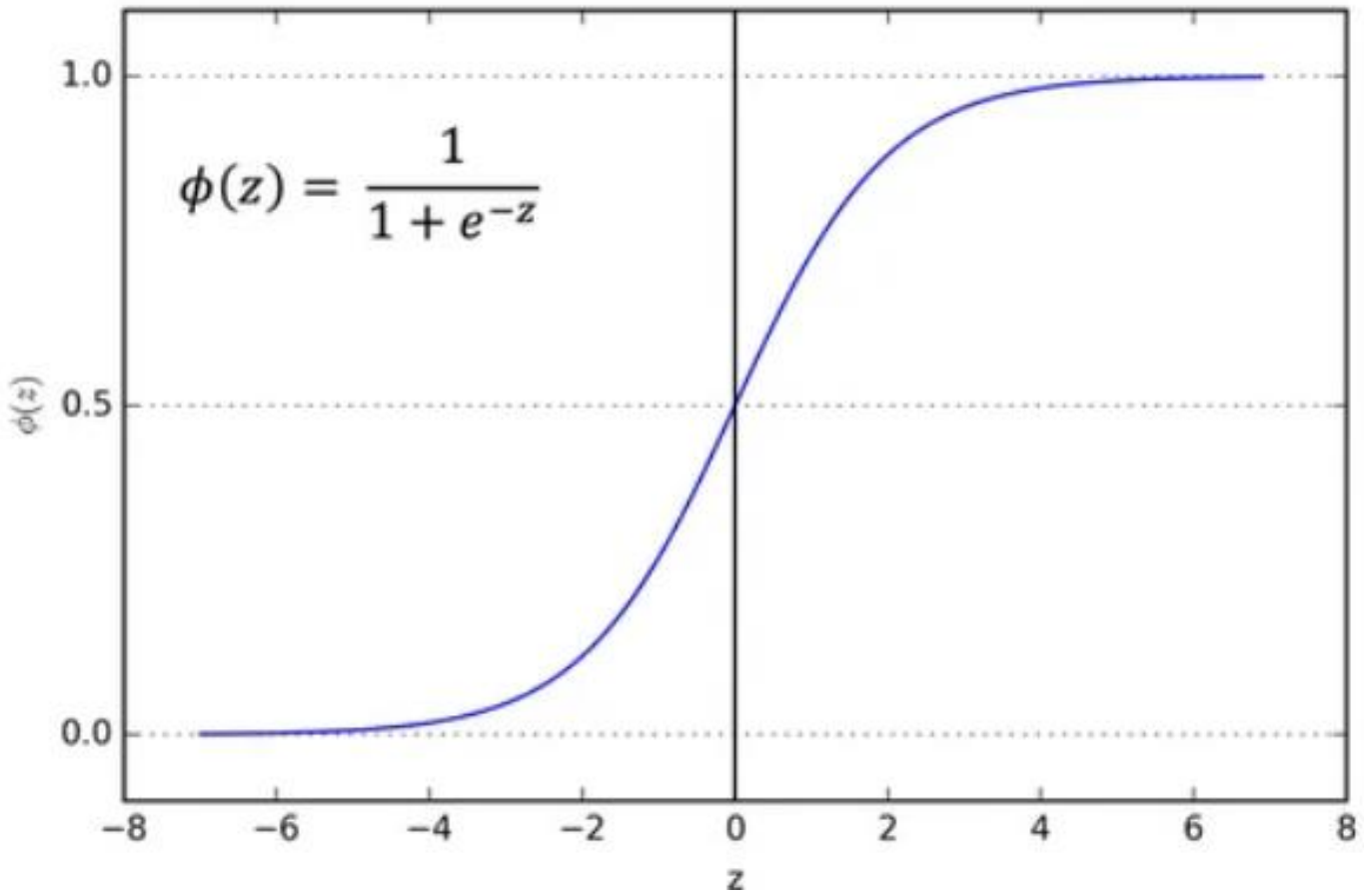


What is Activation Function?

- It’s just a thing function that you use to get the output of node.
- It is also known as Transfer Function.
- It is used to determine the output of neural network like yes or no.
- It maps the resulting values in between 0 to 1 or -1 to 1 etc. (depending upon the function).

Sigmoid or Logistic Activation Function

The **Sigmoid** Function curve looks like a S-shape.

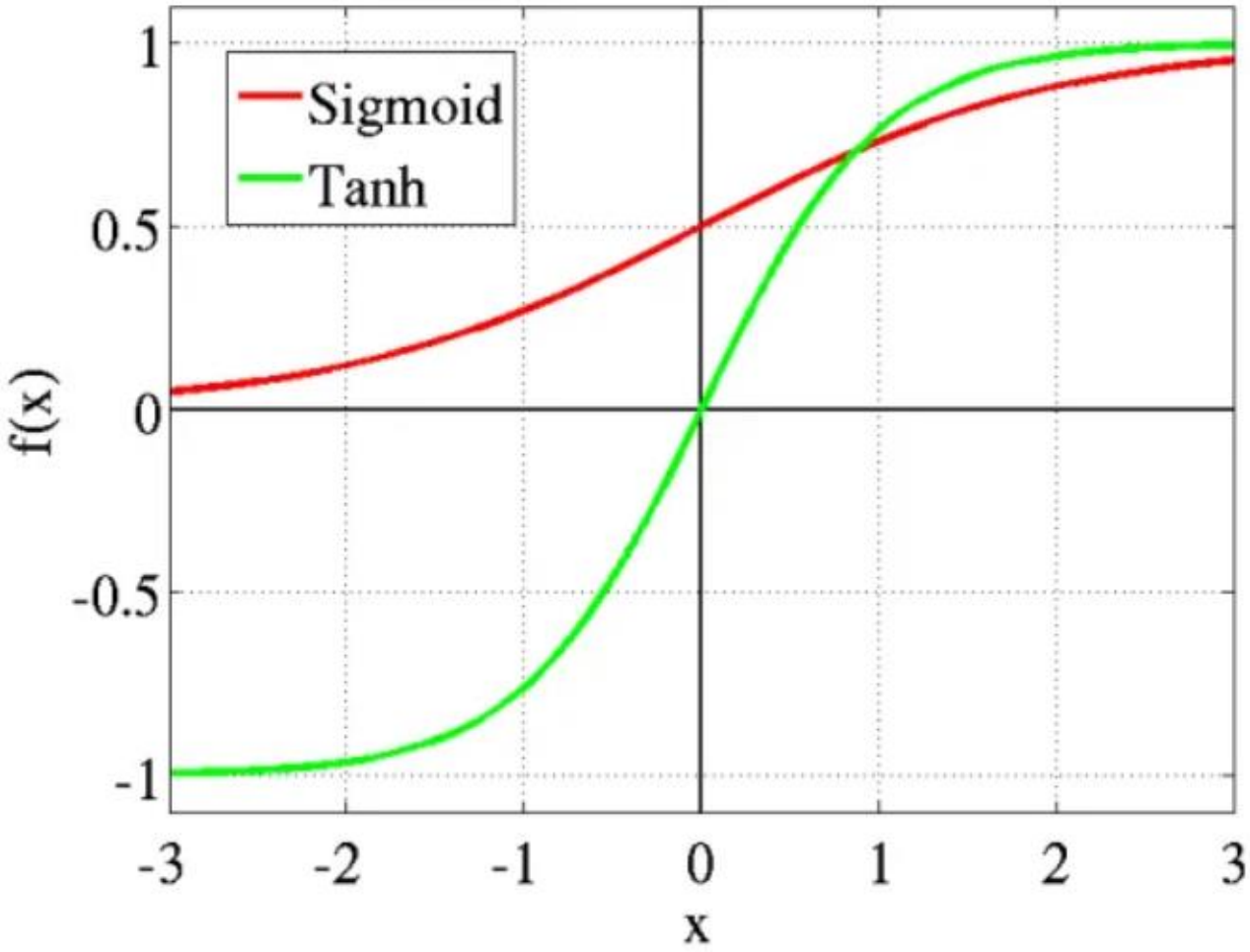


Sigmoid or Logistic Activation Function

- The main reason why we use sigmoid function is because it exists between (0 to 1).
- Therefore, it is especially used for models where we have to predict the probability as an output.
- Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice.
- The function is differentiable.
- That means, we can find the slope of the sigmoid curve at any two points.
- The function is monotonic, but function’s derivative is not.
- The logistic sigmoid function can cause a neural network to get stuck at the training time.
- The softmax function is a more generalized logistic activation function which is used for multiclass classification.

Tanh or hyperbolic tangent Activation Function

- **tanh** is also like logistic sigmoid but better.
- The range of the tanh function is from (-1 to 1).
- tanh is also sigmoidal (s - shaped).

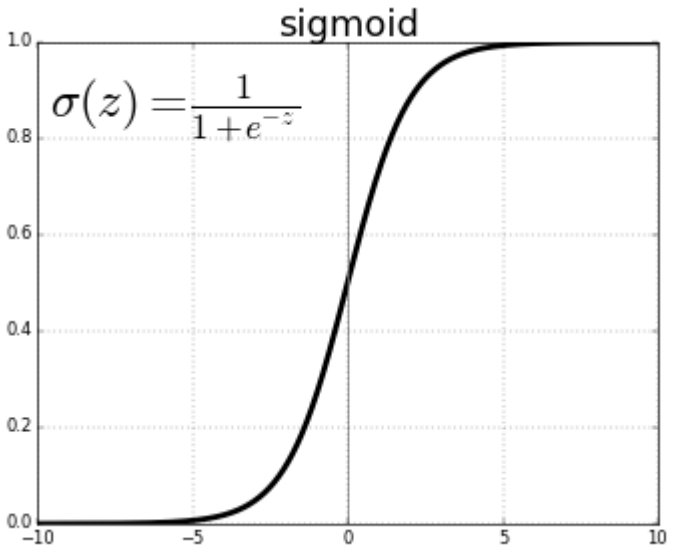


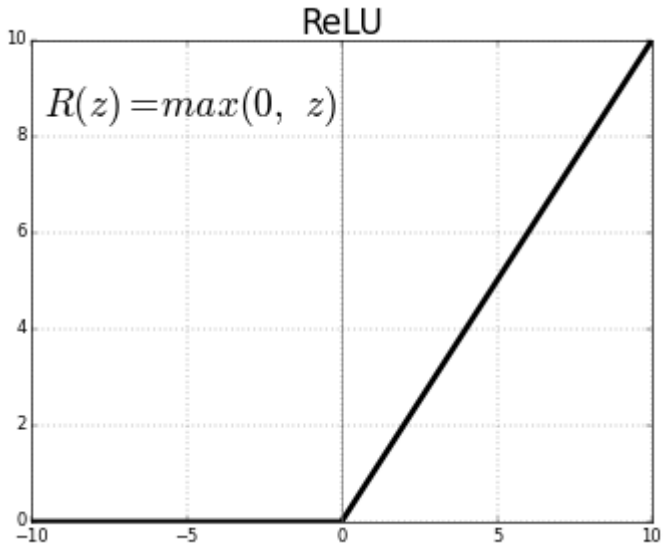
Tanh or hyperbolic tangent Activation Function

- The advantage is that the negative inputs will be mapped strongly negative, and the zero inputs will be mapped near zero in the tanh graph.
- The function is differentiable.
- The function is monotonic while its derivative is not monotonic.
- The tanh function is mainly used classification between two classes.
- Both tanh and logistic sigmoid activation functions are used in feed-forward nets.

ReLU (Rectified Linear Unit) Activation Function

- The **ReLU** is the most used activation function in the world right now.
- Since, it is used in almost all the convolutional neural networks or deep learning.





ReLU (Rectified Linear Unit) Activation Function

- As you can see, the ReLU is half rectified (from bottom). f(z) is zero when z is less than zero and f(z) is equal to z when z is above or equal to zero.
- Range: [0 to infinity)
- The function and its derivative both are monotonic.
- But the issue is that all the negative values become zero immediately which decreases the ability of the model to fit or train from the data properly.
- That means any negative input given to the ReLU activation function turns the value into zero immediately in the graph, which in turns affects the resulting graph by not mapping the negative values appropriately.

Softmax Function

- **Softmax** function is often described as a combination of multiple sigmoids.

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}} \quad \text{for } j = 1, \ldots, K.$$

Softmax Function

- We know that sigmoid returns values between 0 and 1, which can be treated as probabilities of a data point belonging to a particular class.
- Thus sigmoid is widely used for binary classification problems.
- The softmax function can be used for multiclass classification problems.
- This function returns the probability for a datapoint belonging to each individual class.

What is Loss Function?

- A loss function is a function that compares the target and predicted output values; measures how well the neural network models the training data.
- When training, we aim to minimize this loss between the predicted and target outputs.
- The hyperparameters are adjusted to minimize the average loss, we find the weights, wT, and biases, b, that minimize the value of J (average loss).

$$J(w^T, b) = \frac{1}{m} \sum_{i=1}^{m} L(\hat{y}^{(i)}, y^{(i)})$$

Types of Loss Functions

- In supervised learning, there are two main types of loss functions.
- These correlate to the 2 major types of neural networks:
  - **Regression Loss Functions** — used in regression neural networks; given an input value, the model predicts a corresponding output value (rather than pre-selected labels); Ex. Mean Squared Error, Mean Absolute Error
  - **Classification Loss Functions** — used in classification neural networks; given an input, the neural network produces a vector of probabilities of the input belonging to various pre-set categories — can then select the category with the highest probability of belonging; Ex. Binary Cross-Entropy, Categorical Cross-Entropy

Mean Squared Error (MSE)

- One of the most popular loss functions, MSE finds the average of the squared differences between the target and the predicted outputs.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2$$

- This function has numerous properties that make it especially suited for calculating loss.
- The difference is squared, which means it does not matter whether the predicted value is above or below the target value; however, values with a large error are penalized.
- MSE is also a convex function (as shown in the diagram above) with a clearly defined global minimum, this allows us to more easily utilize gradient descent optimization to set the weight values.

Mean Squared Error (MSE)

- However, one disadvantage of this loss function is that it is very sensitive to outliers; if a predicted value is significantly greater than or less than its target value, this will significantly increase the loss.

Mean Absolute Error (MAE)

- MAE finds the average of the absolute differences between the target and the predicted outputs.

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y^{(i)} - \hat{y}^{(i)}|$$

- This loss function is used as an alternative to MSE in some cases.
- As mentioned previously, MSE is highly sensitive to outliers, which can dramatically affect the loss because the distance is squared.
- MAE is used in cases when the training data has a large number of outliers to mitigate this.

Binary Cross-Entropy/Log Loss

- This is the loss function used in binary classification models, where the model takes in an input and has to classify it into one of two pre-set categories.

$$CE\ Loss = \frac{1}{n}\sum_{i=1}^{N} -\left(y_i \cdot log(p_i) + (1 - y_i) \cdot log(1 - p_i)\right)$$

- Classification neural networks work by outputting a vector of probabilities, the probability that the given input fits into each of the pre-set categories; then selecting the category with the highest probability as the final output.

Binary Cross-Entropy/Log Loss

- In binary classification, there are only two possible actual values of y — 0 or 1.
- Thus, to accurately determine loss between the actual and predicted values, it needs to compare the actual value (0 or 1) with the probability that the input aligns with that category (p(i) = probability that the category is 1; 1 — p(i) = probability that the category is 0)

Categorical Cross-Entropy Loss

- In cases where the number of classes is greater than two, we utilize categorical cross-entropy , this follows a very similar process to binary cross-entropy.

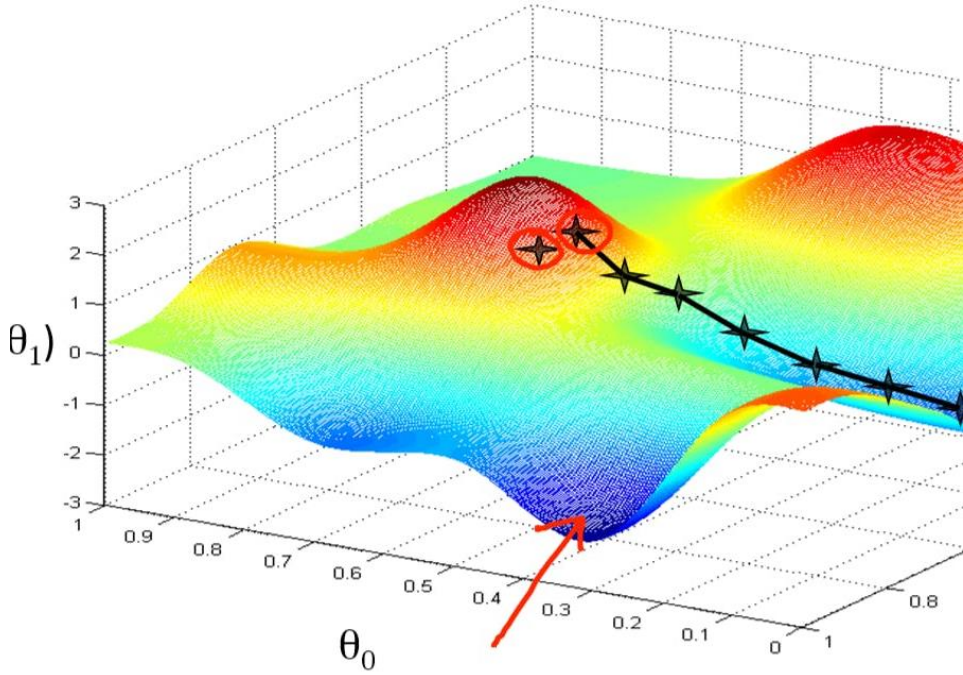
$$CE\ Loss = -\frac{1}{n}\sum_{i=1}^{N}\sum_{j=1}^{M} y_{ij} \cdot log(p_{ij})$$

- Binary cross-entropy is a special case of categorical cross-entropy, where M = 2, the number of categories is 2.

Custom Loss Functions

- Kullback-Leibler (KL) divergence loss
- Mean Absolute Error (MAE)
- Mean Absolute Percentage Error (MAPE)
- Mean Squared Error (MSE)
- Mean Squared Logarithmic Error (MSLE)
- Binary Crossentropy Loss
- Binary Focal Crossentropy Loss
- Sparse Categorical Crossentropy Loss
- Categorical Hinge Loss
- Hinge Loss
- Cosine Similarity
- Logcosh
- Huber loss
- Poisson loss





Gradient Descent Algorithm

Samadhi Chathuranga Rathnayake

MSc, MBA, PGDip, BSc (1st Class)

Gradient Decent

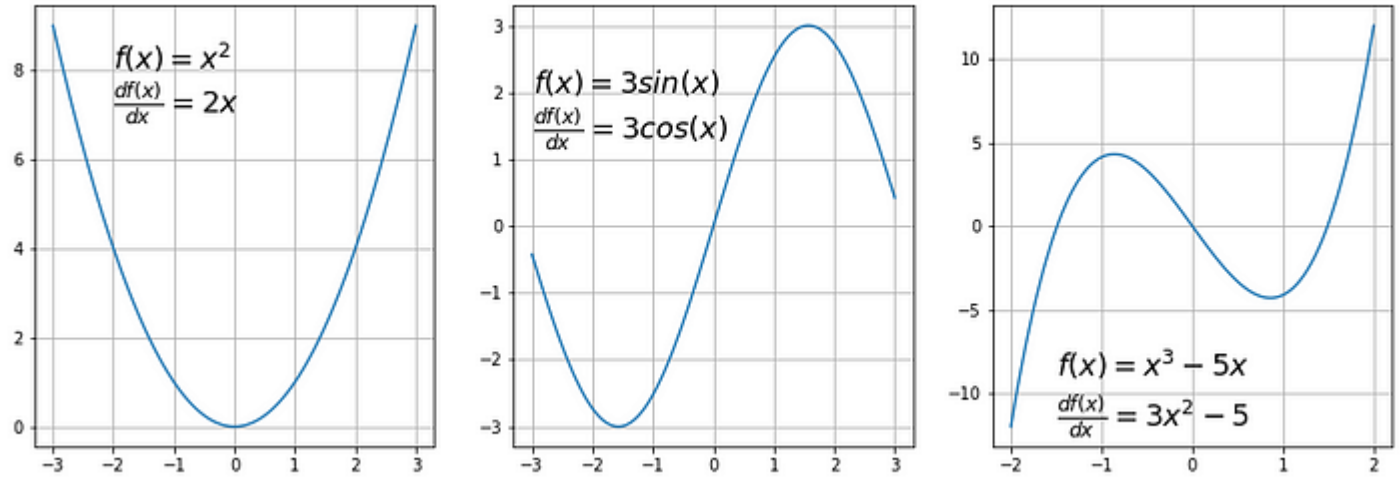
- Gradient descent (GD) is an iterative first-order optimisation algorithm, used to find a local minimum/maximum of a given function.
- This method is commonly used in machine learning (ML) and deep learning (DL) to minimise a cost/loss function.
- This method was proposed long before the era of modern computers by Augustin-Louis Cauchy in 1847.
- Since that time, there was an significant development in computer science and numerical methods.
- That led to numerous improved versions of Gradient Descent.

Function Requirements

- This does not work for all functions.
- There are two specific requirements. A function has to be:
  - Differentiable
  - Convex

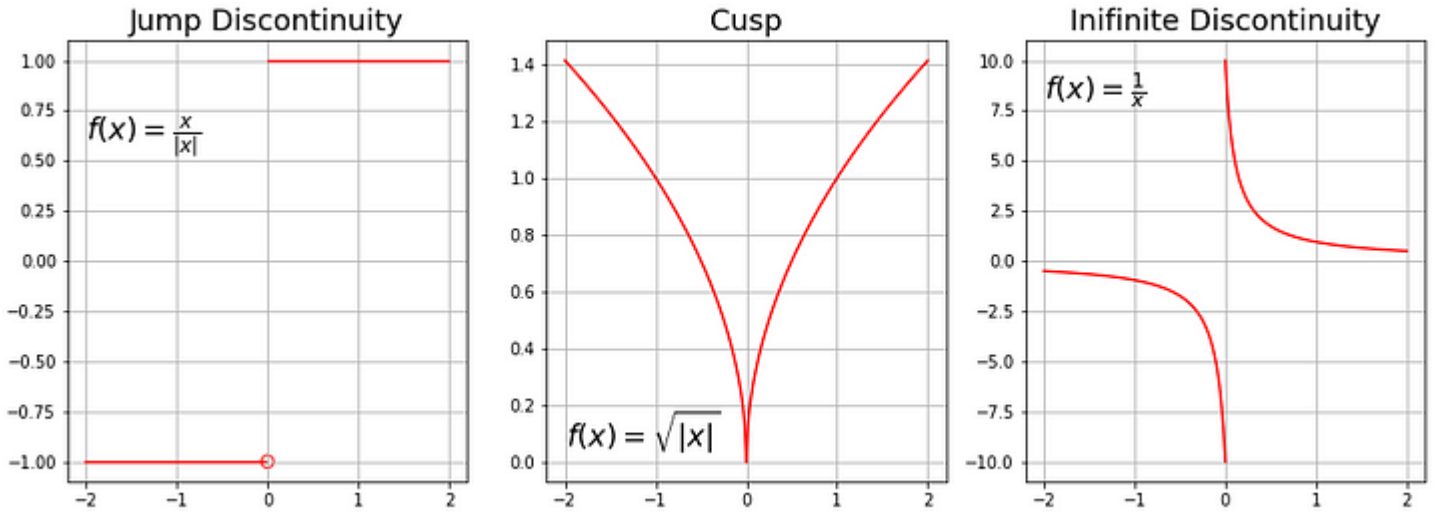
Differentiable

- If a function is differentiable it has a derivative for each point in its domain, not all functions meet these criteria.



Differentiable

- The non-differentiable functions have a step a cusp or a discontinuity:



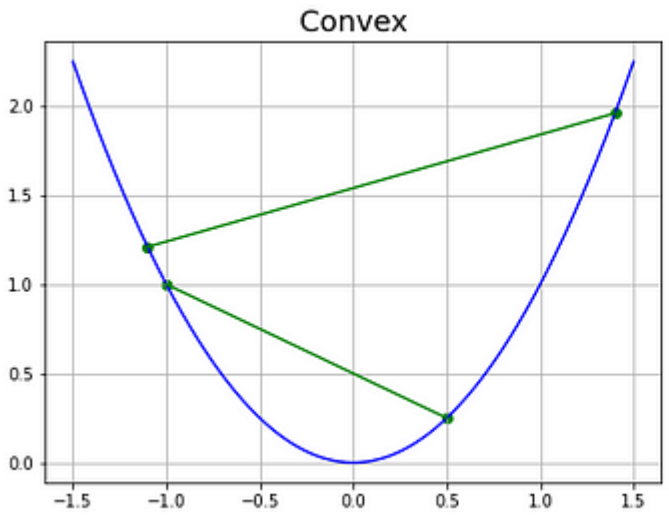
Convex

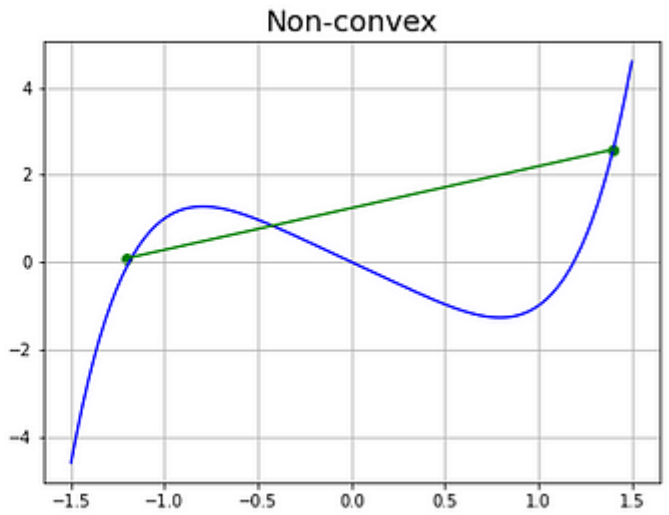
- For a univariate function, convex means that the line segment connecting two function's points lays on or above its curve (it does not cross it).
- If it does it means that it has a local minimum which is not a global one.
- Mathematically, for two points x_1, x_2 laying on the function's curve this condition is expressed as:

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda) f(x_2)$$

Convex

- where λ denotes a point's location on a section line and its value has to be between 0 (left point) and 1 (right point), e.g. λ=0.5 means a location in the middle.
- Below there are two functions with exemplary section lines.





Convex

- Another way to check mathematically if a univariate function is convex is to calculate the second derivative and check if its value is always bigger than 0.

$$\frac{d^2 f(x)}{dx^2} > 0$$

Convex

- Another way to check mathematically if a univariate function is convex is to calculate the second derivative and check if its value is always bigger than 0.

$$\frac{d^2 f(x)}{dx^2} > 0$$

- Ex- $f(x) = x^2 - x + 3$ $\frac{df(x)}{dx} = 2x - 1,$ $\frac{d^2 f(x)}{dx^2} = 2$
- Because the second derivative is always bigger than 0, our function is strictly convex.

What is Gradient?

- Intuitively it is a slope of a curve at a given point in a specified direction.
- In the case of a univariate function, it is simply the first derivative at a selected point.
- In the case of a multivariate function, it is a vector of derivatives in each main direction (along variable axes).
- Because we are interested only in a slope along one axis and we don’t care about others these derivatives are called partial derivatives.

What is Gradient?

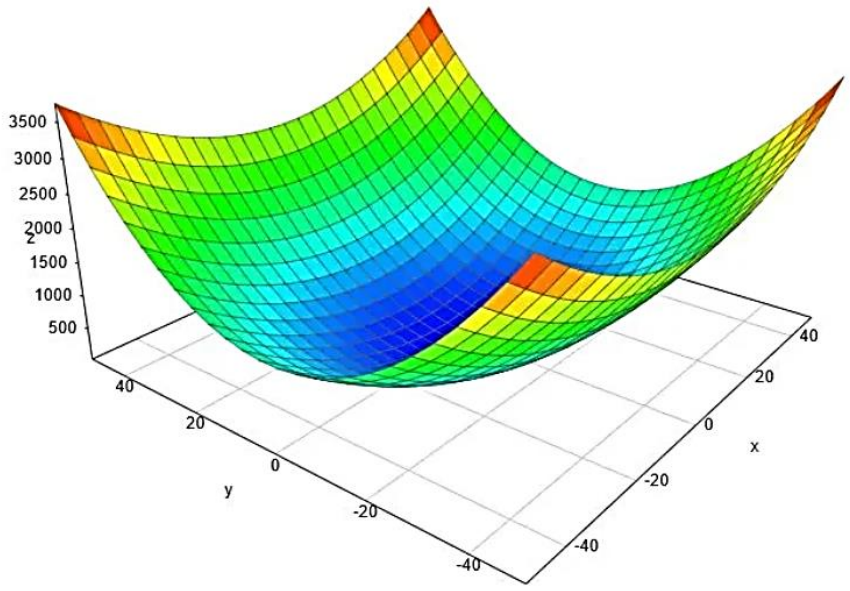
- A gradient for an n-dimensional function f(x) at a given point p is defined as follows:

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

What is Gradient?

Ex- Consider the following function

$$f(x) = 0.5x^2 + y^2$$



What is Gradient?

$$\nabla f(x, y) = \begin{bmatrix} x \\ 2y \end{bmatrix}$$

$$\nabla f(10, 10) = \begin{bmatrix} 10 \\ 20 \end{bmatrix}$$

Ex- Let's assume we are interested in a gradient at point p(10,10):

$$\frac{\partial f(x, y)}{\partial x} = x, \quad \frac{\partial f(x, y)}{\partial y} = 2y$$

Gradient Descent Algorithm (GD)

- Gradient Descent Algorithm iteratively calculates the next point using gradient at the current position, scales it (by a learning rate) and subtracts obtained value from the current position (makes a step).
- It subtracts the value because we want to minimize the function (to maximize it would be adding).
- This process can be written as:

$$p_{n+1} = p_n - \eta \nabla f(p_n)$$

Gradient Descent Algorithm

- **η** is the learning rate which scales the gradient and thus controls the step size.
- The smaller learning rate the longer GD converges, or may reach maximum iteration before reaching the optimum point
- If learning rate is too big the algorithm may not converge to the optimal point (jump around) or even to diverge completely.

Gradient Descent Algorithm

- Steps of GD
  1. Choose a starting point (initialization)
  2. Calculate gradient at this point
  3. Make a scaled step in the opposite direction to the gradient (objective: minimize)
  4. Repeat points 2 and 3 until one of the criteria is met:
  5. Maximum number of iterations reached
  6. Step size is smaller than the tolerance (due to scaling or a small gradient).

Gradient Descent Algorithm

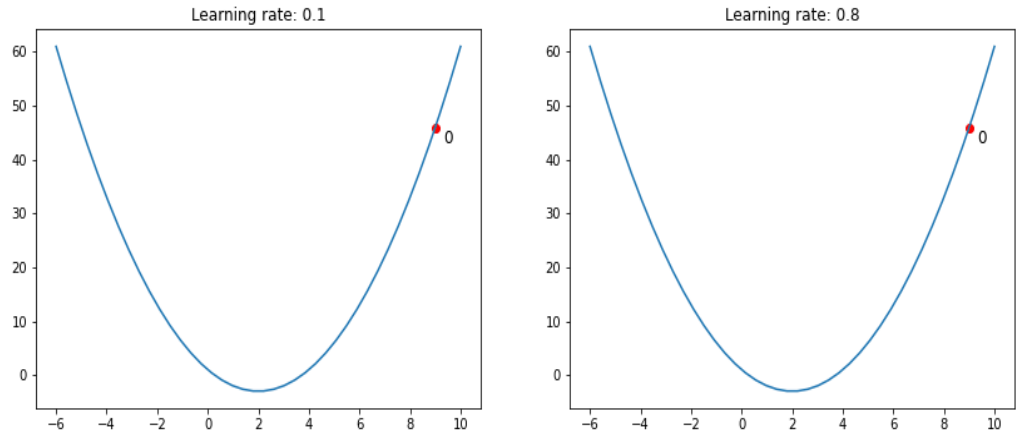
Ex- $f(x) = x^2 - 4x + 1$

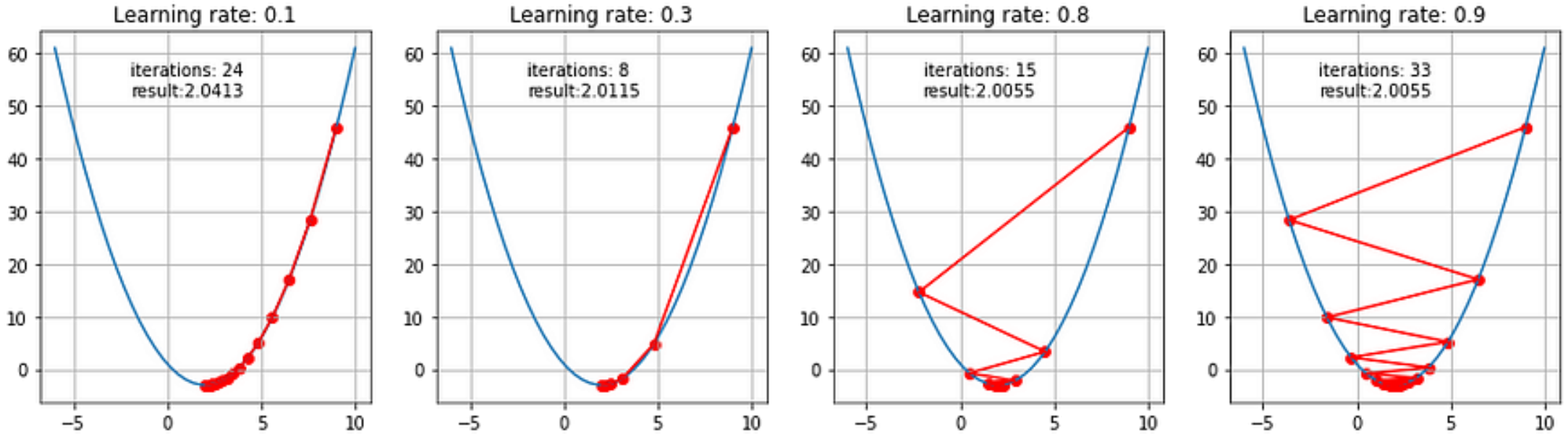
$$\frac{df(x)}{dx} = 2x - 4$$

$$x_1 = 9 - 0.1 \cdot (2 \cdot 9 - 4) = 7.6$$
$$x_2 = 7.6 - 0.1 \cdot (2 \cdot 7.6 - 4) = 6.48$$
$$x_3 = 6.48 - 0.1 \cdot (2 \cdot 6.48 - 4) = 5.584$$

Gradient Descent Algorithm

Ex-





Gradient Descent Algorithm

Examples

Minimizing Loss Function with GD

The loss function

$$J(w,b) = \frac{1}{n}(y_p - y)^2$$

Gradient of J(w,b) with respect to w

$$\begin{aligned}
J'_w &= \frac{\partial J(w,b)}{\partial w} \\
&= \frac{\partial}{\partial w}\left[\frac{1}{n}(y_p - y)^2\right] \\
&= \frac{2(y_p - y)}{n}\frac{\partial}{\partial w}\left[(y_p - y)\right] \\
&= \frac{2(y_p - y)}{n}\frac{\partial}{\partial w}\left[((xW^T + b) - y)\right] \\
&= \frac{2(y_p - y)}{n}\left[\frac{\partial(xW^T + b)}{\partial w} - \frac{\partial(y)}{\partial w}\right] \\
&= \frac{2(y_p - y)}{n}\left[x - 0\right] \\
&= \frac{1}{n}(y_p - y)[2x]
\end{aligned}$$

Minimizing Loss Function with GD

The loss function

$$J(w,b) = \frac{1}{n}(y_p - y)^2$$

Gradient of J(w,b) with respect to b

$$\begin{aligned}
J'_b &= \frac{\partial J(w,b)}{\partial b} \\
&= \frac{\partial}{\partial b}\left[\frac{1}{n}(y_p - y)^2\right] \\
&= \frac{2(y_p - y)}{n}\frac{\partial}{\partial b}\left[(y_p - y)\right] \\
&= \frac{2(y_p - y)}{n}\frac{\partial}{\partial b}\left[((xW^T + b) - y)\right] \\
&= \frac{2(y_p - y)}{n}\left[\frac{\partial(xW^T + b)}{\partial b} - \frac{\partial(y)}{\partial b}\right] \\
&= \frac{2(y_p - y)}{n}\left[1 - 0\right] \\
&= \frac{1}{n}(y_p - y)[2]
\end{aligned}$$

Minimizing Loss Function with GD

- The GD Algorithm is,

$$Param = Param - \gamma \nabla J$$

- So, the weights and bias values will be updated as,

$$w = w - \gamma \nabla J(w, , b)$$
$$b = b - \gamma \nabla J(w, , b)$$

Gradient Descent Learning Rate

- The learning rate is a critical hyperparameter in the context of gradient descent, influencing the size of steps taken during the optimization process to update the model parameters.
- Choosing an appropriate learning rate is crucial for efficient and effective model training.
- When the learning rate is too small, the optimization process progresses very slowly.
- The model makes tiny updates to its parameters in each iteration, leading to sluggish convergence and potentially getting stuck in local minima.
- On the other hand, an excessively large learning rate can cause the optimization algorithm to overshoot the optimal parameter values, leading to divergence or oscillations that hinder convergence.
- Achieving the right balance is essential.

Advantages of GD

- **Widely used**: Gradient descent and its variants are widely used in machine learning and optimization problems because they are effective and easy to implement.
- **Convergence**: Gradient descent and its variants can converge to a global minimum or a good local minimum of the cost function, depending on the problem and the variant used.
- **Scalability**: Many variants of gradient descent can be parallelized and are scalable to large datasets and high-dimensional models.
- **Flexibility**: Different variants of gradient descent offer a range of trade-offs between accuracy and speed, and can be adjusted to optimize the performance of a specific problem.

Disadvantages of GD

- **Choice of learning rate**: The choice of learning rate is crucial for the convergence of gradient descent and its variants. Choosing a learning rate that is too large can lead to oscillations or overshooting while choosing a learning rate that is too small can lead to slow convergence or getting stuck in local minima.
- **Sensitivity to initialization**: Gradient descent and its variants can be sensitive to the initialization of the model's parameters, which can affect the convergence and the quality of the solution.
- **Time-consuming**: Gradient descent and its variants can be time-consuming, especially when dealing with large datasets and high-dimensional models. The convergence speed can also vary depending on the variant used and the specific problem.
- **Local optima**: Gradient descent and its variants can converge to a local minimum instead of the global minimum of the cost function, especially in non-convex problems. This can affect the quality of the solution, and techniques like random initialization and multiple restarts may be used to mitigate this issue.

Different Variants of Gradient Descent

- There are several variants of gradient descent that differ in the way the step size or learning rate is chosen and the way the updates are made. Here are some popular variants.
  - Batch Gradient Descent
  - Mini-batch Gradient Descent
  - Stochastic Gradient Descent (SGD)
  - Momentum-based Gradient Descent
  - Nesterov Accelerated Gradient (NAG)

Batch Gradient Descent

- In batch gradient descent, to update the model parameter values like weight and bias, the entire training dataset is used to compute the gradient and update the parameters at each iteration.
- This can be slow for large datasets but may lead to a more accurate model.
- It is effective for convex or relatively smooth error manifolds because it moves directly toward an optimal solution by taking a large step in the direction of the negative gradient of the cost function.
- However, it can be slow for large datasets because it computes the gradient and updates the parameters using the entire training dataset at each iteration.
- This can result in longer training times and higher computational costs.

Mini-batch Gradient Descent

- In Mini-batch gradient descent a small batch of training examples is used to compute the gradient and update the parameters at each iteration.
- This can be a good compromise between batch gradient descent and Stochastic Gradient Descent, as it can be faster than batch gradient descent and less noisy than Stochastic Gradient Descent.

Ex-

X_1	X_2	X_3	Y (Target)
1	2	3	10
2	1	0	8
3	2	1	12
0	1	2	6

- Consider the following regression model

$$\hat{Y} = w_1 X_1 + w_2 X_2 + w_3 X_3 + b$$

- In the Batch GD, the model uses all 4 records to calculate the error and update the weights.
- Start with initial weights: $w1$=$w2$=$w3$=0, b=0
- Consider the learning rate as 0.01

Ex-

- For each iteration:
  - Predict predictions for all 4 records.
  - Calculate the total loss (Ex- Mean Squared Error).
  - Compute gradient (how much to change weights) using all records.
  - Update the weights and bias once using those gradients.

Ex-

- Predictions for all 4 records

$$\hat{y}_i = 0 \quad \text{for all } i$$

- Loss (MSE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$= \frac{1}{4}[(10-0)^2 + (8-0)^2 + (12-0)^2 + (6-0)^2] = \frac{1}{4}[100 + 64 + 144 + 36] = \frac{344}{4} = 86$$

- Compute Gradients

$$\frac{\partial \text{MSE}}{\partial w_j} = -\frac{2}{n}\sum_{i=1}^{n} x_{ij}(y_i - \hat{y}_i) \qquad \frac{\partial \text{MSE}}{\partial w_j} = -\frac{2}{4}\sum x_{ij} y_i$$

Ex-

- Compute gradients for each weight:

w_1:

$$= -0.5[(1)(10) + (2)(8) + (3)(12) + (0)(6)] = -0.5[10 + 16 + 36 + 0] = -0.5 \times 62 = -31$$

w_2:

$$= -0.5[(2)(10) + (1)(8) + (2)(12) + (1)(6)] = -0.5[20 + 8 + 24 + 6] = -0.5 \times 58 = -29$$

w_3:

$$= -0.5[(3)(10) + (0)(8) + (1)(12) + (2)(6)] = -0.5[30 + 0 + 12 + 12] = -0.5 \times 54 = -27$$

- Bias:

$$\frac{\partial \text{MSE}}{\partial b} = -\frac{2}{n}\sum(y_i - \hat{y}_i) = -0.5(10 + 8 + 12 + 6) = -0.5 \times 36 = -18$$

Ex-

- Update Weights

$$w_j = w_j - \eta \cdot \frac{\partial \text{MSE}}{\partial w_j}$$

$$w_1 = 0 - 0.01 \cdot (-31) = 0.31 \quad w_2 = 0.29 \quad w_3 = 0.27 \quad b = 0.18$$

Ex-

- In Mini Batch GD, the model uses only 2 records at a time to learn.
- Mini Batch 1-

X_1	X_2	X_3	Y
1	2	3	10
2	1	0	8

- Predict Y for these 2 and compute gradients and update weights/bias.
- Mini Batch 2-

X_1	X_2	X_3	Y
3	2	1	12
0	1	2	6

- Do the same.

Stochastic Gradient Descent (SGD)

- Stochastic Gradient Descent (SGD) is a variant of the Gradient Descent algorithm that is used for optimizing machine learning models.
- It addresses the computational inefficiency of traditional Gradient Descent methods when dealing with large datasets in machine learning projects.
- In SGD, instead of using the entire dataset for each iteration, only a single random training example (or a small batch) is selected to calculate the gradient and update the model parameters.
- This random selection introduces randomness into the optimization process, hence the term “stochastic” in stochastic Gradient Descent.

Stochastic Gradient Descent (SGD)

- Stochastic Gradient Descent (SGD) is a version of gradient descent that updates the model using just one training example at a time,

$$w_{t+1} = w_t - \eta \cdot \nabla L(w_t; x^{(i)}, y^{(i)})$$

w_t = current weights

η = learning rate

$\nabla L(w_t; x^{(i)}, y^{(i)})$ = gradient of the loss using **only one** sample $(x^{(i)}, y^{(i)})$

Stochastic Gradient Descent (SGD)

- Ex-

X_1	X_2	X_3	Y
1	2	3	10
2	1	0	8
3	2	1	12
0	1	2	6

- In SGD,
- Pick one row, say (2, 1, 0) → Y=8
- Compute the gradient from this one row
- Update weights immediately
- Move to another random row and repeat

Stochastic Gradient Descent (SGD)

- Simply
  - At each step, randomly pick one training sample.
  - Compute the gradient using just that one sample.
  - Immediately update the model parameters.
  - Repeat for every sample, then continue for more epochs.
- In SGD, since only one sample from the dataset is chosen at random for each iteration, the path taken by the algorithm to reach the minima is usually noisier than your typical Gradient Descent algorithm.
- But that doesn’t matter all that much because the path taken by the algorithm does not matter, as long as we reach the minimum and with a significantly shorter training time.

Ex-

X_1	X_2	X_3	Y (Target)
1	2	3	10
2	1	0	8
3	2	1	12
0	1	2	6

- Consider the following regression model

$$\hat{Y} = w_1 X_1 + w_2 X_2 + w_3 X_3 + b$$

- In the Batch GD, the model uses all 4 records to calculate the error and update the weights.
- Start with initial weights: $w1$=$w2$=$w3$=0, b=0
- Consider the learning rate as 0.01

Ex-

- Record 1: (1, 2, 3) → Y = 10

$$\hat{y} = 0 \quad \text{Error} = y - \hat{y} = 10 - 0 = 10$$

$$\frac{\partial L}{\partial w_1} = -2 \cdot 1 \cdot 10 = -20 \quad \frac{\partial L}{\partial w_2} = -2 \cdot 2 \cdot 10 = -40 \quad \frac{\partial L}{\partial w_3} = -2 \cdot 3 \cdot 10 = -60 \quad \frac{\partial L}{\partial b} = -2 \cdot 10 = -20$$

$$w_1 = 0 + 0.01 \cdot 20 = 0.20 \quad w_2 = 0.40 \quad w_3 = 0.60 \quad b = 0.20$$

Ex-

- Record 2: (2, 1, 0) → Y = 8

$$\hat{y} = (0.20)(2) + (0.40)(1) + (0.60)(0) + 0.20 = 0.4 + 0.4 + 0 + 0.2 = 1.0 \quad \text{Error} = 8 - 1 = 7$$

$$\frac{\partial L}{\partial w_1} = -2 \cdot 2 \cdot 7 = -28 \quad \frac{\partial L}{\partial w_2} = -2 \cdot 1 \cdot 7 = -14 \quad \frac{\partial L}{\partial w_3} = 0 \quad \frac{\partial L}{\partial b} = -2 \cdot 7 = -14$$

$$w_1 = 0.20 + 0.01 \cdot 28 = 0.48 \quad w_2 = 0.40 + 0.14 = 0.54 \quad w_3 = 0.60(unchanged) \quad b = 0.20 + 0.14 = 0.34$$

- Continue the same process.

Comparing Main GD Methods

Method	Data Used per Update	Speed	Stability	Randomness	Good For
Batch GD	Whole dataset	Slow	Very stable	Low	Small datasets
Mini-batch GD	Small batches	Medium	Balanced	Medium	All datasets
Stochastic GD	One data point	Fast	Noisy	High	Large datasets, real-time training

SGD Algorithm

1. Initialization: Randomly initialize the parameters of the model.
2. Set Parameters: Determine the number of iterations and the learning rate (alpha) for updating the parameters.
3. Stochastic Gradient Descent Loop: Repeat the following steps until the model converges or reaches the maximum number of iterations:
    1. Shuffle the training dataset to introduce randomness.
    2. Iterate over each training example (or a small batch) in the shuffled order.
    3. Compute the gradient of the cost function with respect to the model parameters using the current training
    4. example (or batch).
    5. Update the model parameters by taking a step in the direction of the negative gradient, scaled by the learning rate.
    6. Evaluate the convergence criteria, such as the difference in the cost function between iterations of the gradient.
4. Return Optimized Parameters: Once the convergence criteria are met or the maximum number of iterations is reached, return the optimized model parameters.

Advantages of SGD

- **Speed**: SGD is faster than other variants of Gradient Descent such as Batch Gradient Descent and Mini-Batch Gradient Descent since it uses only one example to update the parameters.
- **Memory Efficiency**: Since SGD updates the parameters for each training example one at a time, it is memory-efficient and can handle large datasets that cannot fit into memory.
- **Avoidance of Local Minima**: Due to the noisy updates in SGD, it has the ability to escape from local minima and converges to a global minimum.

Disadvantages of SGD

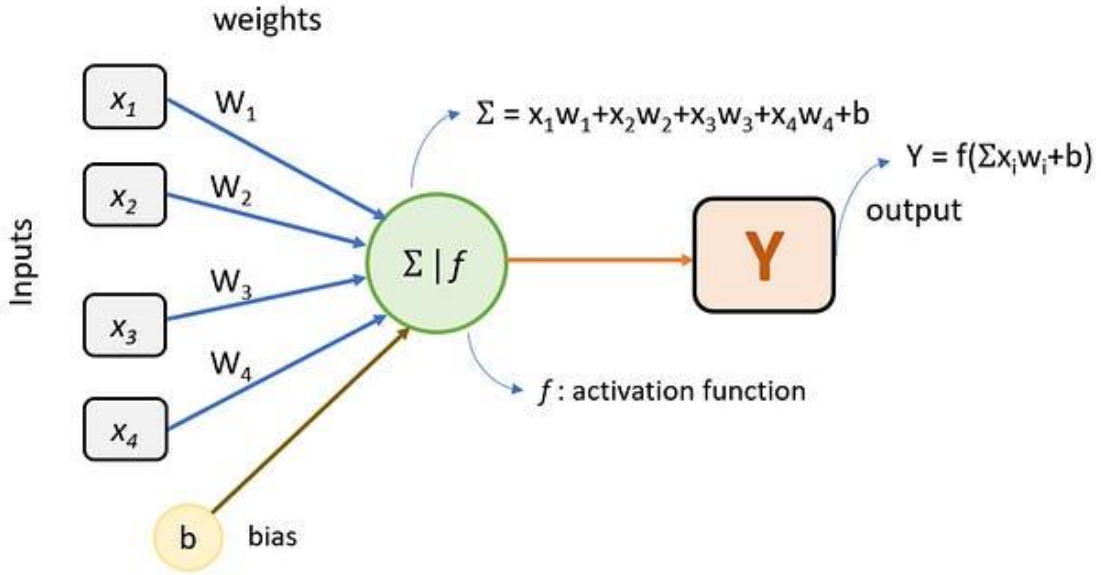
- **Noisy updates**: The updates in SGD are noisy and have a high variance, which can make the optimization process less stable and lead to oscillations around the minimum.
- **Slow Convergence**: SGD may require more iterations to converge to the minimum since it updates the parameters for each training example one at a time.
- **Sensitivity to Learning Rate**: The choice of learning rate can be critical in SGD since using a high learning rate can cause the algorithm to overshoot the minimum, while a low learning rate can make the algorithm converge slowly.

Backpropagation in Neural Network

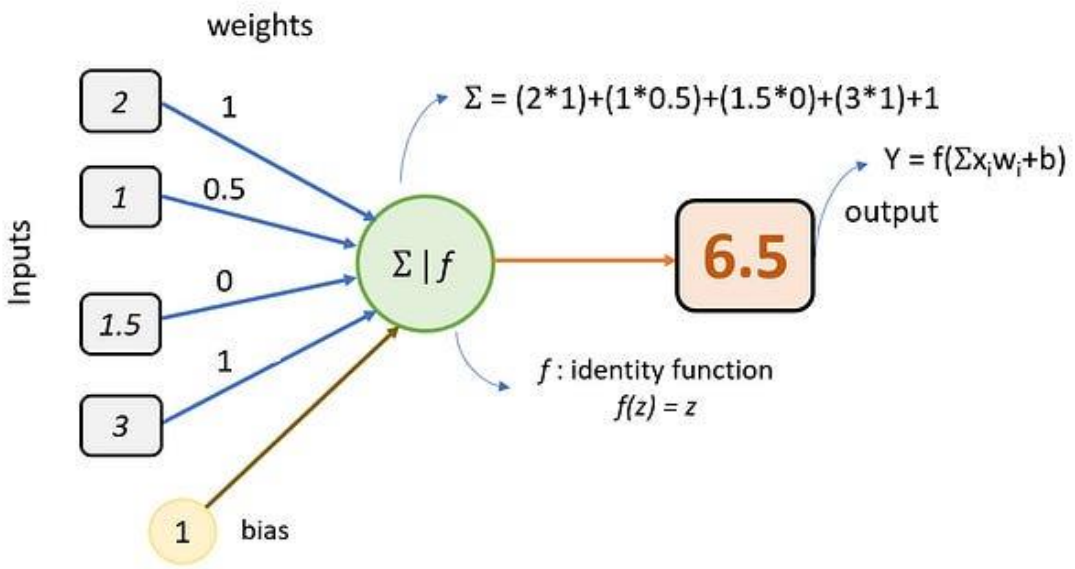
Samadhi Chathuranga Rathnayake

MSc, MBA, PGDip, BSc (1st Class)

Single Layer Perceptron



Single Layered Perceptron

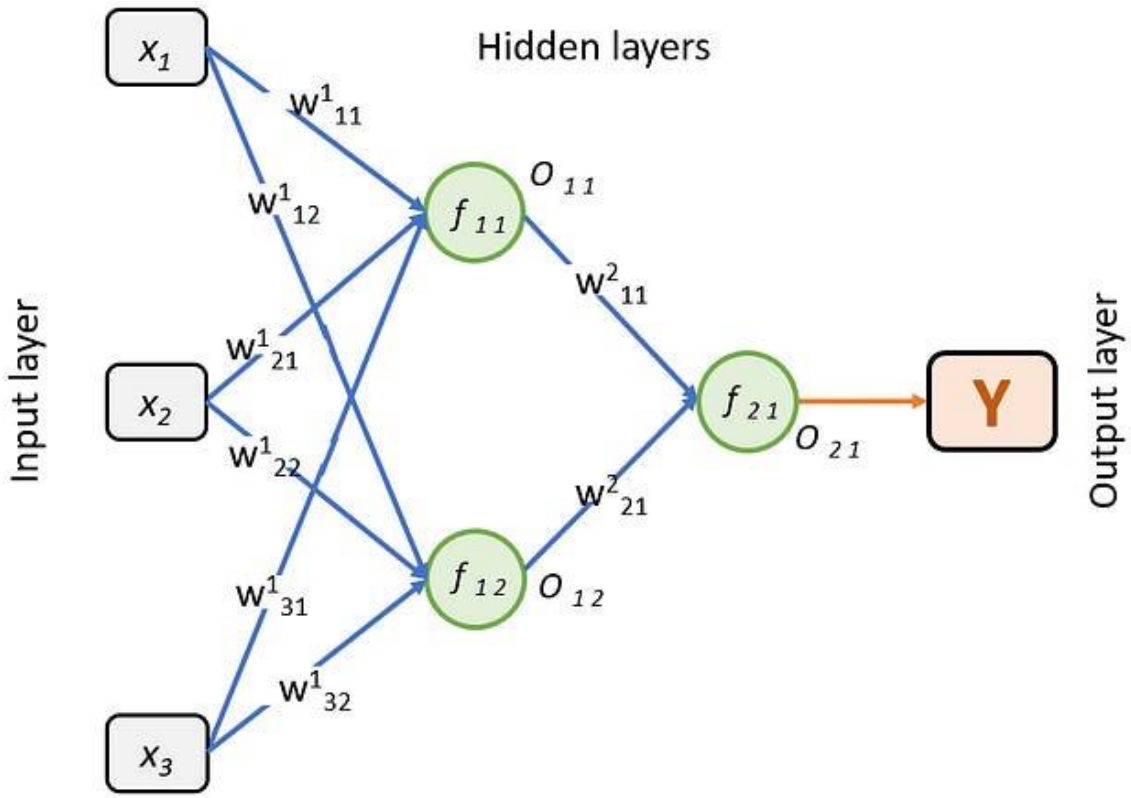


Example

Multi Layer Perceptron

- A Multi-Layered Perceptron NN can have n-number of hidden layers between input and output layer.
- These hidden layer can have n-number of neurons, in which the first hidden layer takes input from input layer and process them using activation function and pass them to next hidden layers until output layer.
- Every neuron in a hidden layer uses a non-linear activation function.

Multi Layer Perceptron



here,

x - inputs

w^n_{ij} - n is num of next hidden layer,

i is neuron num of previous layer,

j is neuron num of next layer

(w^2_{21}- n is next layer,

i is 2nd neuron of previous layer,

j is 1st neuron of next layer)

f_{ij} - i is num of layer and j is num of neuron

O_{ij} - output from i^{th} layer j^{th} neuron

Y - output

Data Preprocessing

- We cannot directly put any form of data into a Neural Network. The data we feed into a NN should be numerical(real values).
- If you have categorical data, it should be converted into numerical values using techniques like one-hot encoding.
- If a column has real values but ranges with huge difference(eg:1 to 10000), it should be scaled down using techniques like normalization, standardization.

Training MLP Networks

- A deep MLP neural network tries to learn underlying pattern or to map inputs and outputs using weights by training with given data.
- So, to achieve this we use a optimization technique called Gradient Descent.
- The weights and bias will be updated using a method called backpropagation.
- We calculate loss using a loss function and calculate the derivate and we update the weights during backpropagation.
- The main goal is to minimize the difference(loss) between predicted output and actual output.

Training MLP Example

X1	X2	X3	Y
1	4	7	10
2	5	8	15
3	6	9	20

Training MLP Example

- Consider a single layer MLP network.
- Consider the following conditions.

Linear activation: $f(z) = z$

MSE Loss: $L = \frac{1}{n}\sum(y_{true} - y_{pred})^2$

SGD: Update after each row

Training MLP Example

- Let:
  - Weights: $w1, w2, w3$
  - Bias: b
  - Assume initial values:
    - $w1=0.5, w2=-0.5, w3=1.0, b=0$
    - Learning rate: $\alpha=0.01$

Training MLP Example

- Row 1
- Forward Pass

$$z = w_1 \times x_1 + w_2 \times x_2 + w_3 \times x_3 + b = (0.5)(1) + (-0.5)(4) + (1.0)(7) + 0 = 0.5 - 2 + 7 = 5.5$$

- Prediction

$$\hat{y} = f(z) = z = 5.5$$

- Loss (MSE)

$$L = (y - \hat{y})^2 = (10 - 5.5)^2 = 20.25$$

Training MLP Example

- Backward Pass

$$\frac{\partial L}{\partial \hat{y}} = -2(y - \hat{y}) = -2(10 - 5.5) = -9.0$$

$$\hat{y} = w_1 \times x_1 + w_2 \times x_2 + w_3 \times x_3 + b$$

$$\frac{\partial \hat{y}}{\partial w_1} = x_1$$

- With the chain rule,

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w_1} = (-2)(y - \hat{y}) \times x_1$$

Training MLP Example

- Others too

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \times x_2$$

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial \hat{y}} \times x_3$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \times 1 = \frac{\partial L}{\partial \hat{y}}$$

Training MLP Example

- So for w1,

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \times x_1 = (-9.0)(1) = -9.0$$

$$\frac{\partial L}{\partial w_2} = (-9.0)(4) = -36.0$$

$$\frac{\partial L}{\partial w_3} = (-9.0)(7) = -63.0$$

$$\frac{\partial L}{\partial b} = -9.0$$

- With SGD

$$w_1 = w_1 - \alpha \times \frac{\partial L}{\partial w_1} = 0.5 - 0.01(-9.0) = 0.59$$

$$w_2 = -0.5 - 0.01(-36.0) = -0.14$$

$$w_3 = 1.0 - 0.01(-63.0) = 1.63$$

$$b = 0.0 - 0.01(-9.0) = 0.09$$

Training MLP Example

- Row 2

$$z = (0.59)(2) + (-0.14)(5) + (1.63)(8) + 0.09 = 1.18 - 0.7 + 13.04 + 0.09 = 13.61$$

$$\hat{y} = z = 13.61$$

$$L = (15 - 13.61)^2 = 1.9321$$

$$\frac{\partial L}{\partial \hat{y}} = -2(15 - 13.61) = -2(1.39) = -2.78$$

$$\frac{\partial L}{\partial w_1} = (-2.78)(2) = -5.56$$

$$\frac{\partial L}{\partial w_2} = (-2.78)(5) = -13.9$$

$$\frac{\partial L}{\partial w_3} = (-2.78)(8) = -22.24$$

$$\frac{\partial L}{\partial b} = -2.78$$

Training MLP Example

- Row 2

$$w_1 = 0.59 - 0.01(-5.56) = 0.6456$$

$$w_2 = -0.14 - 0.01(-13.9) = 0.0$$

$$w_3 = 1.63 - 0.01(-22.24) = 1.8524$$

$$b = 0.09 - 0.01(-2.78) = 0.1178$$

Training MLP Example

- Row 3

$$z = (0.6456)(3) + (0.0)(6) + (1.8524)(9) + 0.1178 = 1.9368 + 0 + 16.6716 + 0.1178 = 18.7262$$

$$L = (20 - 18.7262)^2 = 1.6185$$

$$\frac{\partial L}{\partial \hat{y}} = -2(20 - 18.7262) = -2(1.2738) = -2.5476$$

$$\frac{\partial L}{\partial w_1} = (-2.5476)(3) = -7.6428$$

$$\frac{\partial L}{\partial w_2} = (-2.5476)(6) = -15.2856$$

$$\frac{\partial L}{\partial w_3} = (-2.5476)(9) = -22.9284$$

$$\frac{\partial L}{\partial b} = -2.5476$$

Training MLP Example

- Row 3

$$w_1 = 0.6456 - 0.01(-7.6428) = 0.7220$$

$$w_2 = 0.0 - 0.01(-15.2856) = 0.1529$$

$$w_3 = 1.8524 - 0.01(-22.9284) = 2.0817$$

$$b = 0.1178 - 0.01(-2.5476) = 0.1433$$

Training MLP Example

- After the 1st epoch

$$w_1 = 0.7220$$

$$w_2 = 0.1529$$

$$w_3 = 2.0817$$

$$b = 0.1433$$

Multi Layer Perceptron

- A Multi-Layer Perceptron (MLP) neural network can contain multiple hidden layers between the input and output layers.
- Each hidden layer may consist of any number of neurons.
- The first hidden layer receives inputs from the input layer, processes them using an activation function, and passes the transformed outputs to the subsequent hidden layers, continuing this process until the final output layer.

Training MLP

- Let’s now go step by step through backpropagation in a full neural network with

Inputs: x_1, x_2

Hidden Layer: 2 neurons (h_1, h_2)

Output Layer: 1 neuron (y_{pred})

Training MLP

- Initial Weights and Bias Values

Input → Hidden weights:

w_{11}, w_{21} → for hidden neuron h_1

w_{12}, w_{22} → for hidden neuron h_2

Hidden → Output weights:

v_1, v_2

Biases: b_{h1}, b_{h2}, b_o

Activation: sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$, with derivative

$\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Training MLP

- Forward Pass

Hidden layer neurons

$$z_1 = w_{11}x_1 + w_{21}x_2 + b_{h1}, \quad h_1 = \sigma(z_1)$$

$$z_2 = w_{12}x_1 + w_{22}x_2 + b_{h2}, \quad h_2 = \sigma(z_2)$$

Output neuron

$$z_o = v_1h_1 + v_2h_2 + b_o, \quad y_{pred} = \sigma(z_o)$$

Loss (MSE for one sample)

$$L = \frac{1}{2}(y - y_{pred})^2$$

Training MLP

- Backward Pass

(A) Output neuron error

$$\delta_o = \frac{\partial L}{\partial z_o}$$

Break it down with chain rule:

$$\delta_o = \frac{\partial L}{\partial y_{pred}} \cdot \frac{\partial y_{pred}}{\partial z_o}$$

- $\frac{\partial L}{\partial y_{pred}} = (y_{pred} - y)$
- $\frac{\partial y_{pred}}{\partial z_o} = \sigma'(z_o)$

So:

$$\delta_o = (y_{pred} - y) \cdot \sigma'(z_o)$$

Training MLP

- Backward Pass

(B) Gradients for output weights

$$\frac{\partial L}{\partial v_1} = \delta_o \cdot h_1, \quad \frac{\partial L}{\partial v_2} = \delta_o \cdot h_2$$

$$\frac{\partial L}{\partial b_o} = \delta_o$$

Training MLP

- Backward Pass

(C) Backpropagate error to hidden neurons

For hidden neuron 1:

$$\delta_{h1} = \frac{\partial L}{\partial z_1}$$

Using chain rule:

$$\delta_{h1} = \frac{\partial L}{\partial z_o} \cdot \frac{\partial z_o}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1}$$

- $\frac{\partial L}{\partial z_o} = \delta_o$
- $\frac{\partial z_o}{\partial h_1} = v_1$
- $\frac{\partial h_1}{\partial z_1} = \sigma'(z_1)$

So:

$$\delta_{h1} = \delta_o \cdot v_1 \cdot \sigma'(z_1)$$

Training MLP

- Backward Pass

Similarly for hidden neuron 2:

$$\delta_{h2} = \delta_o \cdot v_2 \cdot \sigma'(z_2)$$

(D) Gradients for hidden weights

For hidden neuron 1:

$$\frac{\partial L}{\partial w_{11}} = \delta_{h1} \cdot x_1, \quad \frac{\partial L}{\partial w_{21}} = \delta_{h1} \cdot x_2$$

$$\frac{\partial L}{\partial b_{h1}} = \delta_{h1}$$

For hidden neuron 2:

$$\frac{\partial L}{\partial w_{12}} = \delta_{h2} \cdot x_1, \quad \frac{\partial L}{\partial w_{22}} = \delta_{h2} \cdot x_2$$

$$\frac{\partial L}{\partial b_{h2}} = \delta_{h2}$$

Training MLP

- Backward Pass

Each parameter θ is updated by gradient descent:

$$\theta \leftarrow \theta - \alpha \cdot \frac{\partial L}{\partial \theta}$$

where α is the **learning rate.**

Training MLP Example

Let’s consider

Inputs: $x_1 = 0.6,\ x_2 = 0.1$

Target: $y = 1.0$

Learning rate: $\alpha = 0.1$

Input → Hidden

- To h_1: $w_{11} = 0.50,\ w_{21} = -0.40,\ b_{h1} = 0.00$
- To h_2: $w_{12} = -0.30,\ w_{22} = 0.80,\ b_{h2} = 0.00$

Hidden → Output

- $v_1 = 0.70,\ v_2 = -0.60,\ b_o = 0.00$

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}},\ \ \sigma'(z) = \sigma(z)(1-\sigma(z))$

Training MLP Example

Forward pass

Hidden:

- $z_1 = w_{11}x_1 + w_{21}x_2 + b_{h1} = 0.26 \rightarrow h_1 = \sigma(z_1) = \mathbf{0.5646}$
- $z_2 = w_{12}x_1 + w_{22}x_2 + b_{h2} = -0.10 \rightarrow h_2 = \sigma(z_2) = \mathbf{0.4750}$

Output:

- $z_o = v_1h_1 + v_2h_2 + b_o = 0.11023 \rightarrow \hat{y} = \sigma(z_o) = \mathbf{0.52753}$

Loss (MSE for one sample):

- $L = \frac{1}{2}(y - \hat{y})^2 = \mathbf{0.11161}$

Training MLP Example

Backward pass

Output neuron

$$\delta_o = \frac{\partial L}{\partial z_o} = \underbrace{(\hat{y} - y)}_{\partial L/\partial \hat{y}} \cdot \underbrace{\hat{y}(1-\hat{y})}_{\partial \hat{y}/\partial z_o}$$

Numbers:

$$\hat{y}(1-\hat{y}) = 0.24924 \rightarrow \delta_o = (0.52753 - 1) \times 0.24924 = \mathbf{-0.11776}$$

Gradients to output parameters:

$$\frac{\partial L}{\partial v_1} = \delta_o h_1 = \mathbf{-0.06649}, \quad \frac{\partial L}{\partial v_2} = \delta_o h_2 = \mathbf{-0.05594}, \quad \frac{\partial L}{\partial b_o} = \delta_o = \mathbf{-0.11776}$$

Training MLP Example

Backward pass

Hidden layer

Backprop error to hidden nets:

$$\delta_{h1} = \delta_o \, v_1 \, \sigma'(z_1), \qquad \delta_{h2} = \delta_o \, v_2 \, \sigma'(z_2)$$

- $\sigma'(z_1) = h_1(1 - h_1) = 0.2458$
- $\sigma'(z_2) = h_2(1 - h_2) = 0.2494$

Numbers:

- $\delta_{h1} = -0.11776 \times 0.70 \times 0.2458 = \mathbf{-0.02026}$
- $\delta_{h2} = -0.11776 \times (-0.60) \times 0.2494 = \mathbf{+0.01762}$

Gradients to input→hidden params:

$$\frac{\partial L}{\partial w_{11}} = \delta_{h1} x_1 = \mathbf{-0.01216}, \quad \frac{\partial L}{\partial w_{21}} = \delta_{h1} x_2 = \mathbf{-0.00203}, \quad \frac{\partial L}{\partial b_{h1}} = \delta_{h1} = \mathbf{-0.02026},$$

$$\frac{\partial L}{\partial w_{12}} = \delta_{h2} x_1 = \mathbf{+0.01057}, \quad \frac{\partial L}{\partial w_{22}} = \delta_{h2} x_2 = \mathbf{+0.00176}, \quad \frac{\partial L}{\partial b_{h2}} = \delta_{h2} = \mathbf{+0.01762}.$$

Training MLP Example

SGD update

$\theta \leftarrow \theta - \alpha \frac{\partial L}{\partial \theta}$ with $\alpha = 0.1$.

Updated input → hidden

- $w_{11} = 0.50 - 0.1(-0.01216) = \mathbf{0.50122}$
- $w_{21} = -0.40 - 0.1(-0.00203) = \mathbf{-0.39980}$
- $b_{h1} = 0 - 0.1(-0.02026) = \mathbf{0.00203}$
- $w_{12} = -0.30 - 0.1(0.01057) = \mathbf{-0.30106}$
- $w_{22} = 0.80 - 0.1(0.00176) = \mathbf{0.79982}$
- $b_{h2} = 0 - 0.1(0.01762) = \mathbf{-0.00176}$

Updated hidden → output

- $v_1 = 0.70 - 0.1(-0.06649) = \mathbf{0.70665}$
- $v_2 = -0.60 - 0.1(-0.05594) = \mathbf{-0.59441}$
- $b_o = 0 - 0.1(-0.11776) = \mathbf{0.01178}$

Regularization and Advanced Optimization

Samadhi Chathuranga Rathnayake

MSc, MBA, PGDip, BSc (1st Class)



Regularization

- Regularization in neural networks, specifically in Multi-Layer Perceptron (MLPs), is a set of techniques used to prevent overfitting, ensuring the model generalizes well to new, unseen data.
- Overfitting occurs when a model learns the noise in the training data rather than the actual underlying pattern, resulting in poor performance on test data.

Limiting Hidden Units

- Limiting the number of hidden units (neurons) in the hidden layers of a Multi-Layer Perceptron (MLP) is an effective technique to prevent overfitting.
- By reducing the model's capacity, it forces the network to learn the most significant features and patterns in the data rather than memorizing the training set.

Limiting Hidden Units Helps

- **Reduced Model Complexity**: A smaller number of hidden units means a simpler model, which is less likely to overfit the training data.
- **Better Generalization**: Simplifying the model improves its ability to generalize to new, unseen data.
- **Improved Training Efficiency**: Fewer parameters result in faster training times and reduced computational cost.

Heuristic Methods for Determining Hidden Units (Neurons)

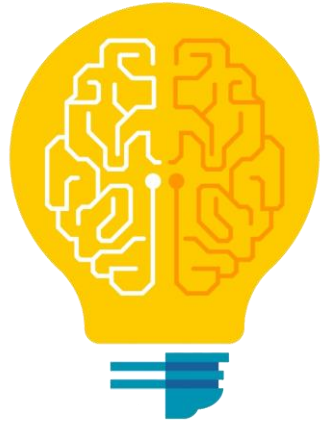
- Between Input and Output Layer Sizes:
  - A common heuristic is to set the number of hidden units to a value between the size of the input layer and the size of the output layer.
  - For example, if you have 10 input features and 1 output, you might choose a number of hidden units between 1 and 10.
- Rule of $\frac{2}{3}$ of Input Plus Output Units:
  - Another heuristic is to use $\frac{2}{3}$ of the size of the input layer plus the size of the output layer.
  - For instance, if you have 15 input features and 3 output classes, the number of hidden units could be $\frac{2}{3} \times 15 + 3 \approx 13$.

Geometric Pyramid Rule (the Master's rule)

- For one hidden layer the number of neurons in the hidden layer is equal to:

$$nbrHID = \sqrt{(nbrINP \times nbrOUT)}$$

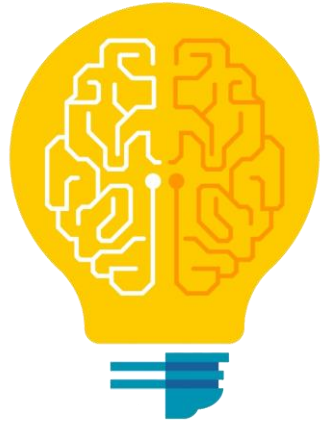
$nbrHID$ – $the\ number\ of\ neurons\ in\ the\ hidden\ layer$,
$nbrINP$ – $the\ number\ of\ neurons\ in\ the\ input\ layer$,
$nbrOUT$ – $the\ number\ of\ neurons\ in\ the\ output\ layer$.



Geometric Pyramid Rule (the Master's rule)

- For two hidden layers:

$$nbrHID1 = nbrOUT \times (r^2)$$
$$nbrHID2 = nbrOUT \times r$$



where,

$$r = (nbrINP/nbrOUT)^{1/3}$$

$nbrHID1$– *the number of neurons in the first hidden layer*
$nbrHID2$– *the number of neurons in the second hidden layer*

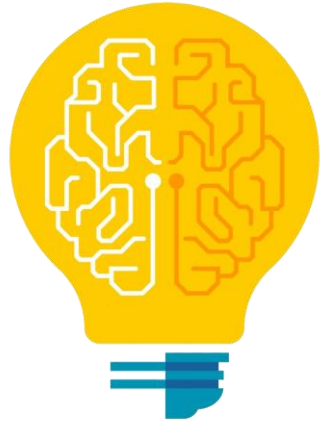
Geometric Pyramid Rule (the Master's rule)

- For three hidden layers:

$$nbrHID1 = nbrOUT \times (r^3)$$
$$nbrHID2 = nbrOUT \times (r^2)$$
$$nbrHID3 = nbrOUT \times r$$

where,

$$r = (nbrINP/nbrOUT)^{1/4}$$



$nbrHID1$– *the number of neurons in the first hidden layer*
$nbrHID2$– *the number of neurons in the second hidden layer*
$nbrHID3$– *the number of neurons in the third hidden layer*

Number of Hidden Layers

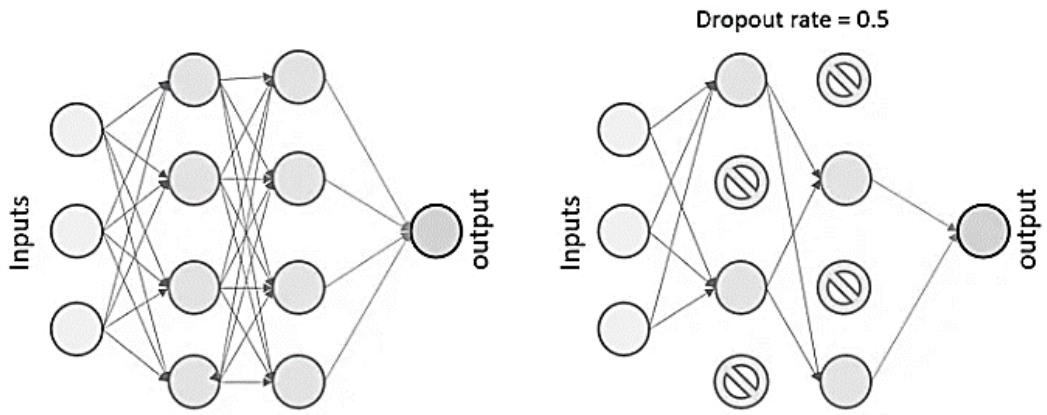
- Selecting the number of hidden layers in a Multilayer Perceptron (MLP) is a crucial step in designing the neural network architecture.
- The choice of the number of hidden layers depends on the complexity of the task, the nature of the data, and the computational resources available.

Number of Hidden Layers

- **Simple Tasks**: For tasks such as linear regression, basic classification, or simple pattern recognition, one or two hidden layers might be sufficient.
- **Complex Tasks**: For more complex tasks such as image recognition, natural language processing, or problems with high-dimensional data, deeper networks with more hidden layers are often beneficial.

Dropout

- During training phase, we randomly drop out or zero out some percentage of neurons in each layer.
- It can be placed after any hidden layer and also, we can specify the dropout rate which determine the number of neurons to drop.
- But, during testing entire network is used without dropout.



Dropout

- Dropout forces the network to learn redundant representations of the data.
- Since any neuron can be dropped out, the network cannot rely on specific neurons and must ensure that other neurons can also capture important patterns.
- This reduces the chance of co-adaptation, where multiple neurons learn to compensate for each other's presence, leading to overfitting.

Dropout

- Dropout introduces noise into the training process, making the network less sensitive to the noise present in the training data.
- This noise helps to prevent the network from fitting too closely to the training data, thus reducing overfitting.

Batch Normalization

- During training, before providing input to feed forward network we normalize them and scale down the values to a particular range.
- Even though we pass normalized data, after passing through n number of hidden layers they are not in same scale.
- These batch input as they pass through multiple neurons with activation function their distribution changes.
- This is called **internal covariate shift**.
- This can be overcome by introducing a layer of Batch normalization.

Batch Normalization

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

- For a given batch, calculate the mean and variance of the activations.
- Normalize the activations using these statistics:

where:

- x is the activation from the previous layer.
- μ is the mean of the activations in the batch.
- σ^2 is the variance of the activations in the batch.
- ϵ is a small constant added to prevent division by zero.

Early Stopping

- Early stopping is a form of regularization, which is a technique that reduces the complexity of the model and prevents overfitting.
- Early stopping works by monitoring a metric, such as the validation error, during the training process and stopping the training when the metric stops improving or starts worsening.
- This way, the model is saved at the point where it has the best performance on the validation data, which is assumed to be a good approximation of the test data.
- Early stopping prevents the model from learning the noise and irrelevant patterns in the training data that can harm its generalization ability.

Hyper Parameter Optimization

- Hyperparameter optimization for regularization in neural networks involves tuning hyperparameters that control the regularization mechanisms, which help prevent overfitting and improve the generalization of the model.
- Popular approaches are,
  - Grid Search
  - Random Search

Weight Decay

- Weight Decay, or L2 Regularization, is a regularization technique applied to the weights of a neural network.
- Minimize a loss function compromising both the primary loss function and a penalty on the L2 Norm of the weights.

$$L_{\text{reg}}(W) = L(W) + \lambda \|W\|_2^2$$

where:

- $\|W\|_2^2 = \sum_j W_j^2$ is the L2 norm of the weight matrix W.
- λ is the regularization parameter that controls the strength of the penalty.

Weight Decay

- When using gradient descent to minimize the regularized loss function, the gradient of the regularized loss with respect to the weights is modified.
- The update rule for weights W becomes:

$$W \leftarrow W - \eta \left(\frac{\partial L}{\partial W} + 2\lambda W \right)$$

where:

- η is the learning rate.
- $\frac{\partial L}{\partial W}$ is the gradient of the original loss function.

Weight Decay

- The term $2\lambda W$ causes each weight W to be reduced by an amount proportional to its current value.
- This is called **weight decay**.
- The larger the weight, the more it is reduced, which discourages weights from becoming too large.
- The regularization parameter λ is crucial for balancing the trade-off between fitting the training data and keeping the model weights small.
- It is typically chosen through cross-validation:
  - Small λ: Less regularization, risk of overfitting.
  - Large λ: More regularization, risk of underfitting.

Alternatives to Weight Decay

- L1 regularization: This penalizes the sum of the absolute values of the weights, encouraging sparsity in the model.
- Dropout: This randomly sets some neurons to zero during training, preventing them from co-adapting and promoting feature independence.
- Early stopping: This stops training once the validation loss starts to increase, preventing overfitting.

L1 Regularization

- L1 regularization adds a penalty term to the loss function proportional to the sum of the absolute values of the model's weights.

$$L_{\text{reg}}(W) = L(W) + \lambda \|W\|_1$$

where:

- $\|W\|_1 = \sum_j |W_j|$ is the L1 norm of the weight matrix W.
- λ is the regularization parameter that controls the strength of the penalty.

L1 Regularization

- L1 regularization has several effects on the training of an MLP:
  - Sparsity: The penalty term $\lambda\|W\|_1$ encourages many weights to be exactly zero.
  - This leads to a sparse model where only a subset of the features contributes to the final predictions.
  - Feature Selection: By setting many weights to zero, L1 regularization effectively selects a subset of important features, which can improve the interpretability of the model.

L1 Regularization

- The gradient of the L1 penalty term with respect to the weights W is:

$$\frac{\partial}{\partial W_j}(\lambda \sum_j |W_j|) = \lambda \cdot \text{sign}(W_j)$$

where:

- $\text{sign}(W_j)$ is the sign function, which returns +1 if $W_j > 0$, -1 if $W_j < 0$, and 0 if $W_j = 0$.

- With the Gradient Decent Algorithm,

$$W_j \leftarrow W_j - \eta \left(\frac{\partial L}{\partial W_j} + \lambda \cdot \text{sign}(W_j) \right)$$

L1 Regularization

1. **Positive Weights:** If $W_j > 0$, the sign function returns +1. The update rule becomes:
   $$W_j \leftarrow W_j - \eta \left(\frac{\partial L}{\partial W_j} + \lambda \right)$$
   This reduces W_j more aggressively, encouraging it to decrease.

2. **Negative Weights:** If $W_j < 0$, the sign function returns -1. The update rule becomes:
   $$W_j \leftarrow W_j - \eta \left(\frac{\partial L}{\partial W_j} - \lambda \right)$$
   This increases W_j more aggressively (making it less negative), also encouraging it to move towards zero.

3. **Near Zero Weights:** When W_j is small, the gradient $\frac{\partial L}{\partial W_j}$ might be smaller in magnitude than λ. In such cases, the update rule can drive W_j to zero:
   $$W_j \leftarrow W_j - \eta \left(\frac{\partial L}{\partial W_j} + \lambda \cdot \text{sign}(W_j) \right)$$
   If $\frac{\partial L}{\partial W_j}$ is sufficiently small, the term $\lambda \cdot \text{sign}(W_j)$ can dominate, leading to:
   $$W_j \approx 0$$

More About Optimization

- Optimizers are used to update parameters and to reduce loss efficiently.
- There are multiple optimizing technique which we use to find our global minima like Gradient descent, Stochastic gradient descent, Stochastic gradient descent with momentum, Nesterov accelerated gradient, Adagrad, Adadelta, RMSProp, Adam.
- The primary goal of optimization in machine learning is to minimize the loss function $L(\theta)$, where θ represents the model parameters (weights and biases).

$$\min_{\theta} L(\theta)$$

Stochastic Gradient Descent (SGD) Optimizer

- A simple optimization algorithm where the gradient is computed for each mini-batch.

$$\theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$$

- Can be slow to converge due to the constant learning rate.
- Sensitive to the choice of learning rate.

Momentum Optimizer

- Enhances SGD by adding a momentum term to the update rule.

$$v \leftarrow \beta v + \eta \nabla_\theta L(\theta)$$

$$\theta \leftarrow \theta - v$$

- Helps accelerate gradients in the right direction, leading to faster convergence.
- Reduces oscillations.

Nesterov Accelerated Gradient (NAG)

- An improvement over Momentum that looks ahead at the future position of the parameters.

$$\text{Lookahead} \leftarrow \theta - \beta v$$

$$v \leftarrow \beta v + \eta \nabla_\theta L(\text{Lookahead})$$

$$\theta \leftarrow \theta - v$$

- Provides more responsive updates by incorporating the lookahead step.

Adaptive Gradient Optimization (Adagrad)

- Adapts the learning rate for each parameter based on historical gradients.

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot \nabla_\theta L(\theta)$$

where G_t is the sum of the squares of the gradients up to time t, and ϵ is a small constant to prevent division by zero.

- Adjusts the learning rate for each parameter, allowing different learning rates for different parameters.
- Can become overly aggressive and lead to very small learning rates.

Root Mean Square Propagation (RMSprop)

- Combines the ideas of Adagrad with a moving average of squared gradients.

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta)\nabla_\theta L(\theta)^2$$

where:

- $E[g^2]_t$ is the moving average at time step t.
- β is the decay rate for the moving average (typically close to 1, e.g., 0.9).
- $\nabla_\theta L(\theta)^2$ is the element-wise square of the gradient at time step t.

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \odot \nabla_\theta L(\theta)$$

- Prevents the learning rate from becoming too small, providing a more stable training process.

Adaptive Moment Estimation (Adam)

- Combines the benefits of both Momentum and RMSprop.

$$m \leftarrow \beta_1 m + (1 - \beta_1) \nabla_\theta L(\theta)$$

$$v \leftarrow \beta_2 v + (1 - \beta_2) \nabla_\theta L(\theta)^2$$

$$\hat{m} \leftarrow \frac{m}{1 - \beta_1^t}$$

$$\hat{v} \leftarrow \frac{v}{1 - \beta_2^t}$$

Adaptive Moment Estimation (Adam)

$$\theta \leftarrow \theta - \frac{\eta \hat{m}}{\sqrt{\hat{v} + \epsilon}}$$

where m is the first moment (mean), v is the second moment (variance), β_1 and β_2 are the decay rates, and t is the timestep.

- Adaptively adjusts the learning rate for each parameter.
- Combines the advantages of Momentum and RMSprop.

When to Use SGD

- Suitable when the **dataset is very large** and training is performed using mini-batches.
- Useful when you want a **simple optimizer with low memory requirements**.
- Good when you want **direct control over the learning rate** and learning-rate schedule.
- Often suitable when **good generalization** is more important than reaching a good solution very quickly.
- Can work well when enough training time is available.
- Less suitable when the loss surface causes **strong oscillations or very slow convergence**.

When to Use SGD with Momentum

- Suitable when ordinary SGD is **converging slowly**.
- Useful when gradients repeatedly move in a **similar direction**.
- Helps when SGD **oscillates between directions**, especially in narrow regions of the loss surface.
- Suitable for training **deep neural networks** where plain SGD may require many iterations.
- Often useful when you want the generalization properties of SGD but need **faster and smoother convergence**.
- Requires choosing an appropriate **learning rate and momentum coefficient**.

When to Use Nesterov Accelerated Gradient

- Suitable when you want the advantages of **Momentum with a more responsive update**.
- Useful when momentum may move the parameters **too strongly in the previous direction**.
- The look-ahead gradient helps the optimizer **correct its direction earlier**.
- Can be useful for loss surfaces containing **curves, valleys and changing gradient directions**.
- Suitable when carefully tuned SGD-based optimization is preferred over adaptive optimizers.
- Requires tuning both the **learning rate and momentum coefficient**.

When to Use AdaGrad

- Suitable when the data contains **sparse features**.
- Particularly useful when some parameters receive **frequent updates while others receive rare updates**.
- Automatically provides a **different effective learning rate for each parameter**.
- Can be useful for **text data and other high-dimensional sparse data**.
- Less suitable for very long neural-network training because accumulated squared gradients continually increase.
- The effective learning rate may eventually become **extremely small**, slowing learning.

When to Use RMSProp

- Suitable when AdaGrad's learning rate becomes **too small during long training**.
- Useful for **non-stationary and changing optimization problems**.
- Provides an **adaptive learning rate for each parameter**.
- Suitable for neural networks where gradients can have **very different magnitudes across parameters**.
- Often provides faster and more stable training than basic SGD when limited tuning is available.
- A useful choice when you want adaptive learning rates without AdaGrad's continuous accumulation problem.

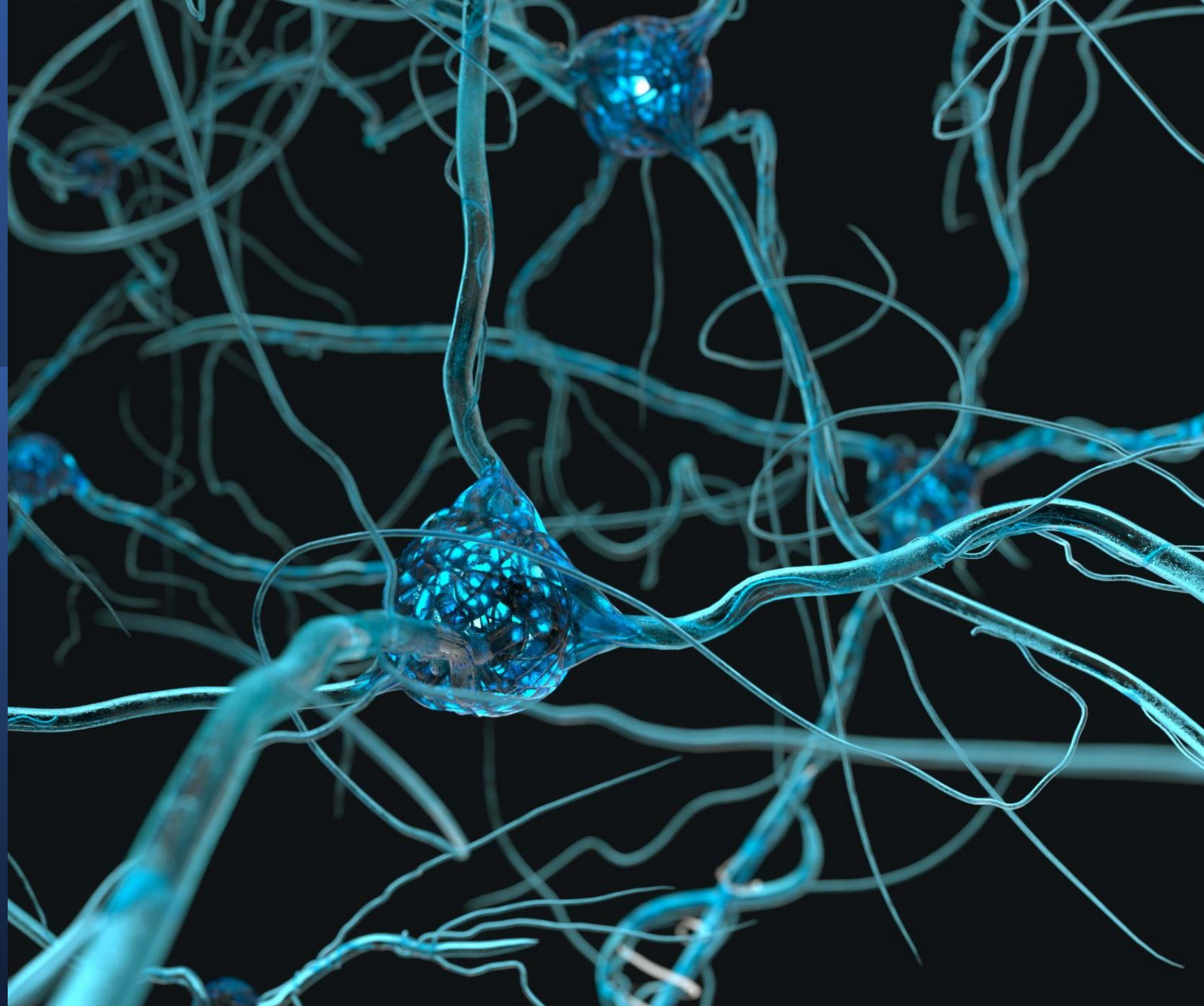
When to Use Adam

- Suitable as a **strong general-purpose starting optimizer** for many neural-network problems.
- Useful when you want **fast convergence with relatively little manual tuning**.
- Suitable when different parameters require **different effective learning rates**.
- Works well with **large datasets, many parameters and noisy mini-batch gradients**.
- Often useful for deep learning problems involving **complex or sparse gradients**.
- Combines **momentum-like gradient averaging with RMSProp-like adaptive scaling**.
- However, Adam is **not automatically the best optimizer for every problem**; SGD with Momentum can sometimes provide better final generalization.

Samadhi Chathuranga Rathnayake

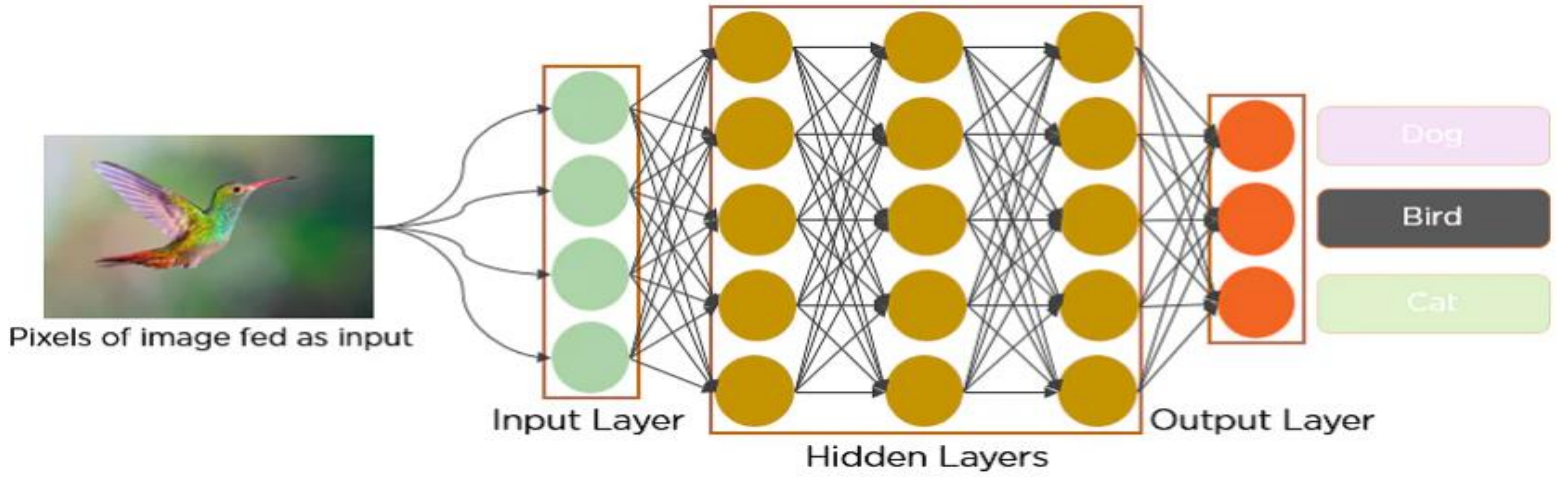
MSc, MBA, PGDip, BSc (1st Class)

Convolutional Neural Networks (CNN)



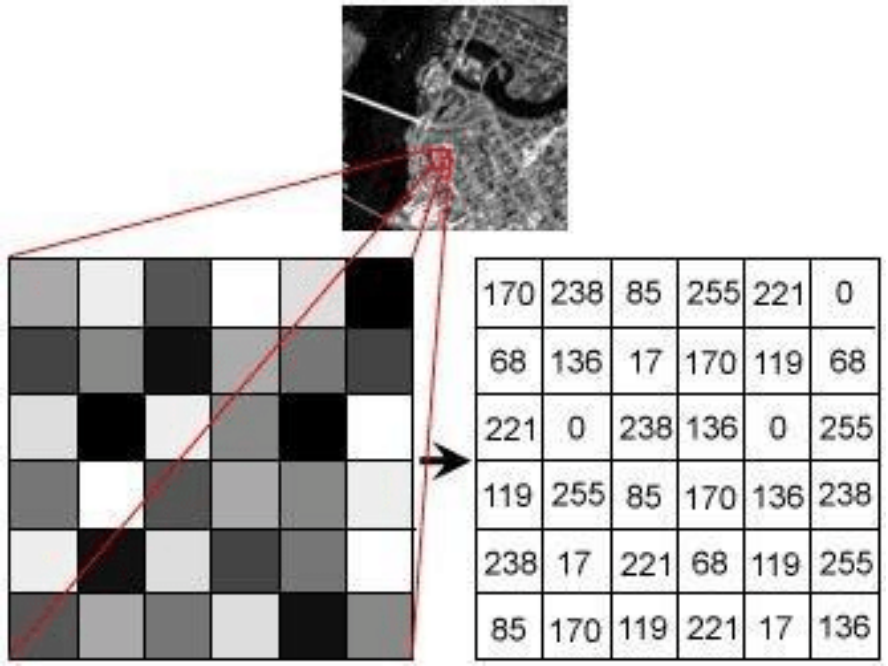
Introduction to CNN

- Convolutional Neural Networks (CNNs) are a type of deep learning algorithm specifically designed for working with grid-like data structures, such as images.
- They have revolutionized computer vision by achieving high accuracy in tasks like image classification, object detection, and segmentation.
- It has two parts; Feature Extraction and Classification.



Representation of Pixels

- An image is represented as a grid of pixels, where each pixel corresponds to a small portion of the image.
- The values in the pixels represent the intensity or color of that specific part of the image.



170	238	85	255	221	0
68	136	17	170	119	68
221	0	238	136	0	255
119	255	85	170	136	238
238	17	221	68	119	255
85	170	119	221	17	136

Grayscale Image (1 Channel)

- Each pixel in a grayscale image represents an intensity value.
- The intensity value is typically a single number ranging from 0 to 255, where 0 represents black, 255 represents white, and values in between represent varying shades of gray.



Color Image (3 Channels)

- Each pixel in a color image is represented by three values corresponding to the Red, Green, and Blue (RGB) channels.
- Each channel typically has a value ranging from 0 to 255.
- For example, a pixel value of (255, 0, 0) represents red, (0, 255, 0) represents green, (0, 0, 255) represents blue, and (255, 255, 255) represents white.



Pixel Representation of Grey Scale Images

- For a grayscale image, consider the following small 3x3 pixel image

$$\begin{bmatrix} 100 & 150 & 200 \\ 50 & 120 & 180 \\ 0 & 60 & 255 \end{bmatrix}$$

Pixel Representation of Color Images

- For a color image, consider the following small 2x2 pixel image with RGB values

$$\begin{bmatrix} (255 & 0 & 0) & (0 & 255 & 0) \\ (0 & 0 & 255) & (255 & 255 & 0) \end{bmatrix}$$

Normalized Pixel Values

- In some cases, pixel values are normalized to a range between 0 and 1, especially when used as input to neural networks.
- Normalization helps in speeding up the training process and can lead to better performance.
- For example, if the original pixel values range from 0 to 255, you can normalize them by dividing by 255.

Grey Scale Image Normalization

- Consider the previous small 3x3 pixel image

$$\begin{bmatrix} 100/255 & 150/255 & 200/255 \\ 50/255 & 120/255 & 180/255 \\ 0/255 & 60/255 & 255/255 \end{bmatrix} = \begin{bmatrix} 0.39 & 0.59 & 0.78 \\ 0.20 & 0.47 & 0.71 \\ 0 & 0.24 & 1 \end{bmatrix}$$

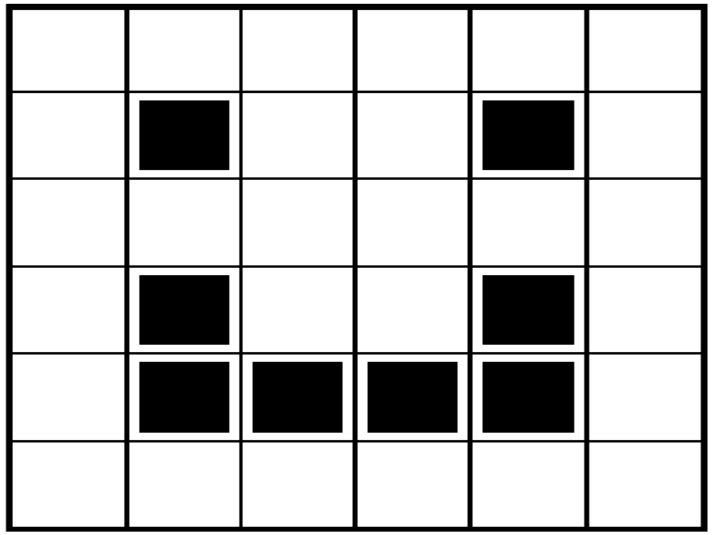
Color Image Normalization

- Consider the previous small 2x2 pixel image with RGB values

$$\begin{bmatrix} (255/255 & 0/255 & 0/255) & (0/255 & 255/255 & 0/255) \\ (0/255 & 0/255 & 255/255) & (255/255 & 255/255 & 0/255) \end{bmatrix} = \begin{bmatrix} (1 & 0 & 0) & (0 & 1 & 0) \\ (0 & 0 & 1) & (1 & 1 & 0) \end{bmatrix}$$

Introduction to CNN

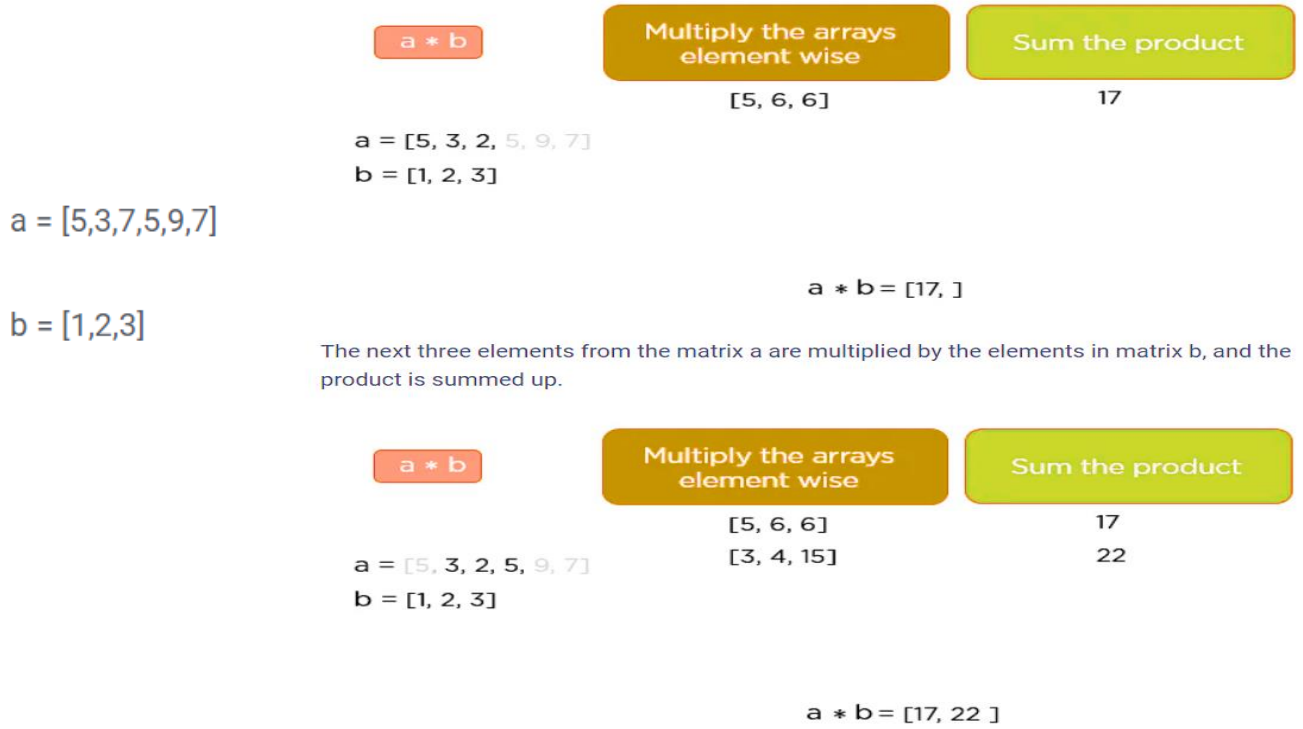
- In CNN, every image is represented in the form of an array of pixel values.



```
1 1 1 1 1 1
1 0 1 1 0 1
1 1 1 1 1 1
1 0 1 1 0 1
1 0 0 0 0 1
1 1 1 1 1 1
```

Convolutional Operations

- Any convolutional neural network's foundation is the convolution process.
- Let's examine the convolution operation using two 1-dimensional matrices, a and b.



- This process continues until the convolution operation is complete.

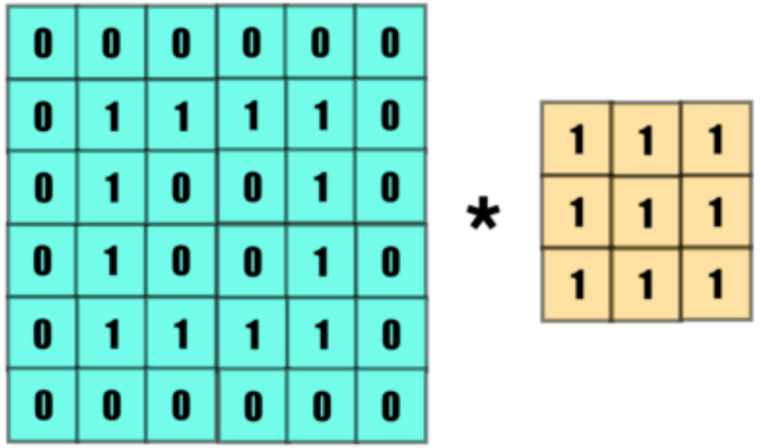
Layers of Convolution Neural Network

- A convolution neural network has multiple hidden layers that help in extracting information from an image. The four important layers in CNN are:

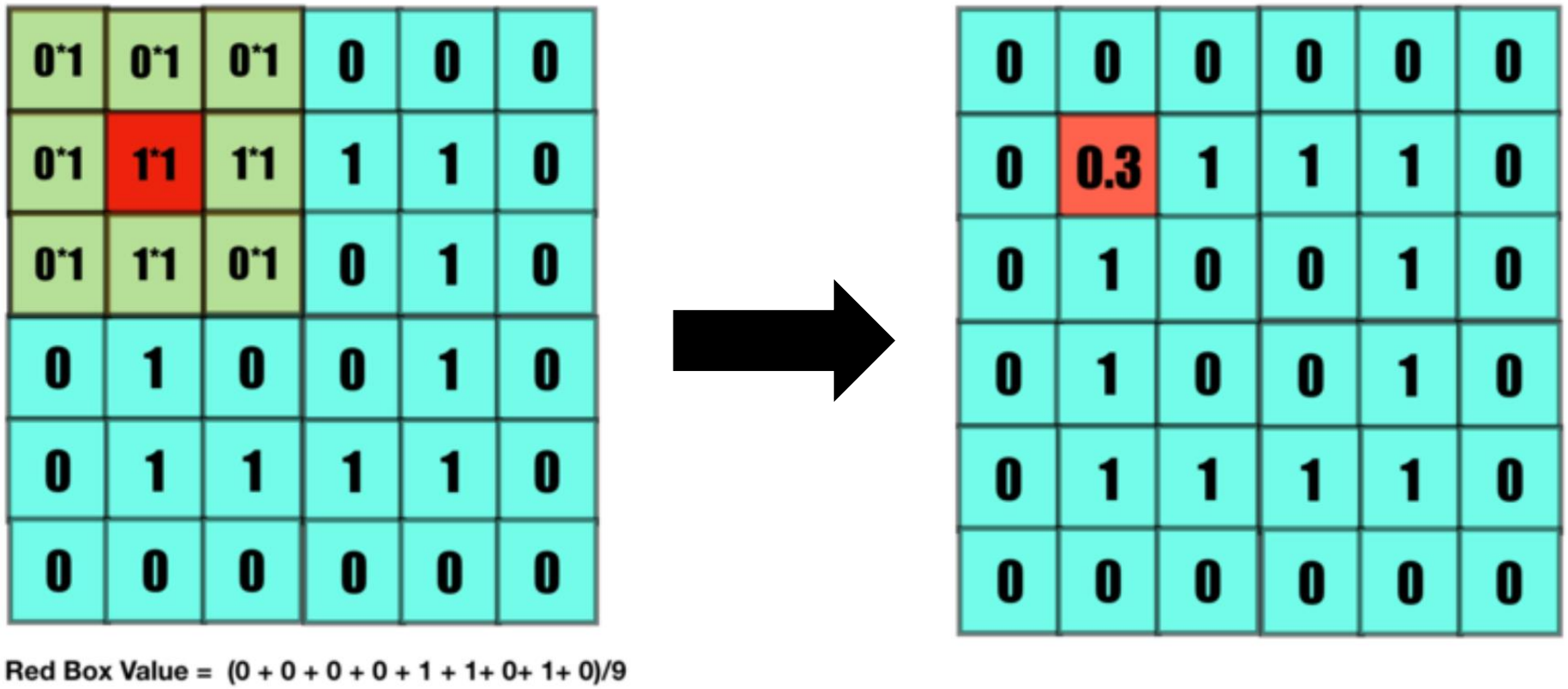
  - Convolution layer
  - ReLU (Rectified Linear Unit) layer
  - Pooling layer
  - Fully connected layer

Convolution Layer

- The process of extracting useful elements from an image begins with this.
- Multiple filters work together to perform the convolution action in a convolution layer.
- Each image can be thought of as a matrix of pixel values.

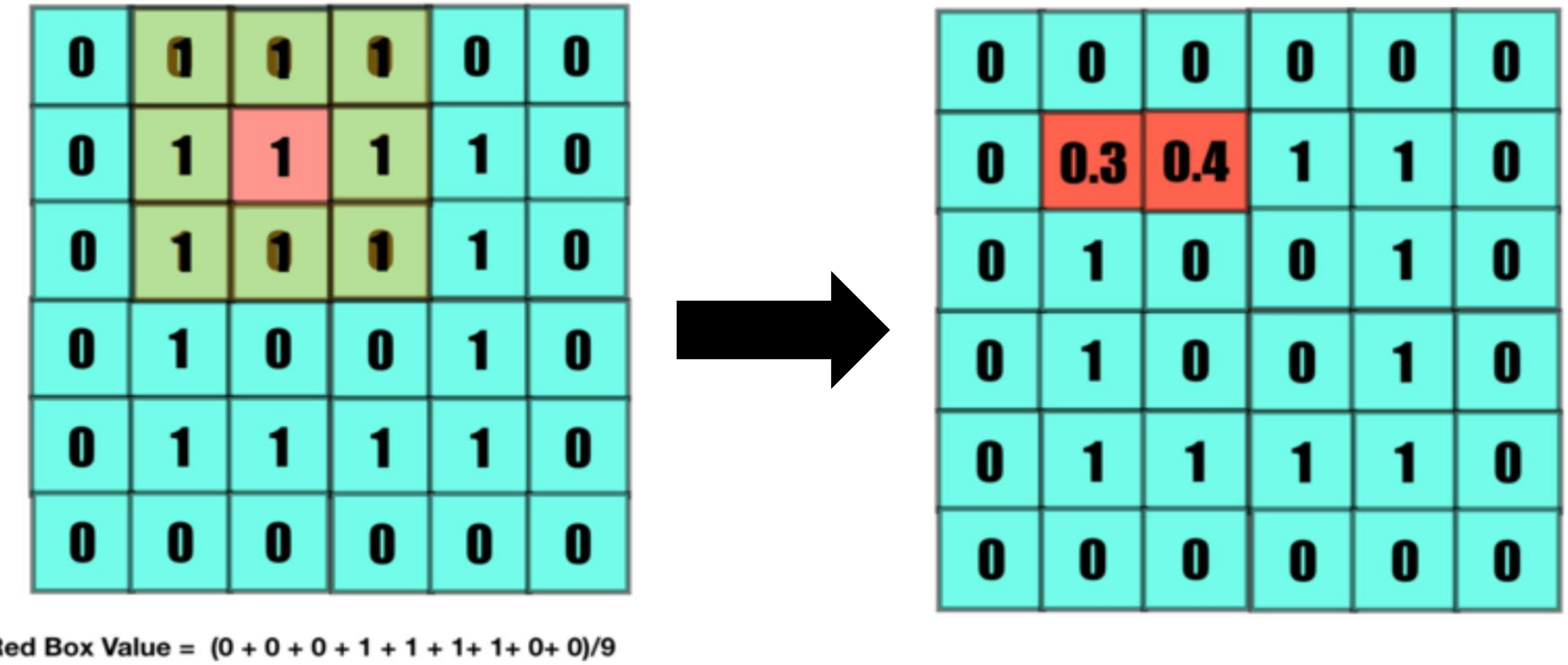


Convolution Layer



Red Box Value = (0 + 0 + 0 + 0 + 1 + 1+ 0+ 1+ 0)/9

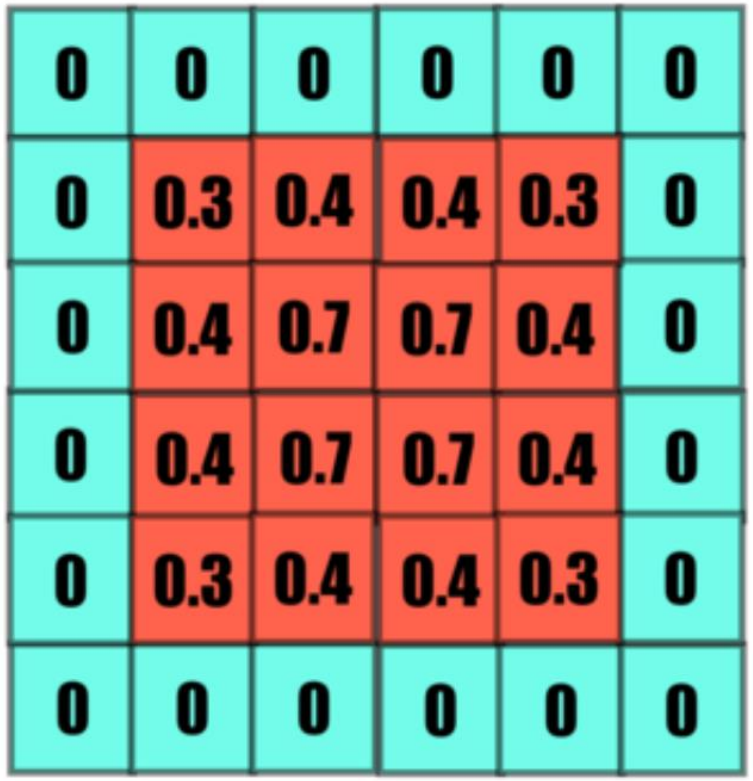
Convolution Layer



Red Box Value = (0 + 0 + 0 + 1 + 1 + 1+ 1+ 0+ 0)/9

Convolution Layer

- Get up to



Convolution Layer - Stride

- This refers to the number of pixels by which the convolution filter (or kernel) moves across the input image or feature map.
- Stride determines how much the filter shifts at each step as it slides over the input during the convolution operation.
- Ex-
  - A stride of 1 means the filter moves one pixel at a time, which results in the output feature map having a similar size to the input (assuming no padding).
  - A stride of 2 means the filter moves two pixels at a time, which effectively reduces the spatial dimensions of the output feature map.
- The stride affects the size of the output feature map.
- Larger strides lead to smaller output feature maps because the filter overlaps less with the input.

Convolution Layer - Stride

Convolution Operation with Stride 1

INPUT IMAGE

18	54	51	239	244	188
55	121	75	78	95	88
35	24	204	113	109	221
3	154	104	235	25	130
15	253	225	159	78	233
68	85	180	214	245	0

WEIGHT

1	0	1
0	1	0
1	0	1

429

Convolution Operation with Stride 2

INPUT IMAGE

18	54	51	239	244
55	121	75	78	95
35	24	204	113	109
3	154	104	235	25
15	253	225	159	78

WEIGHT

1	0	1
0	1	0
1	0	1

429

Convolution Layer - Stride

- For an input dimension W and a stride S, the output dimension $W*$ is approximately,

$$W^* = \frac{W - Filter\ Size}{S} + 1$$

  - For an input image of size 28x28 with a 3x3 filter and a stride of 1, the output feature map will be 26x26 (assuming no padding).
  - For the same input and filter size but with a stride of 2, the output feature map will be 13x13.
- Higher stride values result in fewer sliding operations, thus reducing the computational load and memory usage.
- However, too high a stride can lead to a loss of detailed information due to the larger steps taken by the filter.

Convolution Layer - Padding

- **Padding** virtually extends the matrix to cater to border values as described in the image below. The pink layer isn't a part of the feature matrix, but helps in convolution.

0	0	0	0	0	0	0	0
0	18	54	51	239	244	188	0
0	55	121	75	78	95	88	0
0	35	24	204	113	109	221	0
0	3	154	104	235	25	130	0
0	15	253	225	159	78	233	0
0	68	85	180	214	245	0	0
0	0	0	0	0	0	0	0

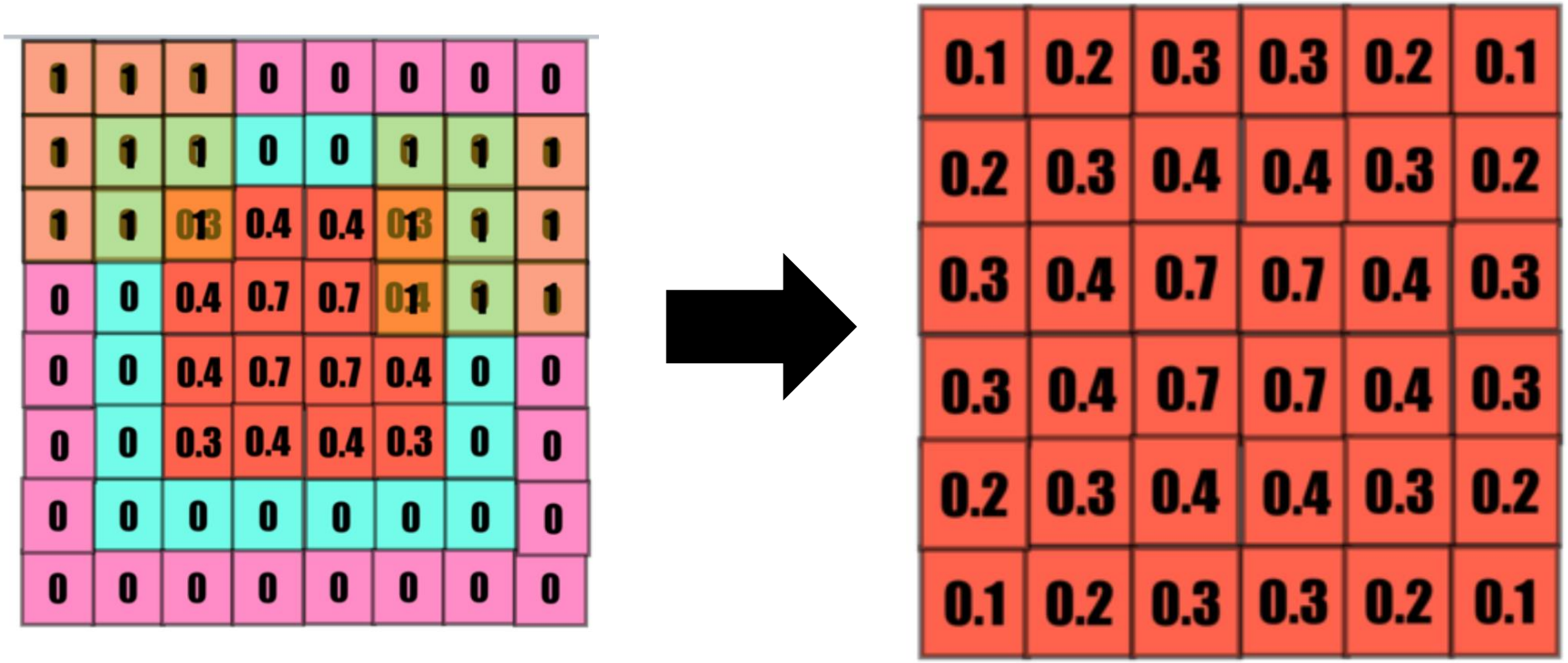
WEIGHT

1	0	1
0	1	0
1	0	1

139

Convolution Layer - Padding

- Get back to the same example.



Convolution Layer

- This is extracting particular information from the original image matrix.



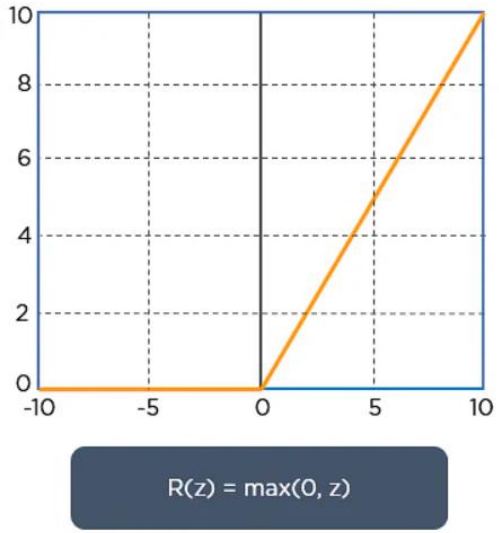
Original Image



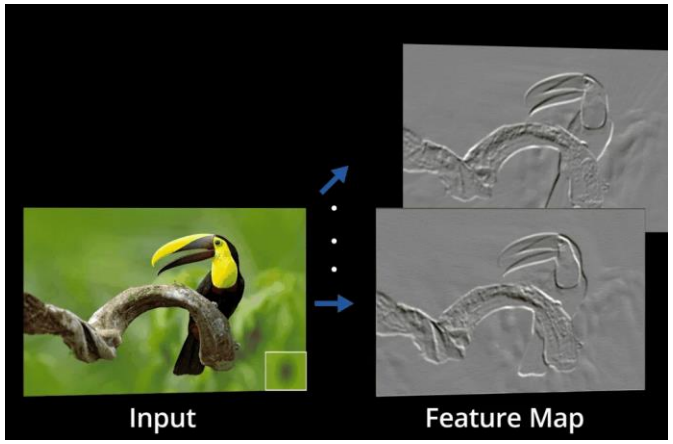
Convoluted image

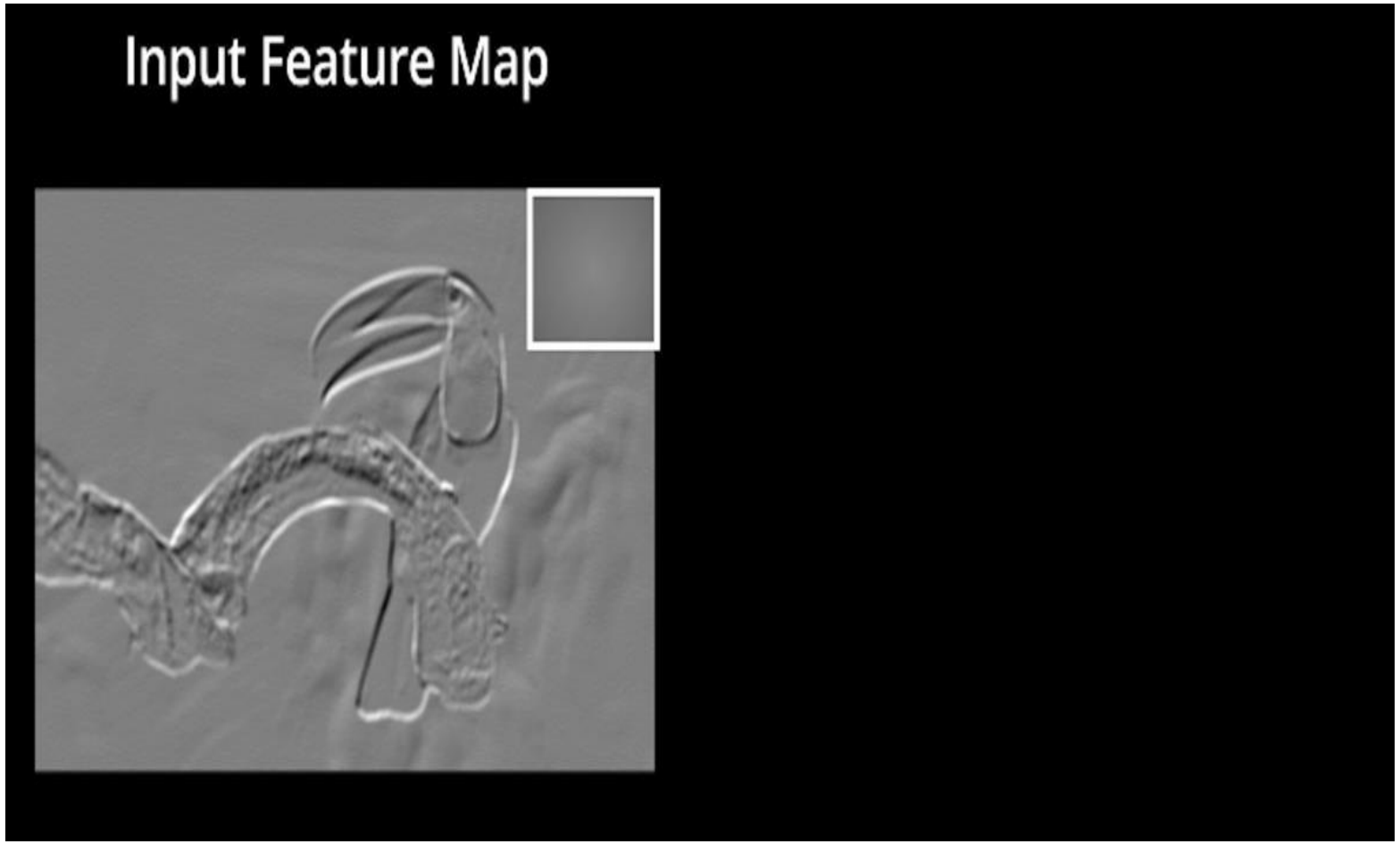
ReLU layer

- ReLU stands for the rectified linear unit.
- Once the feature maps are extracted, the next step is to move them to a ReLU layer.
- ReLU performs an element-wise operation and sets all the negative pixels to 0.
- It introduces non-linearity to the network, and the generated output is a rectified feature map.



ReLU layer



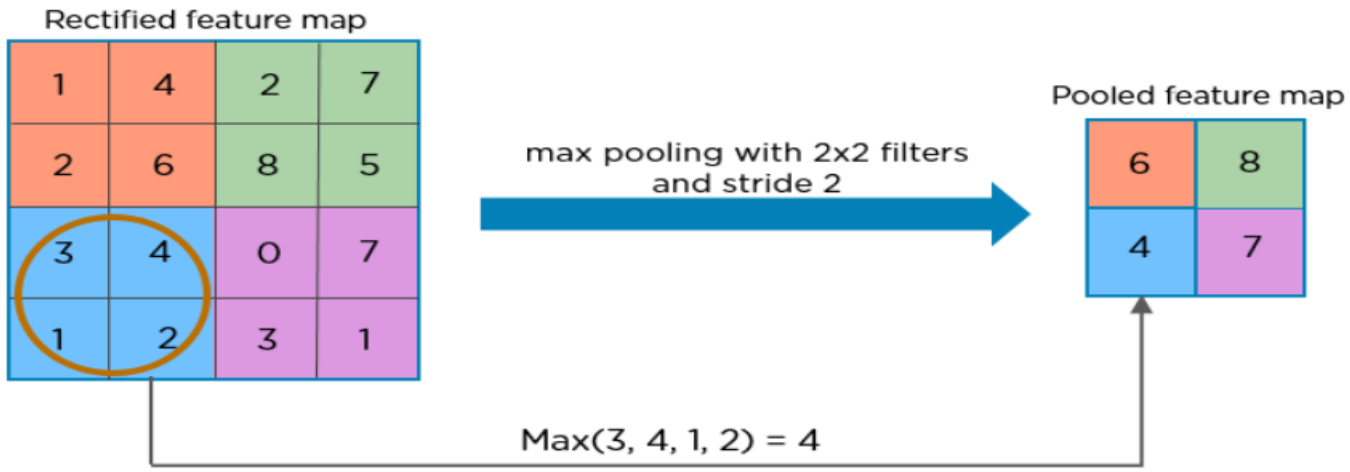


Pooling Layer

- Pooling is a down-sampling operation that reduces the dimensionality of the feature map.
  - Max pooling
  - Average pooling
  - L2 norm pooling
- Pooling is a crucial operation in Convolutional Neural Networks (CNNs) that helps to progressively reduce the spatial dimensions (width and height) of the input representations, reducing the number of parameters and computations in the network.
- It also helps to control overfitting by making the detection of features more robust.

Pooling Layer

- The rectified feature map now goes through a pooling layer to generate a pooled feature map.

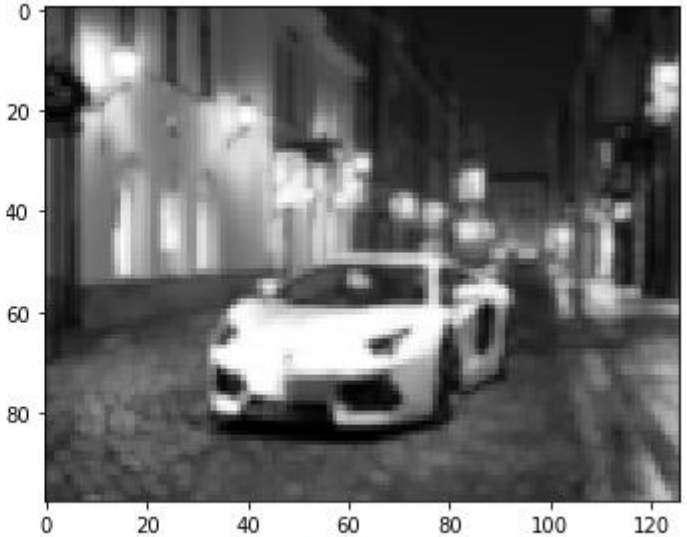


Pooling Layer

- To distinguish distinct portions of the image, such as edges, corners, bodies, feathers, eyes, and beak, the pooling layer employs a variety of filters.



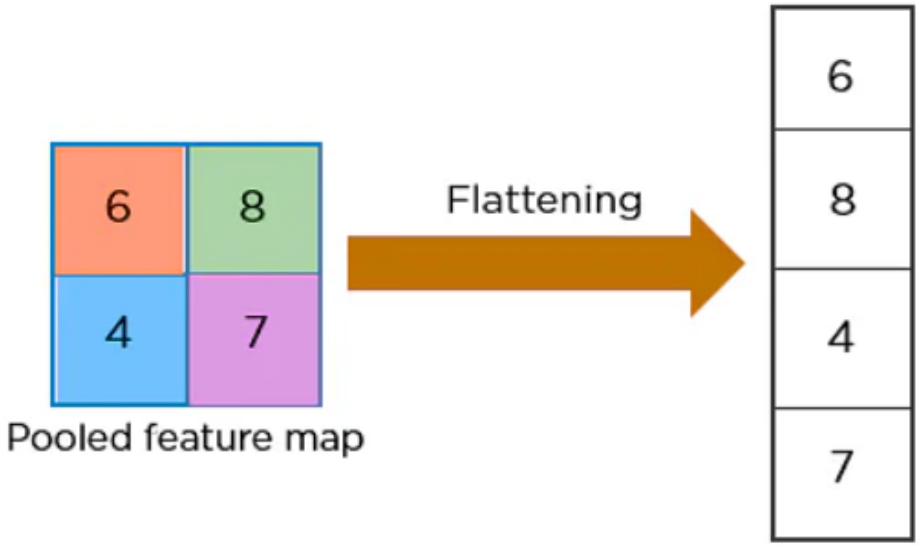
Convoluted image



MAX Pooling

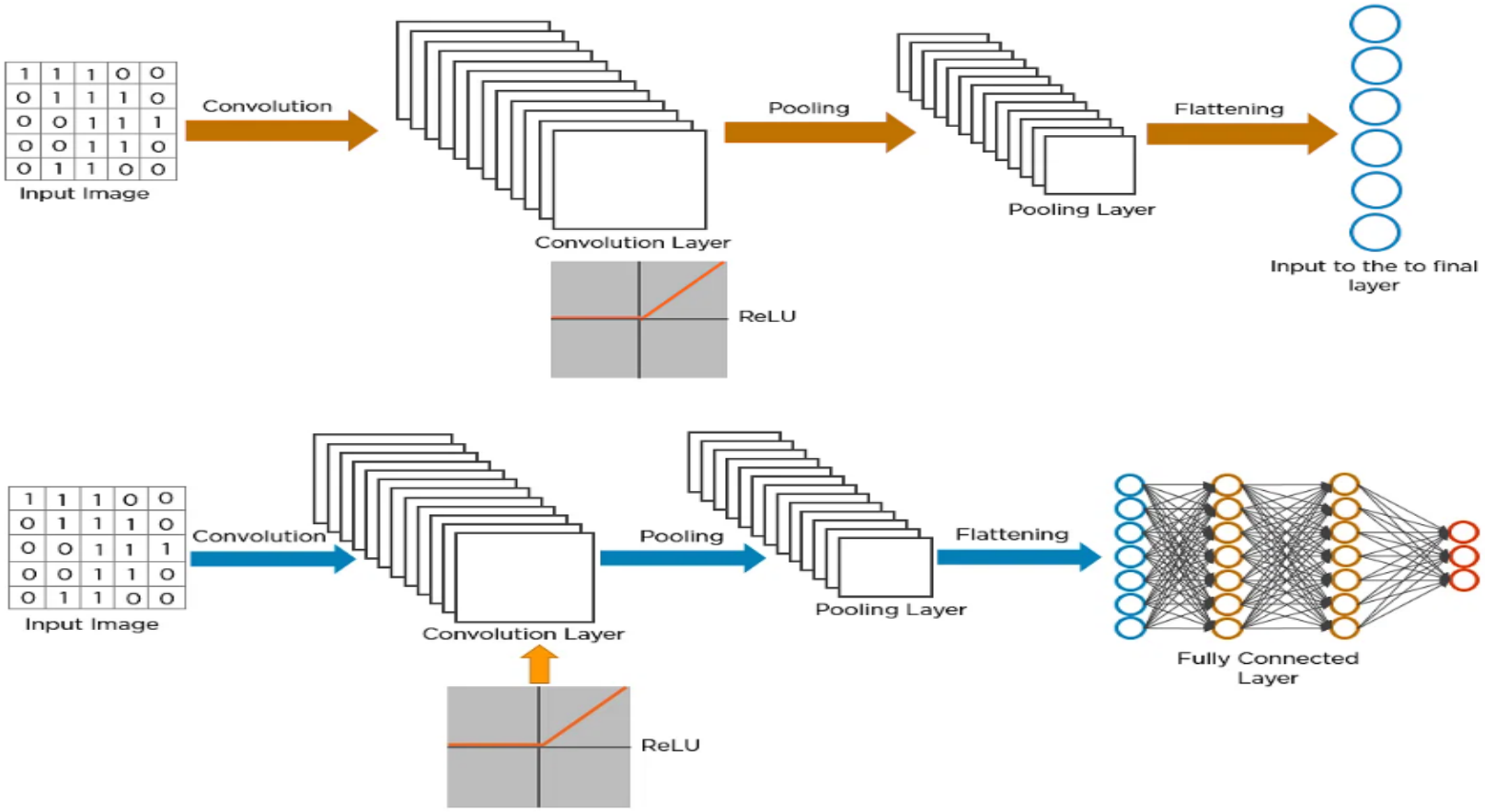
Flattening

- The next step in the process is called flattening.
- Flattening is used to convert all the resultant 2-Dimensional arrays from pooled feature maps into a single long continuous linear vector.

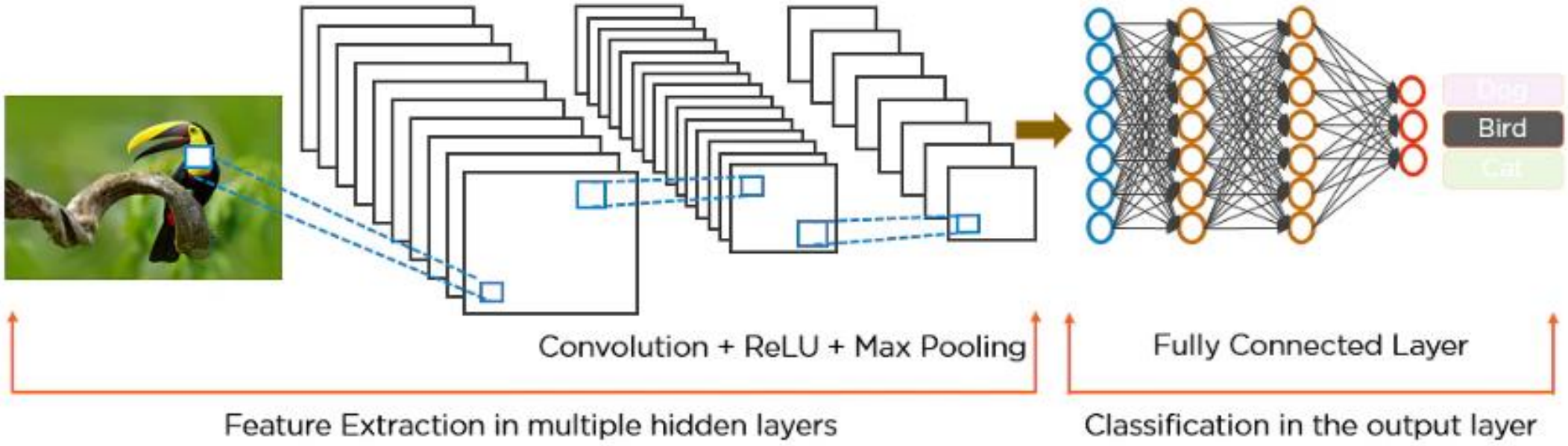


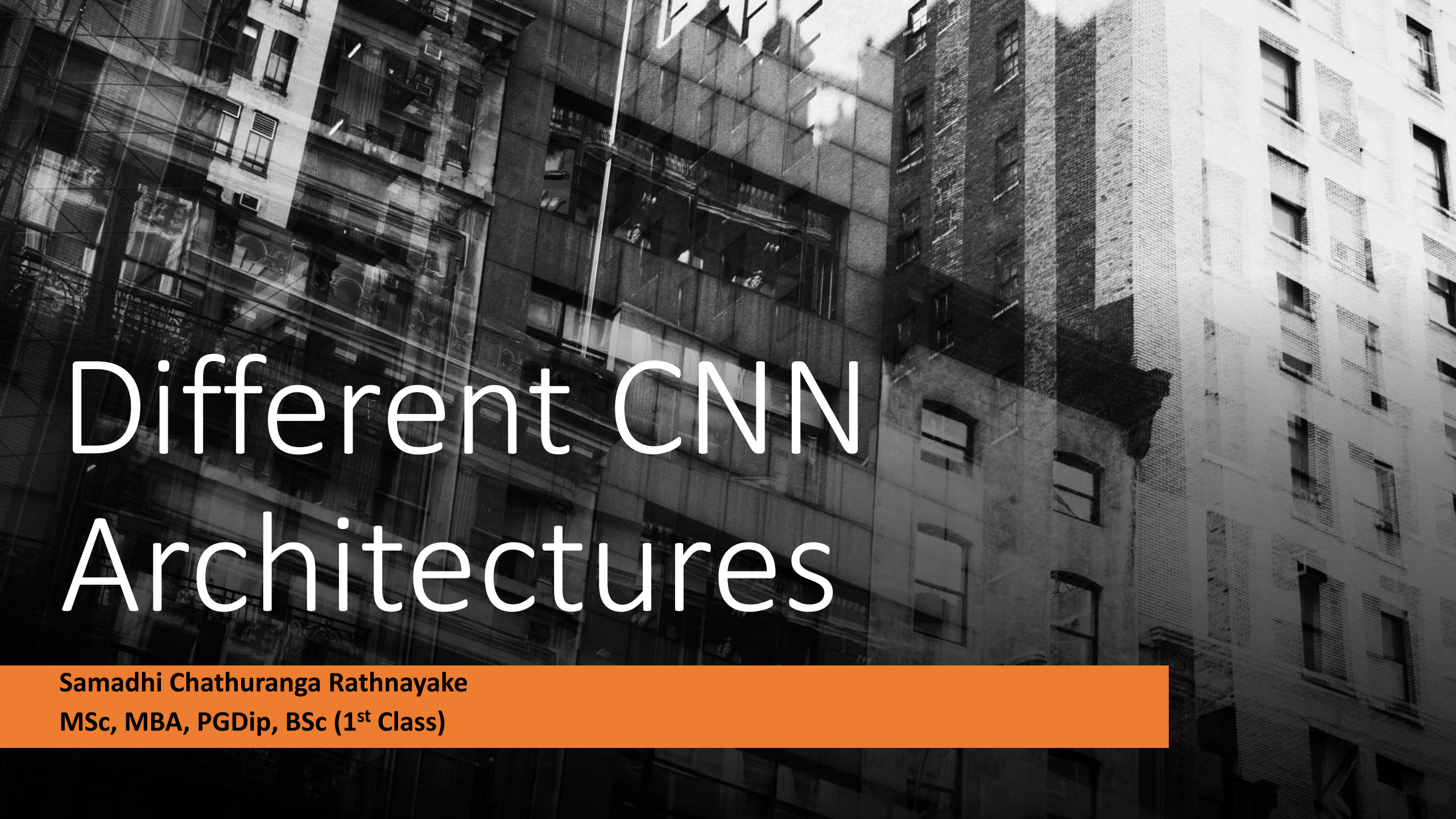
Fully Connected Layer

- The flattened matrix is fed as input to the fully connected layer to classify the image.



Fully Connected Layer





Different CNN Architectures

Samadhi Chathuranga Rathnayake

MSc, MBA, PGDip, BSc (1st Class)

Traditional CNN

Convolutional Neural Networks, commonly referred to as CNNs, are a specialized kind of neural network architecture that is designed to process data with a grid-like topology.

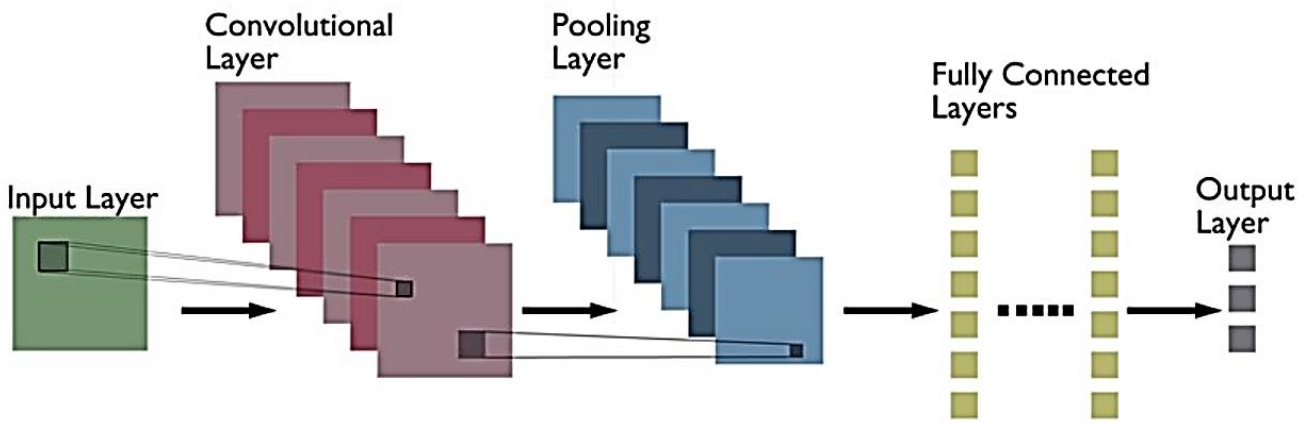
This makes them particularly well-suited for dealing with spatial and temporal data, like images and videos, that maintain a high degree of correlation between adjacent elements.

CNNs are similar to other neural networks, but they have an added layer of complexity due to the fact that they use a series of convolutional layers.

Convolutional layers perform a mathematical operation called convolution, a sort of specialized matrix multiplication, on the input data.

The convolution operation helps to preserve the spatial relationship between pixels by learning image features using small squares of input data. .

Traditional CNN



LeNet – First CNN Architecture

LeNet was developed in 1998 by Yann LeCun, Corinna Cortes, and Christopher Burges for handwritten digit recognition problems.

LeNet was one of the first successful CNNs and is often considered the “Hello World” of deep learning. It is one of the earliest and most widely-used CNN architectures and has been successfully applied to tasks such as handwritten digit recognition.

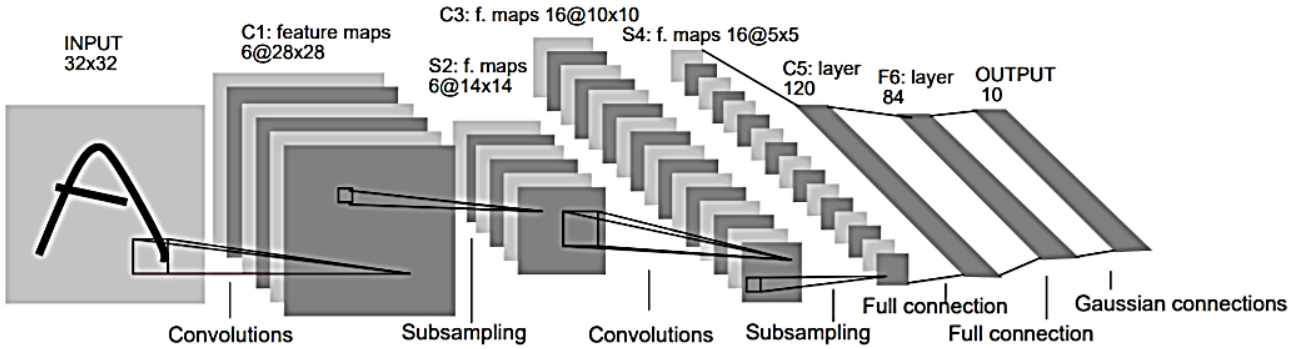
The LeNet architecture consists of multiple convolutional and pooling layers, followed by a fully-connected layer.

The model has five convolution layers followed by two fully connected layers.

LeNet was the beginning of CNNs in deep learning for computer vision problems. However, LeNet could not train well due to the vanishing gradients problem.

To solve this issue, a shortcut connection layer known as max-pooling is used between convolutional layers to reduce the spatial size of images which helps prevent overfitting and allows CNNs to train more effectively.

LeNet – First CNN Architecture



AlexNet – Deep Learning Architecture that popularized CNN

AlexNet was developed by Alex Krizhevsky, Ilya Sutskever, and Geoff Hinton.

AlexNet network had a very similar architecture to LeNet, but was deeper, bigger, and featured Convolutional Layers stacked on top of each other.

AlexNet was the first large-scale CNN and was used to win the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012.

The AlexNet architecture was designed to be used with large-scale image datasets and it achieved state-of-the-art results at the time of its publication.

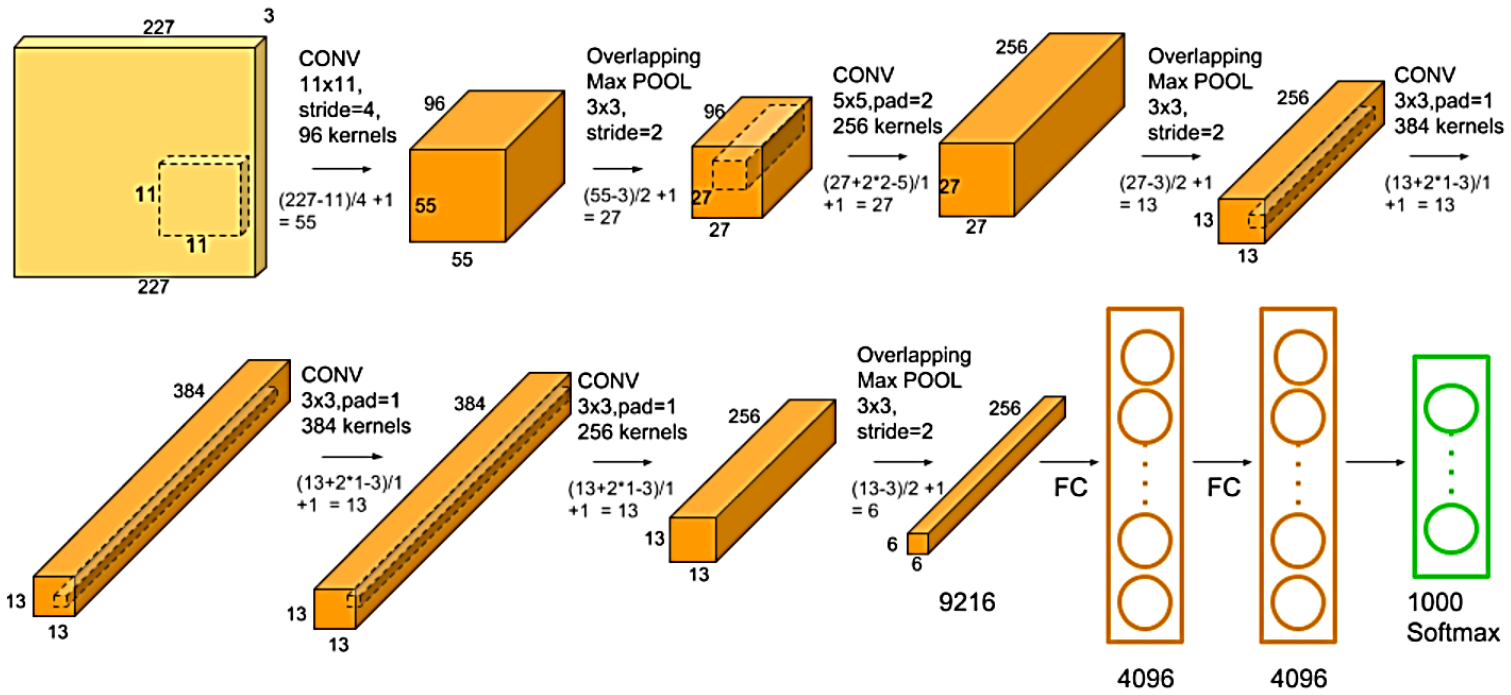
AlexNet is composed of 5 convolutional layers with a combination of max-pooling layers, 3 fully connected layers, and 2 dropout layers.

The activation function used in all layers is Relu.

The activation function used in the output layer is Softmax.

The total number of parameters in this architecture is around 60 million.

AlexNet – Deep Learning Architecture that popularized CNN



ZFNet – Deep Learning Architecture that popularized CNN



ZFnet is the CNN architecture that uses a combination of fully-connected layers and CNNs.



ZF Net was developed by Matthew Zeiler and Rob Fergus. It was the ILSVRC 2013 winner.



The network has relatively fewer parameters than AlexNet, but still outperforms it on ILSVRC 2012 classification task by achieving top accuracy with only 1000 images per class.



It was an improvement on AlexNet by tweaking the architecture hyperparameters, in particular by expanding the size of the middle convolutional layers and making the stride and filter size on the first layer smaller.

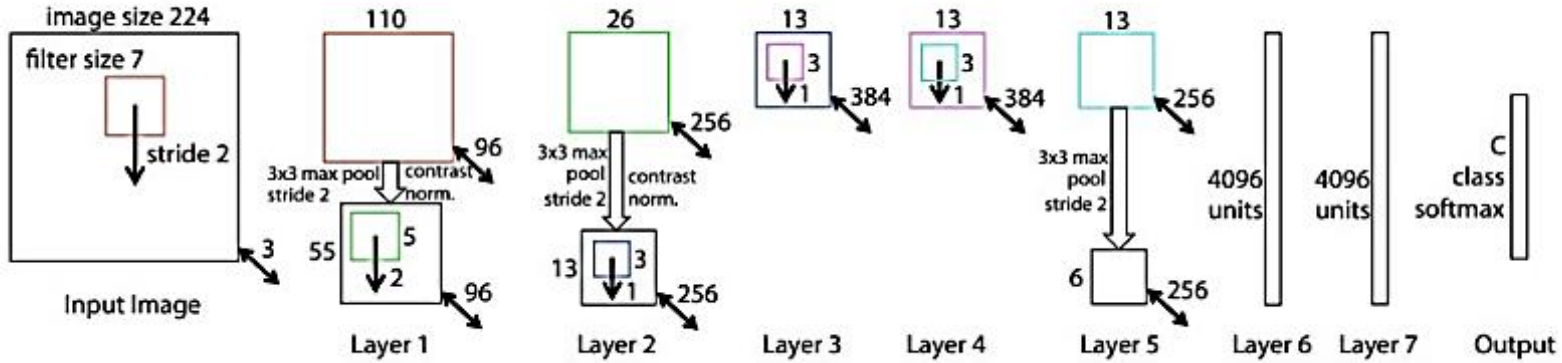


It is based on the Zeiler and Fergus model, which was trained on the ImageNet dataset.

ZFNet – Deep Learning Architecture that popularized CNN

ZF Net CNN architecture consists of a total of seven layers: Convolutional layer, max-pooling layer (downscaling), concatenation layer, convolutional layer with linear activation function, and stride one, dropout for regularization purposes applied before the fully connected output.

This CNN model is computationally more efficient than AlexNet by introducing an approximate inference stage through deconvolutional layers in the middle of CNNs.



ZFNet – Deep Learning Architecture that popularized CNN

GoogLeNet – CNN Architecture used by Google

GoogLeNet is the CNN architecture used by Google to win ILSVRC 2014 classification task.

It was developed by Jeff Dean, Christian Szegedy, Alexandro Szegedy et al.

It has been shown to have a notably reduced error rate in comparison with previous winners AlexNet (Ilsvrc 2012 winner) and ZF-Net (Ilsvrc 2013 winner).

In terms of error rate, the error is significantly lesser than VGG (2014 runner up).

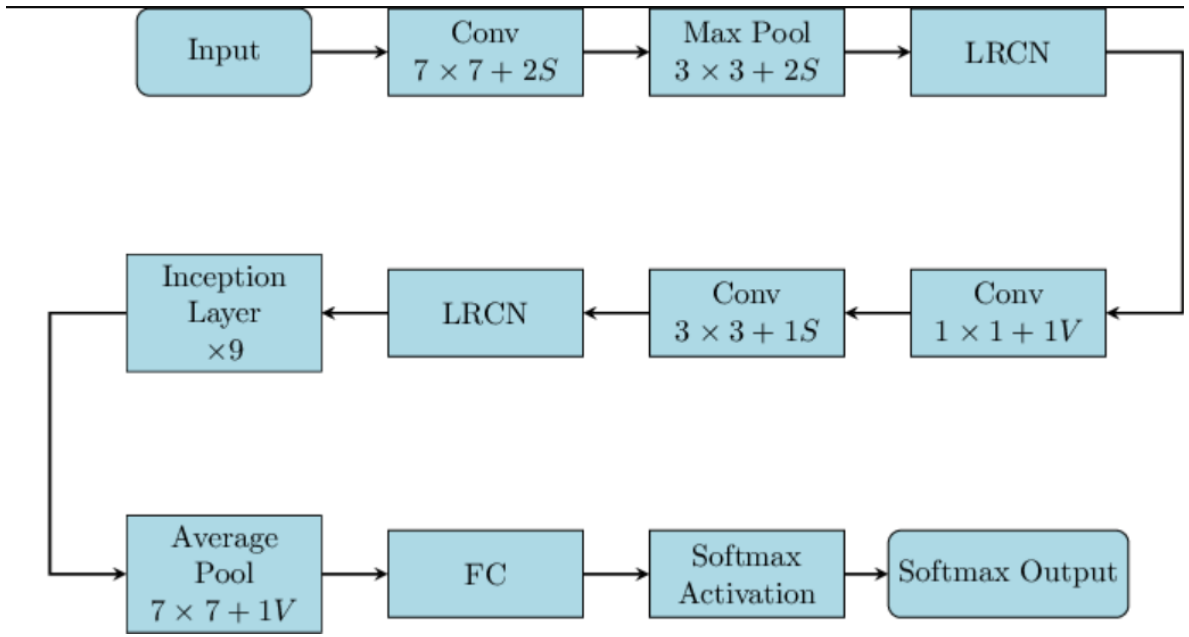
It achieves deeper architecture by employing a number of distinct techniques, including 1×1 convolution and global average pooling.

GoogleNet CNN architecture is computationally expensive.

GoogLeNet – CNN Architecture used by Google

To reduce the parameters that must be learned, it uses heavy unpooling layers on top of CNNs to remove spatial redundancy during training and also features shortcut connections between the first two convolutional layers before adding new filters in later CNN layers.

Real-world applications/examples of GoogLeNet CNN architecture include Street View House Number (SVHN) digit recognition task, which is often used as a proxy for roadside object detection.



GoogLeNet – CNN Architecture used by Google

VGGNet – CNN Architecture with Large Filters

VGGNet is the CNN architecture that was developed by Karen Simonyan, Andrew Zisserman et al. at Oxford University.

The full form of “VGG16” stands for “Visual Geometry Group 16“. This name comes from the Visual Geometry Group at the University of Oxford, where this neural network architecture was developed.

The “16” in the name indicates that the model contains 16 layers that have weights; this includes convolutional layers as well as fully connected layers.

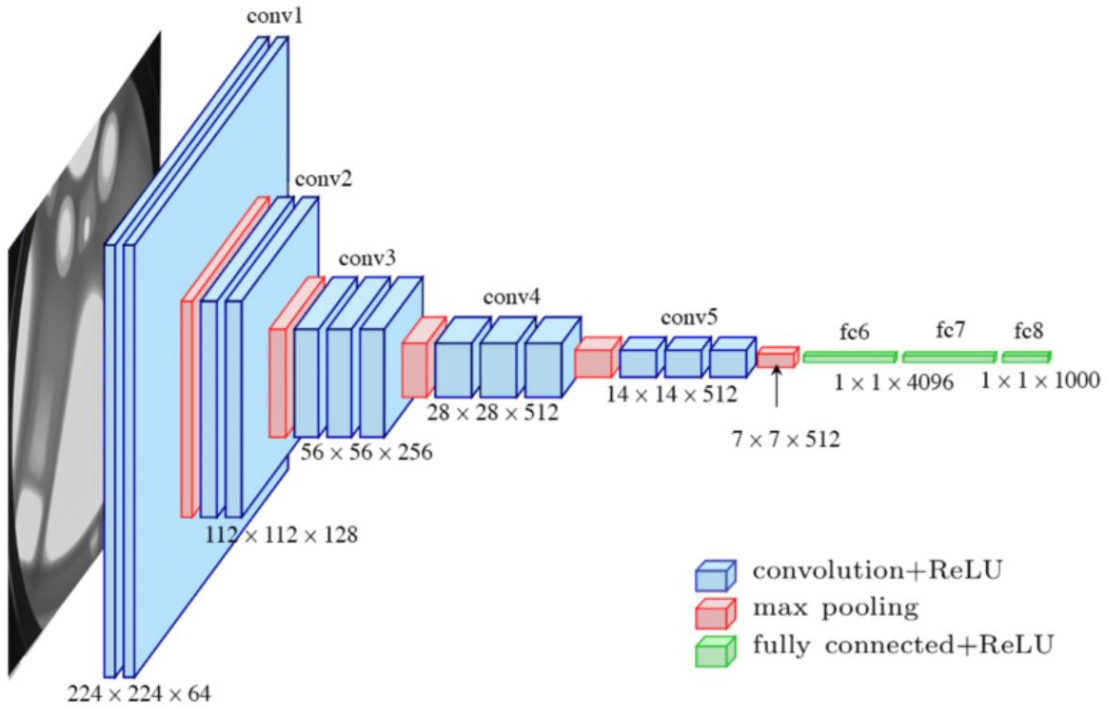
VGGNet is a 16-layer CNN with up to 95 million parameters and trained on over one billion images (1000 classes).

It can take large input images of 224 x 224-pixel size for which it has 4096 convolutional features.

VGGNet – CNN Architecture with Large Filters

- CNNs with such large filters are expensive to train and require a lot of data, which is the main reason why CNN architectures like GoogLeNet (AlexNet architecture) work better than VGGNet for most image classification tasks where input images have a size between 100 x 100-pixel and 350 x 350 pixels.
- Real-world applications/examples of VGGNet CNN architecture include the ILSVRC 2014 classification task, which was also won by GoogleNet CNN architecture.
- The VGG CNN model is computationally efficient and serves as a strong baseline for many applications in computer vision due to its applicability for numerous tasks including object detection.
- Its deep feature representations are used across multiple neural network architectures like YOLO, SSD, etc.

VGGNet – CNN Architecture with Large Filters



ResNet – CNN Architecture That Also Got Used for NLP Tasks apart from Image Classification

ResNet is the CNN architecture that was developed by Kaiming He et al. to win the ILSVRC 2015 classification task with a top-five error of only 15.43%.

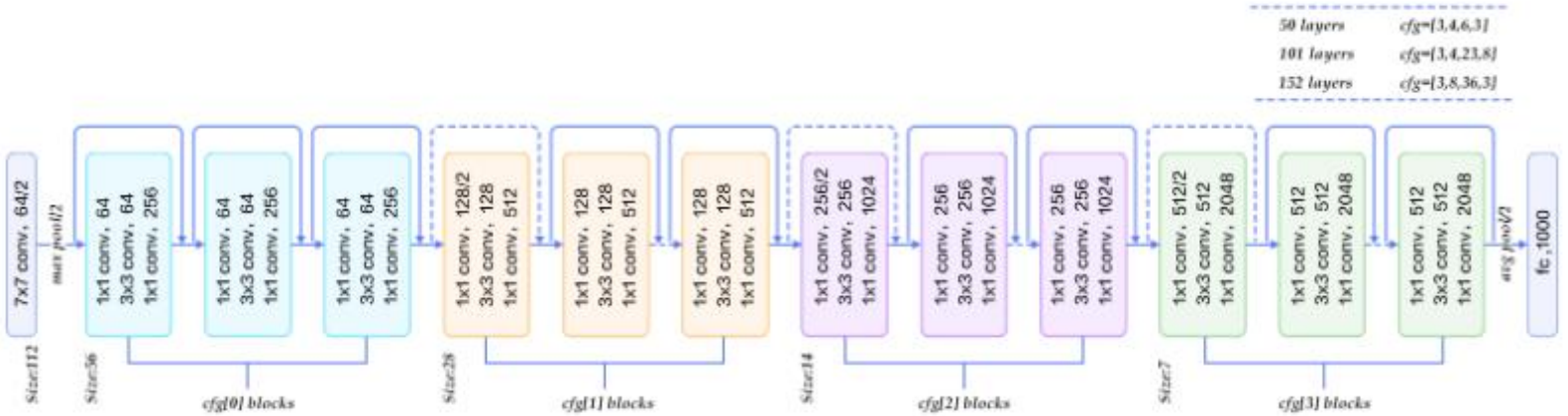
The network has 152 layers and over one million parameters, which is considered deep even for CNNs because it would have taken more than 40 days on 32 GPUs to train the network on the ILSVRC 2015 dataset.

CNNs are mostly used for image classification tasks with 1000 classes, but ResNet proves that CNNs can also be used successfully to solve natural language processing problems like sentence completion or machine comprehension, where it was used by the Microsoft Research Asia team in 2016 and 2017 respectively.

Real-life applications/examples of ResNet CNN architecture include Microsoft's machine comprehension system, which has used CNNs to generate the answers for more than 100k questions in over 20 categories.

The CNN architecture ResNet is computationally efficient and can be scaled up or down to match the computational power of GPUs.

ResNet – CNN Architecture That Also Got Used for NLP Tasks apart from Image Classification



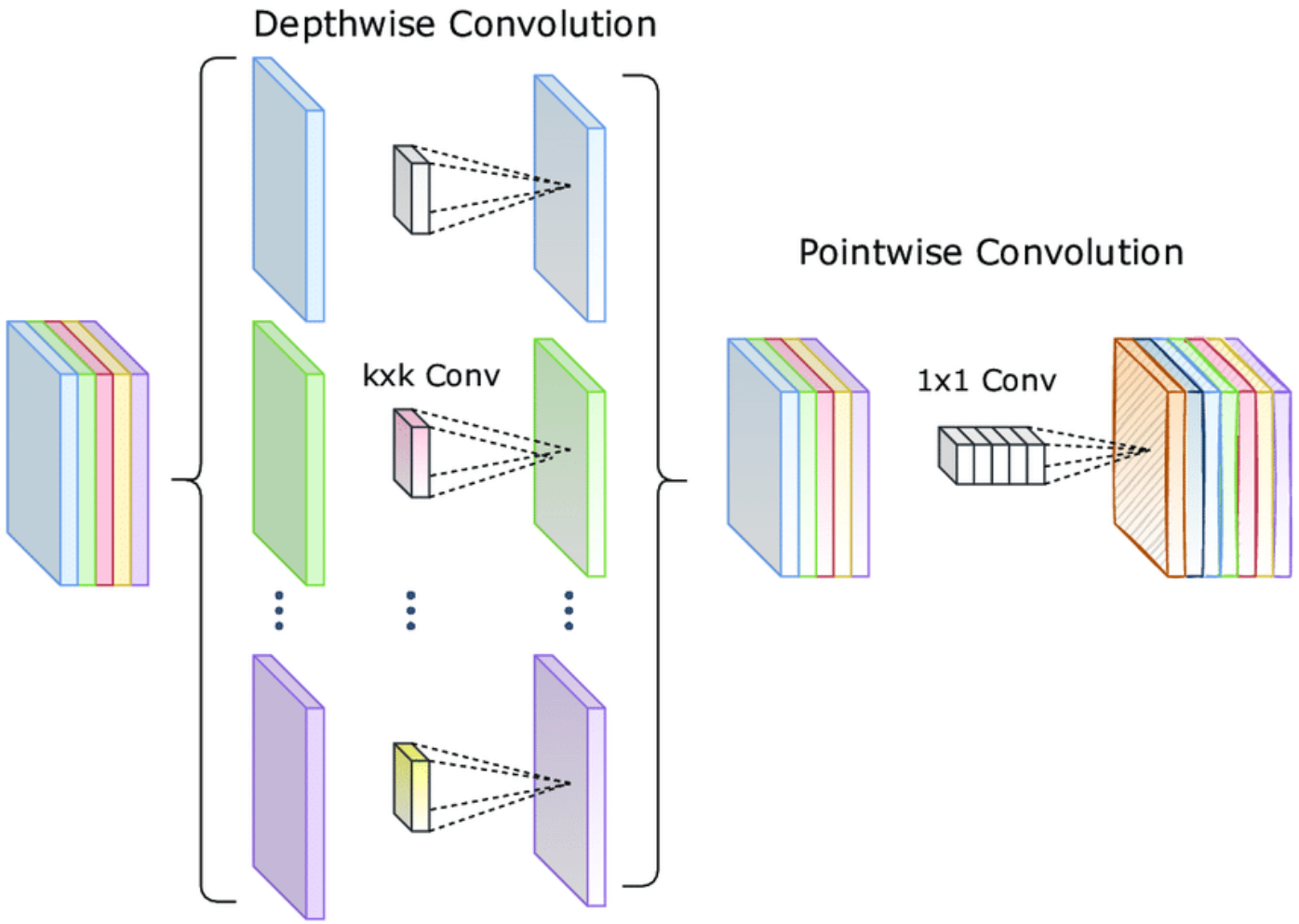
MobileNets – CNN Architecture for Mobile Devices

MobileNets are CNNs that can be fit on a mobile device to classify images or detect objects with low latency.

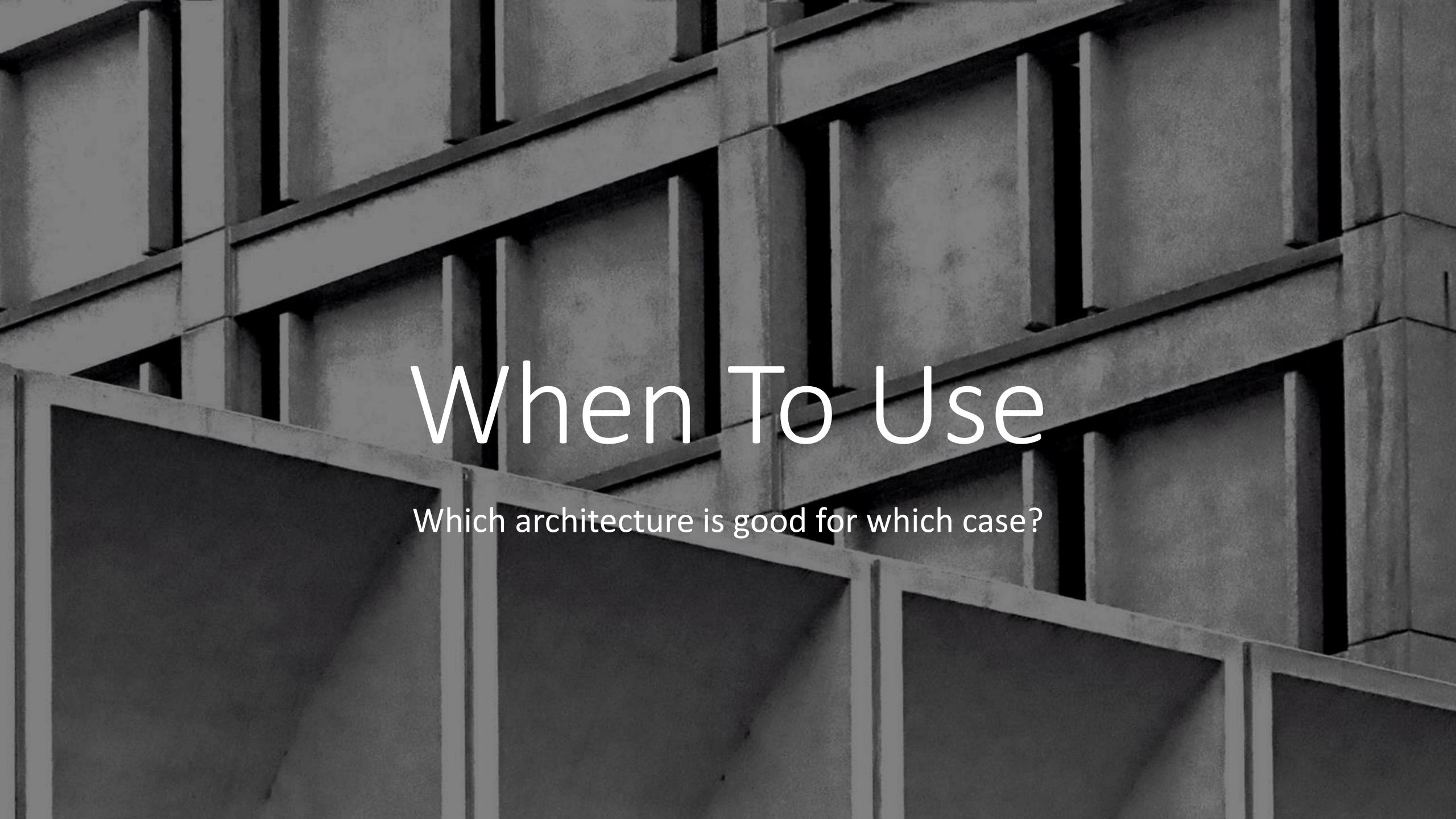
MobileNets have been developed by Andrew G Trillion et al.. They are usually very small CNN architectures, which makes them easy to run in real-time using embedded devices like smartphones and drones.

The architecture is also flexible, so it has been tested on CNNs with 100-300 layers and it still works better than other architectures like VGGNet.

Real-life examples of MobileNets CNN architecture include CNNs that is built into Android phones to run Google's Mobile Vision API, which can automatically identify labels of popular objects in images.



MobileNets – CNN Architecture for Mobile Devices



When To Use

Which architecture is good for which case?

When Should We Use a Traditional CNN?

- Use when the image classification problem is **relatively simple**.
- Suitable when the dataset is **small or medium-sized**.
- Good when images contain clear spatial patterns such as shapes, edges and textures.
- Suitable for learning CNN concepts or developing a **baseline model**.
- Can be used for simple binary or multi-class image classification.
- Example tasks include basic defect detection, simple medical image classification and object-category recognition.
- Avoid unnecessarily deep CNNs when the problem can be solved with a simpler model.

When Should We Use LeNet?

- Use for **simple images with relatively low complexity**.
- Particularly suitable for **handwritten digit and character recognition**.
- Works well with small grayscale images such as handwritten numbers.
- Suitable when computational resources are limited.
- Useful when the number of classes and visual variations are relatively small.
- Good for teaching or experimenting with the basic CNN architecture.
- Example tasks include digit recognition, character recognition and simple document-symbol classification.

When Should We Use AlexNet?

- Use when the image problem is **more complex than problems suited to LeNet**.
- Suitable for larger image datasets containing many object categories.
- Useful when the model must learn more complex visual features.
- Can be used for general-purpose **image classification**.
- Suitable when sufficient computational resources are available.
- Historically important for understanding how deeper CNNs improved large-scale image recognition.
- For modern applications, newer architectures may provide better efficiency and accuracy.

When Should We Use ZFNet?

- Use when an **AlexNet-style architecture needs improvement or refinement**.
- Suitable for large-scale image classification problems.
- Useful when better feature extraction from early convolutional layers is required.
- Smaller filters and strides can help preserve more detailed visual information.
- Suitable when understanding **what CNN layers are learning** is important.
- Useful for studying and improving CNN architecture design.
- Example tasks include object recognition and large-scale image classification.

When Should We Use GoogLeNet?

- Use when a **deep CNN is required without an extremely large parameter count**.
- Suitable for complex large-scale image classification.
- Useful when objects appear at **different sizes or scales** within images.
- Its architecture can extract different types of features at the same stage.
- Suitable when both model depth and computational efficiency are important.
- Can be applied to object recognition and visual recognition problems.
- Example applications include large-scale classification and digit or roadside-object recognition.

When Should We Use VGGNet?

- Use when **strong and detailed visual feature extraction** is important.
- Suitable for transfer learning when pretrained VGG models are available.
- Useful as a feature extractor for other computer-vision models.
- Can support image classification and object-detection applications.
- Its simple repeated architecture makes it relatively easy to understand and modify.
- Suitable when computational resources and memory are not major limitations.
- Avoid it for highly resource-constrained applications because VGG models can be large and computationally demanding.

When Should We Use ResNet?

- Use when the problem requires a **very deep neural network**.
- Suitable for complex images requiring highly detailed feature learning.
- Residual connections make very deep networks easier to train.
- Strong choice for image classification, object detection and other computer-vision tasks.
- Very useful for **transfer learning** because pretrained ResNet models are widely applicable.
- Suitable when high predictive performance is more important than having an extremely small model.
- ResNet can be scaled using different depths depending on the available computational resources.

When Should We Use MobileNet?

- Use when the model must run on **resource-constrained devices**.
- Particularly suitable for smartphones, embedded systems and edge devices.
- Good when **low latency and fast inference** are important.
- Suitable when memory, processing power or battery usage is limited.
- Can perform image classification and object detection.
- Useful for real-time applications involving cameras, robots, drones and mobile devices.
- Choose MobileNet when **efficiency and portability are more important than using a very large CNN**.



Transfer Learning

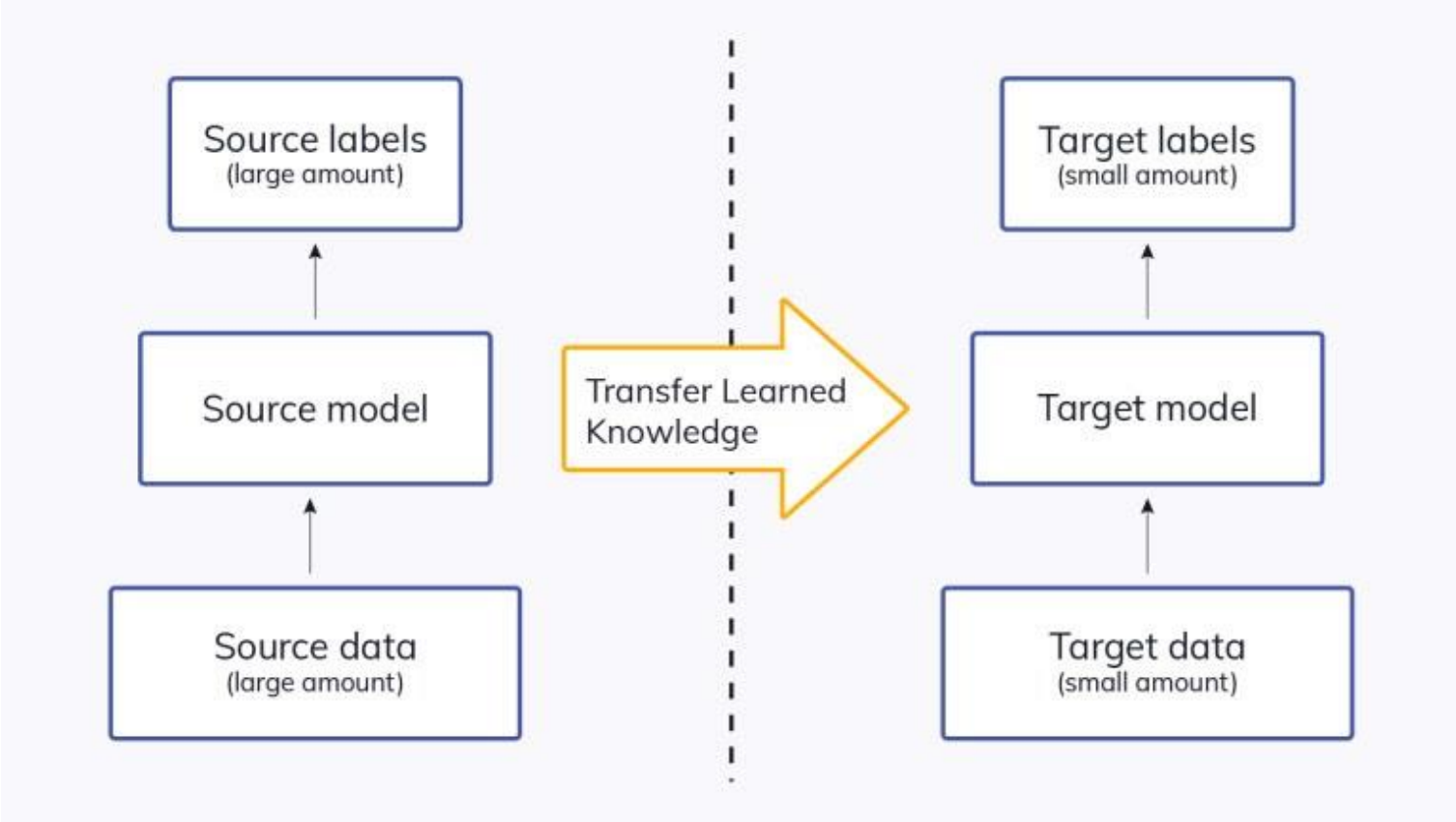
Samadhi Chathuranga Rathnayake

MSc, MBA, PGDip, BSc (1st Class)

What is Transfer Learning

- A model trained on one task is repurposed on a second, related task as an optimization that allows rapid progress when modeling the second task.
- By applying transfer learning to a new task, one can achieve significantly higher performance than training with only a small amount of data.
- Transfer learning is about leveraging feature representations from a pre-trained model, so you don’t have to train a new model from scratch.
- The pre-trained models are usually trained on massive datasets that are a standard benchmark in the computer vision frontier.
- The weights obtained from the models can be reused in other tasks.

What is Transfer Learning



Traditional Machine Learning VS Transfer Learning

Traditional machine learning models require training from scratch, which is computationally expensive and requires a large amount of data to achieve high performance.

On the other hand, transfer learning is computationally efficient and helps achieve better results using a small data set.

Traditional Machine Learning VS Transfer Learning

Traditional ML has an isolated training approach where each model is independently trained for a specific purpose, without any dependency on past knowledge.

Contrary to that, transfer learning uses knowledge acquired from the pre-trained model to proceed with the task.

Traditional Machine Learning VS Transfer Learning



Transfer learning models achieve optimal performance faster than the traditional ML models.



It is because the models that leverage knowledge (features, weights, etc.) from previously trained models already understand the features.

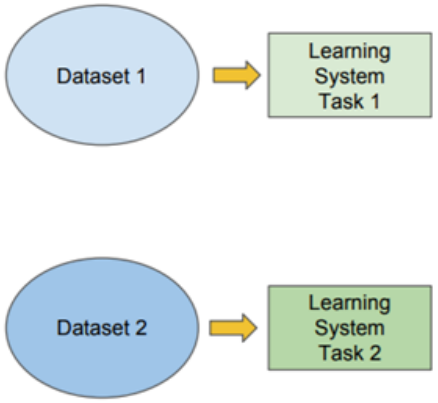


It makes it faster than training neural networks from scratch.

Traditional Machine Learning VS Transfer Learning

Traditional ML

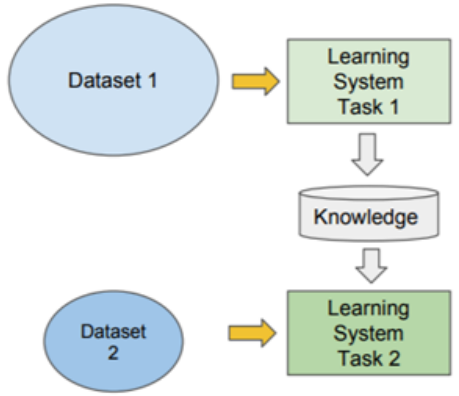
- Isolated, single task learning:
  - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



vs

Transfer Learning

- Learning of a new tasks relies on the previous learned tasks:
  - Learning process can be faster, more accurate and/or need less training data



Classical Transfer Learning Strategies

Different transfer learning strategies and techniques are applied based on the domain of the application, the task at hand, and the availability of data.

Before deciding on the strategy of transfer learning, it is crucial to have an answer of the following questions:

- **Which** part of the knowledge can be transferred from the source to the target to improve the performance of the target task?
- **When** to transfer and when not to, so that one improves the target task performance/results and does not degrade them?
- **How** to transfer the knowledge gained from the source model based on our current domain/task?

Traditionally, transfer learning strategies fall under three major categories depending upon the task domain and the amount of labeled/unlabeled data present.

- Inductive Transfer Learning
- Transductive Transfer Learning
- Unsupervised Transfer Learning

Inductive Transfer Learning

- Inductive Transfer Learning requires the source and target domains to be the same, though the specific tasks the model is working on are different.
- The algorithms try to use the knowledge from the source model and apply it to improve the target task.
- The pre-trained model already has expertise on the features of the domain and is at a better starting point than if we were to train it from scratch.
- Inductive transfer learning is further divided into two subcategories depending upon whether the source domain contains labeled data or not.
- These include multi-task learning and self-taught learning, respectively.
- Ex-An example of inductive transfer learning is training a model for object detection in one domain (e.g., cars in traffic images) and then transferring the learned knowledge to detect objects in a different domain (e.g., pedestrians in urban settings). Although the tasks differ, the general ability to detect objects can be transferred.

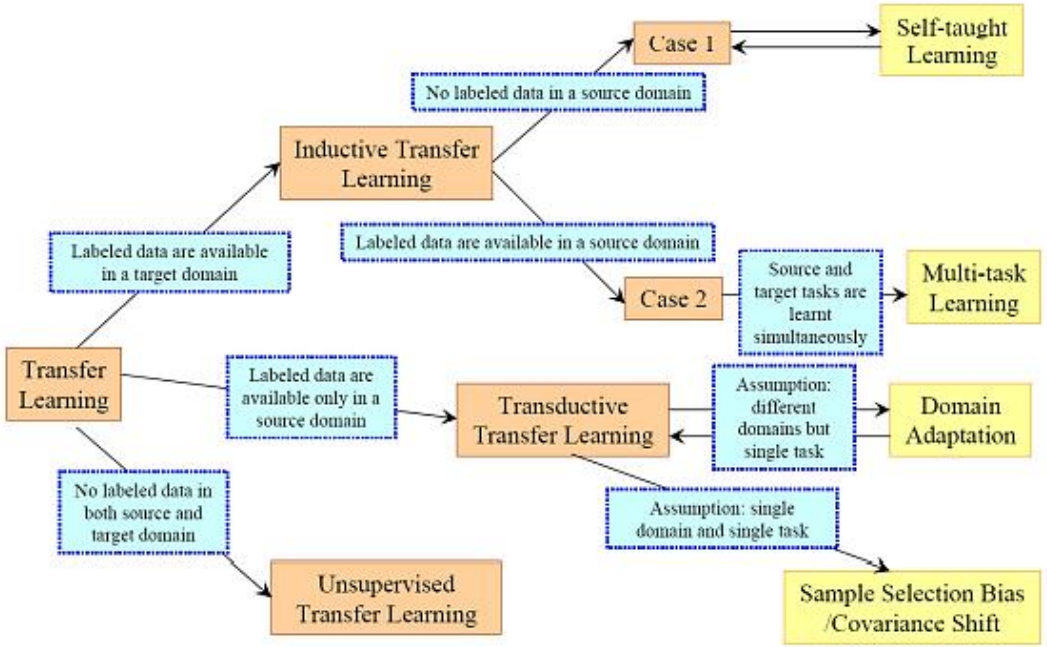
Transductive Transfer Learning

- Scenarios where the domains of the source and target tasks are not exactly the same but interrelated uses the Transductive Transfer Learning strategy.
- One can derive similarities between the source and target tasks.
- These scenarios usually have a lot of labeled data in the source domain, while the target domain has only unlabeled data.
- Ex-An example of transductive transfer learning could be using a sentiment analysis model trained on English movie reviews to classify sentiments in French movie reviews. Although the task (sentiment analysis) is the same, the domains (languages) differ, and the model needs to adapt its learned knowledge to perform well on the target domain (French).

Unsupervised Transfer Learning

- Unsupervised Transfer Learning is similar to Inductive Transfer learning.
- The only difference is that the algorithms focus on unsupervised tasks and involve unlabeled datasets both in the source and target tasks.
- Ex- An example of unsupervised transfer learning is using a model trained on labeled images of objects (source domain) to perform clustering on a different set of unlabeled images (target domain). Here, the learned representations or features from the source task help the model group similar objects together in the target domain, even though the target data has no labels.

Classical Transfer Learning Strategies



Approaches of Transfer Learning

- Now, we'll go through another way of categorizing transfer learning strategies based on the similarity of the domain and independent of the type of data samples present for training.
  - Homogeneous Transfer Learning
  - Heterogeneous Transfer Learning

Homogeneous Transfer Learning

- Homogeneous Transfer learning approaches are developed and proposed to handle situations where the domains are of the same feature space.
- In Homogeneous Transfer learning, domains have only a slight difference in marginal distributions.
- These approaches adapt the domains by correcting the sample selection bias or covariate shift.

Instance Transfer

- It covers a simple scenario in which there is a large amount of labeled data in the source domain and a limited number in the target domain.
- Both the domains and feature spaces differ only in marginal distributions.
- For example, suppose we need to build a model to diagnose cancer in a specific region where the elderly are the majority.
- Limited target-domain instances are given, and relevant data are available from another region where young people are the majority.
- Directly transferring all the data from another region may be unsuccessful since the marginal distribution difference exists, and the elderly have a higher risk of cancer than younger people.
- In this scenario, it is natural to consider adapting the marginal distributions.
- Instance-based Transfer learning reassigns weights to the source domain instances in the loss function.

Parameter Transfer

- The parameter-based transfer learning approaches transfer the knowledge at the model/parameter level.
- This approach involves transferring knowledge through the shared parameters of the source and target domain learner models.
- One way to transfer the learned knowledge can be by creating multiple source learner models and optimally combining the re-weighted learners similar to ensemble learners to form an improved target learner.
- The idea behind parameter-based methods is that a well-trained model on the source domain has learned a well-defined structure, and if two tasks are related, this structure can be transferred to the target model.

Parameter Transfer

- In general, there are two ways to share the weights in deep learning models: soft weight sharing and hard weight sharing.
  - In **soft weight sharing**, the model is expected to be close to the already learned features and is usually penalized if its weights deviate significantly from a given set of weights.
  - In **hard weight sharing**, we share the exact weights among different models.

Feature-Representation Transfer

- Feature-based approaches transform the original features to create a new feature representation.
- This approach can further be divided into two subcategories, i.e., asymmetric and symmetric Feature-based Transfer Learning.
  - **Asymmetric approaches** transform the source features to match the target ones. In other words, we take the features from the source domain and fit them into the target feature space. There can be some information loss in this process due to the marginal difference in the feature distribution.
  - **Symmetric approaches** find a common latent feature space and then transform both the source and the target features into this new feature representation.

Relational-Knowledge Transfer

- Relational-based transfer learning approaches mainly focus on learning the relations between the source and a target domain and using this knowledge to derive past knowledge and use it in the current context.
- Such approaches transfer the logical relationship or rules learned in the source domain to the target domain.
- For example, if we learn the relationship between different elements of the speech in a male voice, it can help significantly to analyze the sentence in another voice.

Heterogeneous Transfer Learning



Transfer learning involves deriving representations from a previous network to extract meaningful features from new samples for an inter-related task.



However, these approaches forget to account for the difference in the feature spaces between the source and target domains.



It is often challenging to collect labeled source domain data with the same feature space as the target domain, and Heterogeneous Transfer learning methods are developed to address such limitations.



This technique aims to solve the issue of source and target domains having differing feature spaces and other concerns like differing data distributions and label spaces.



Heterogeneous Transfer Learning is applied in cross-domain tasks such as cross-language text categorization, text-to-image classification, and many others.

Transfer Learning for Deep Learning



Off-the-shelf Pre-trained Models as Feature Extractors

Deep learning systems are layered architectures that learn different features at different layers.

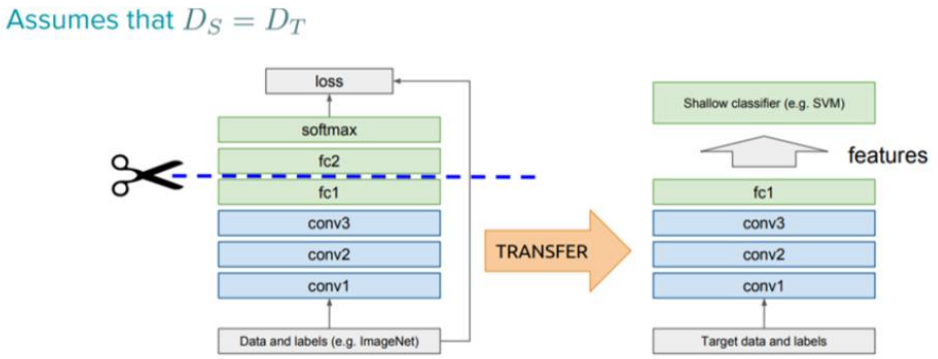
Initial layers compile higher-level features that narrow down to fine-grained features as we go deeper into the network.

These layers are finally connected to the last layer (usually a fully connected layer, in the case of supervised learning) to get the final output.

This opens the scope of using popular pre-trained networks (such as Oxford VGG Model, Google Inception Model, Microsoft ResNet Model) without its final layer as a fixed feature extractor for other tasks.

Off-the-shelf Pre-trained Models as Feature Extractors

- The key idea here is to leverage the pre-trained model's weighted layers to extract features, but not update the model's weights during training with new data for the new task.
- The pre-trained models are trained on a large and general enough dataset and will effectively serve as a generic model of the visual world.



Fine Tuning Off-the-shelf Pre-trained Models

- This is a more engaging technique, where we do not just directly depend on the features extracted from the pre-trained models and replace the final layer but also selectively retrain some of the previous layers.
- Deep neural networks are layered structures and have many tunable hyperparameters.
- The role of the initial layers is to capture generic features, while the later ones focus more on the explicit task at hand.
- It makes sense to fine-tune the higher-order feature representations in the base model to make them more relevant for the specific task.
- We can re-train some layers of the model while keeping some frozen in training.

Fine Tuning Off-the-shelf Pre-trained Models

- An example is depicted in the following figure on an object detection task, where initial lower layers of the network learn very generic features and the higher layers learn very task-specific features.

Fine-tuning: supervised domain adaptation

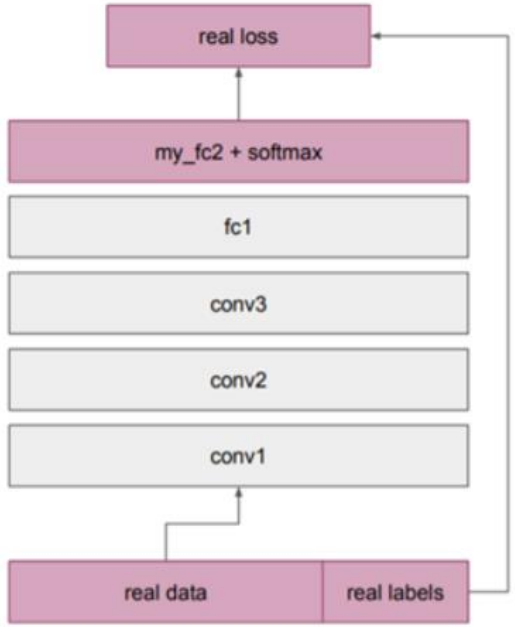
Train deep net on "nearby" task for which it is easy to get labels using standard backprop

- E.g. ImageNet classification
- Pseudo classes from augmented data
- Slow feature learning, ego-motion

Cut off top layer(s) of network and replace with supervised objective for target domain

Fine-tune network using backprop with labels for target domain until validation loss starts to increase

Aligns D_S with D_T





Freezing vs. Fine-tuning

- One logical way to increase the model's performance even further is to re-train (or "fine-tune") the weights of the top layers of the pre-trained model alongside the training of the classifier you added.
- This will force the weights to be updated from generic feature maps the model has learned from the source task.
- Fine-tuning will allow the model to apply past knowledge in the target domain and re-learn some things again.
- Moreover, one should try to fine-tune a small number of top layers rather than the entire model.
- The first few layers learn elementary and generic features that generalize to almost all types of data.



Freezing vs. Fine-tuning

- Therefore, it's wise to freeze these layers and reuse the basic knowledge derived from the past training.
- As we go higher up, the features are increasingly more specific to the dataset on which the model was trained.
- Fine-tuning aims to adapt these specialized features to work with the new dataset, rather than overwrite the generic learning.

Freezing vs. Fine-tuning

Freeze or fine-tune?

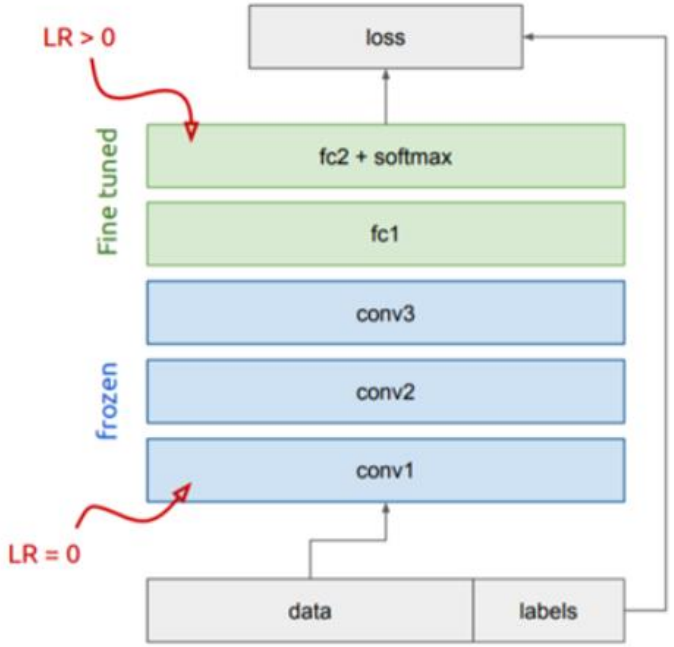
Bottom *n* layers can be frozen or fine tuned.

- **Frozen**: not updated during backprop
- **Fine-tuned**: updated during backprop

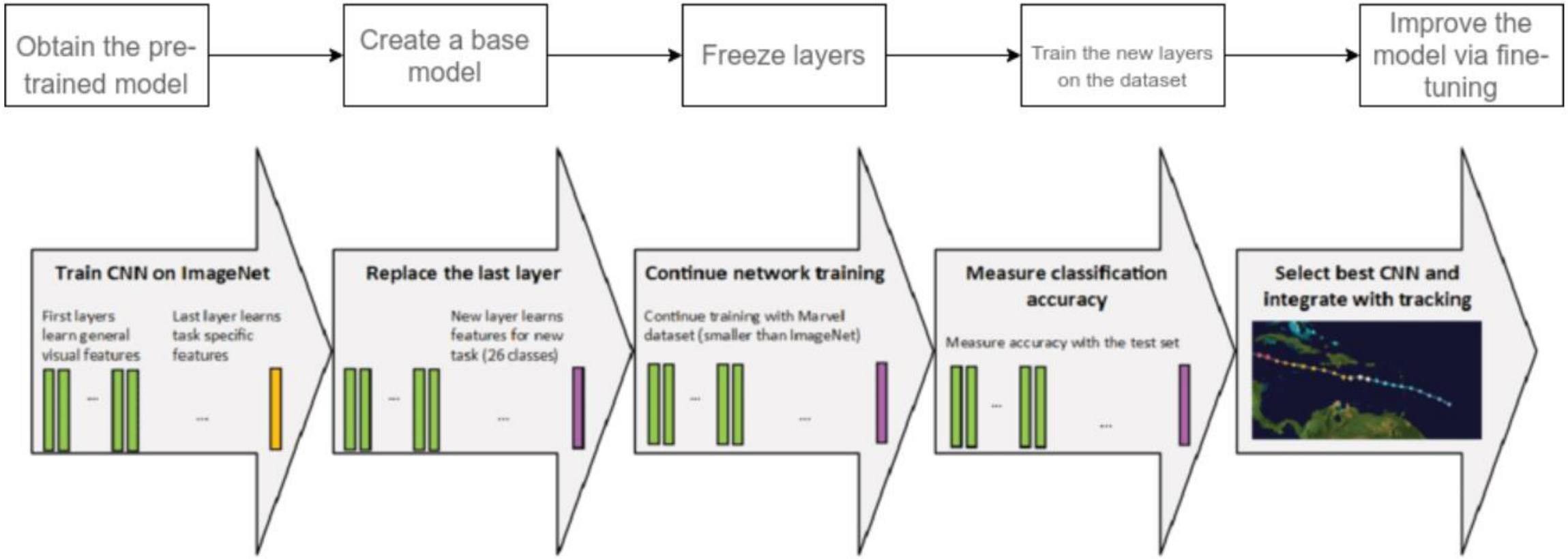
Which to do depends on target task:

- **Freeze**: target task labels are scarce, and we want to avoid overfitting
- **Fine-tune**: target task labels are more plentiful

In general, we can set learning rates to be different for each layer to find a tradeoff between freezing and fine tuning



Steps of Transfer Learning



Obtain pre-trained model

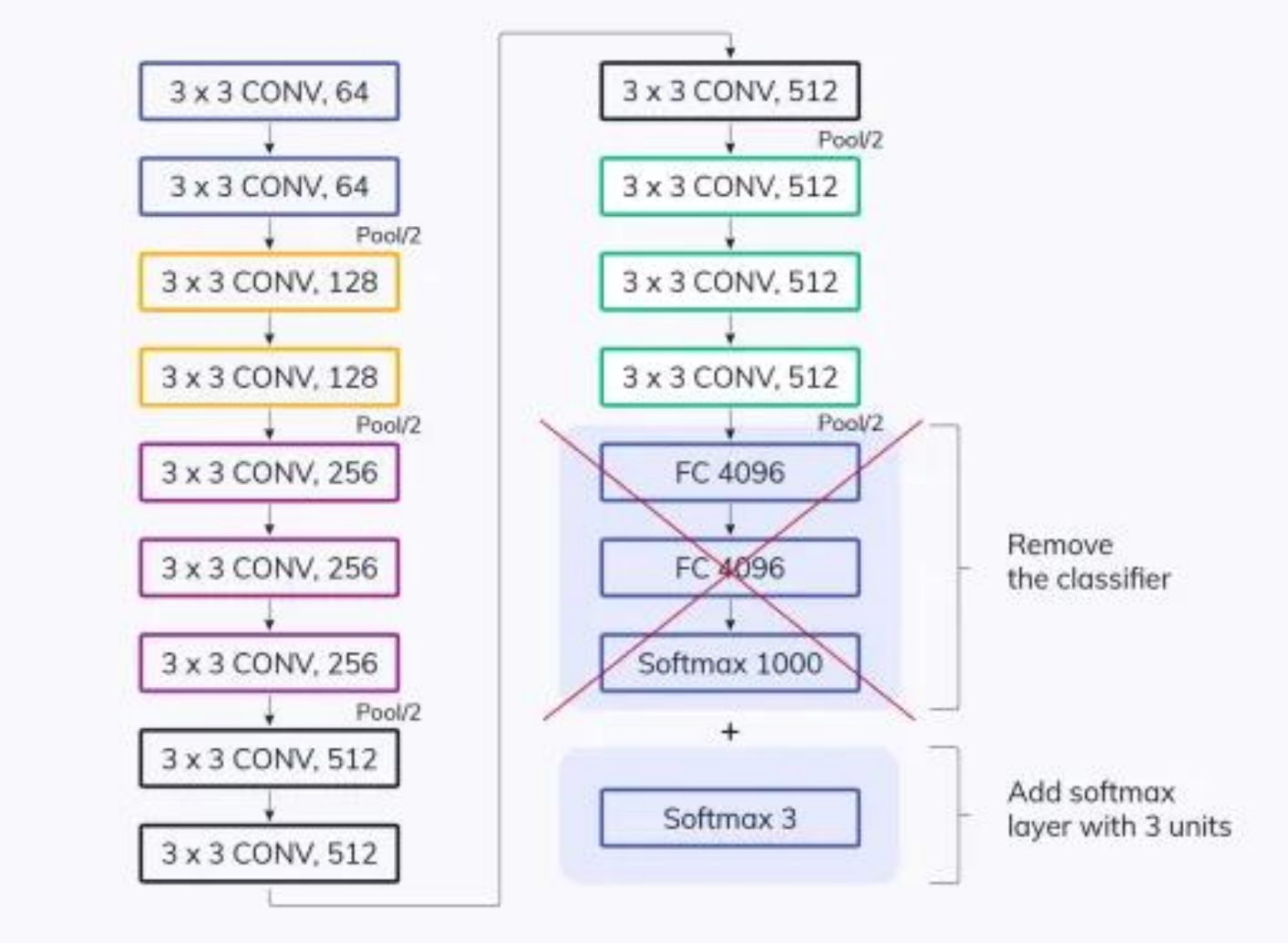
- The first step is to choose the pre-trained model we would like to keep as the base of our training, depending on the task.
- Transfer learning requires a strong correlation between the knowledge of the pre-trained source model and the target task domain for them to be compatible.
- For computer vision:
  - VGG-16
  - VGG-19
  - Inception V3
  - XCeption
  - ResNet-50
- For NLP tasks:
  - Word2Vec
  - GloVe
  - FastText

Create a base model

- The base model is one of the architectures such as ResNet or Xception which we have selected in the first step to be in close relation to our task.
- We can either download the network weights which saves the time of additional training of the model.
- Else, we will have to use the network architecture to train our model from scratch.
- There can be a case where the base model will have more neurons in the final output layer than we require in our use case.
- In such scenarios, we need to remove the final output layer and change it accordingly.



Create a base model



Freeze layers

Freezing the starting layers from the pre-trained model is essential to avoid the additional work of making the model learn the basic features.

If we do not freeze the initial layers, we will lose all the learning that has already taken place.

This will be no different from training the model from scratch and will be a loss of time, resources, etc.

Add new trainable layers

The only knowledge we are reusing from the base model is the feature extraction layers.

We need to add additional layers on top of them to predict the specialized tasks of the model.

These are generally the final output layers.

Train the new layers

The pre-trained model's final output will most likely differ from the output we want for our model.

For example, pre-trained models trained on the ImageNet dataset will output 1000 classes.

However, we need our model to work for two classes.

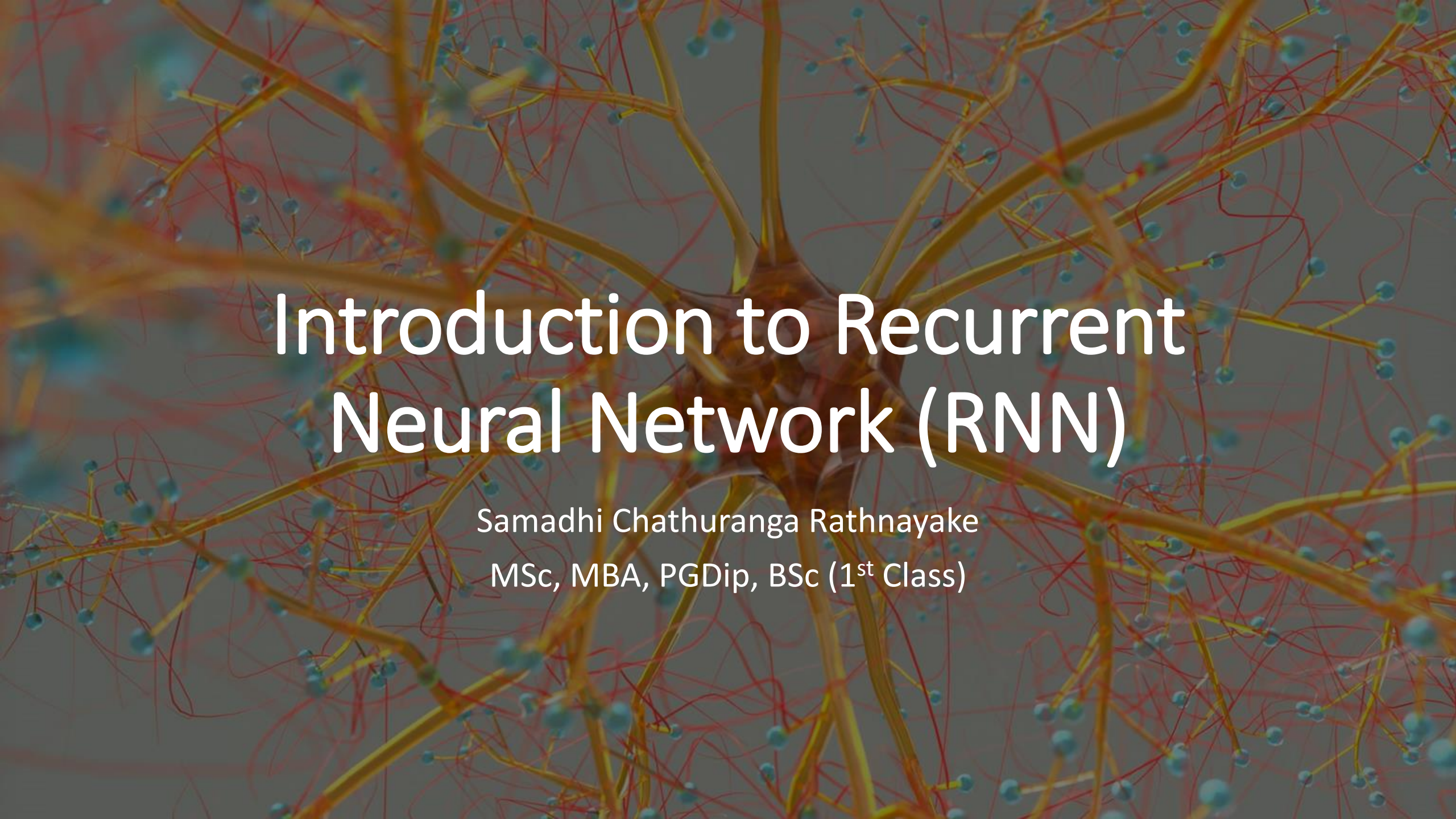
In this case, we have to train the model with a new output layer in place.

Fine-tune your model

One method of improving the performance is fine-tuning.

Fine-tuning involves unfreezing some part of the base model and training the entire model again on the whole dataset at a very low learning rate.

The low learning rate will increase the performance of the model on the new dataset while preventing overfitting.



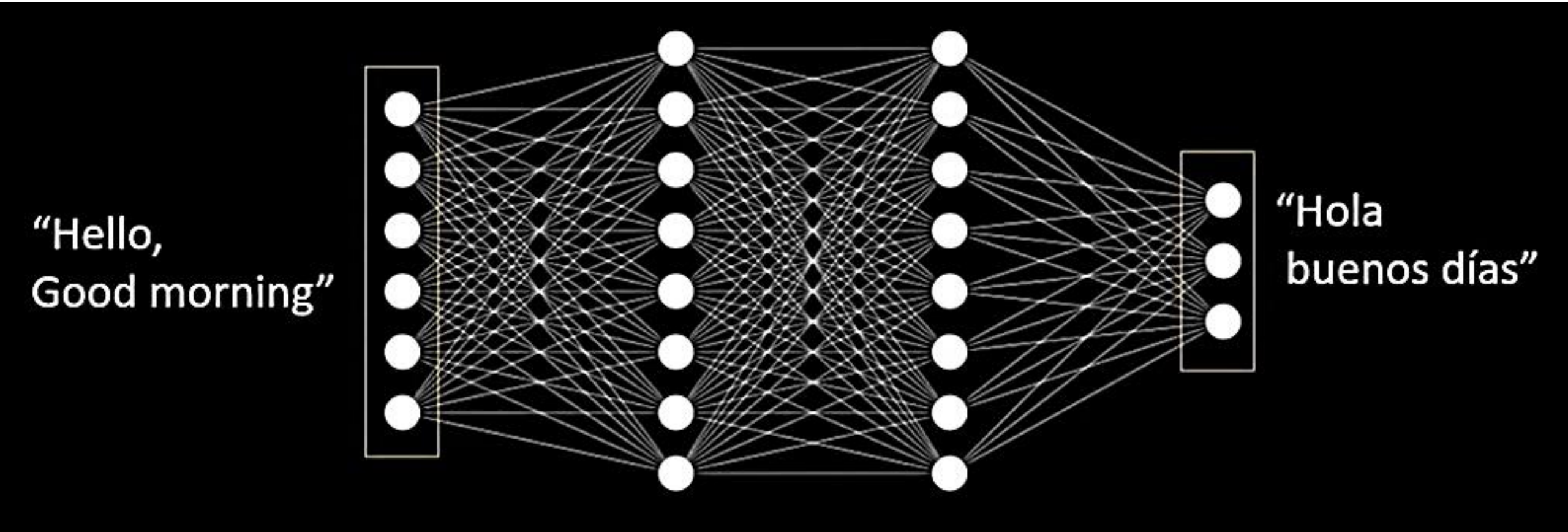
Introduction to Recurrent Neural Network (RNN)

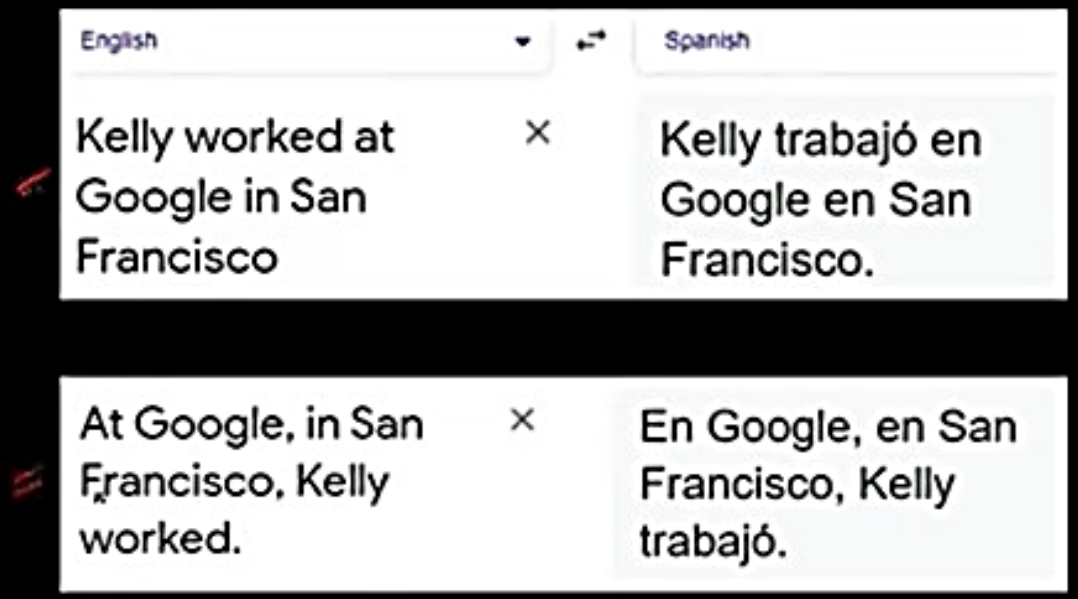
Samadhi Chathuranga Rathnayake

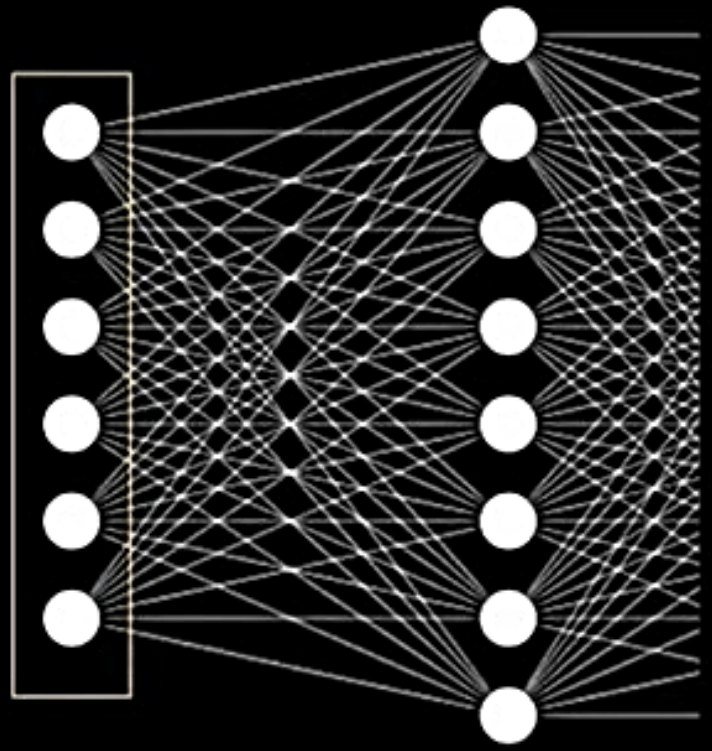
MSc, MBA, PGDip, BSc (1st Class)

Text Data in Feedforward Neural Networks

- In the feedforward networks, there is a fixed length for the inputs





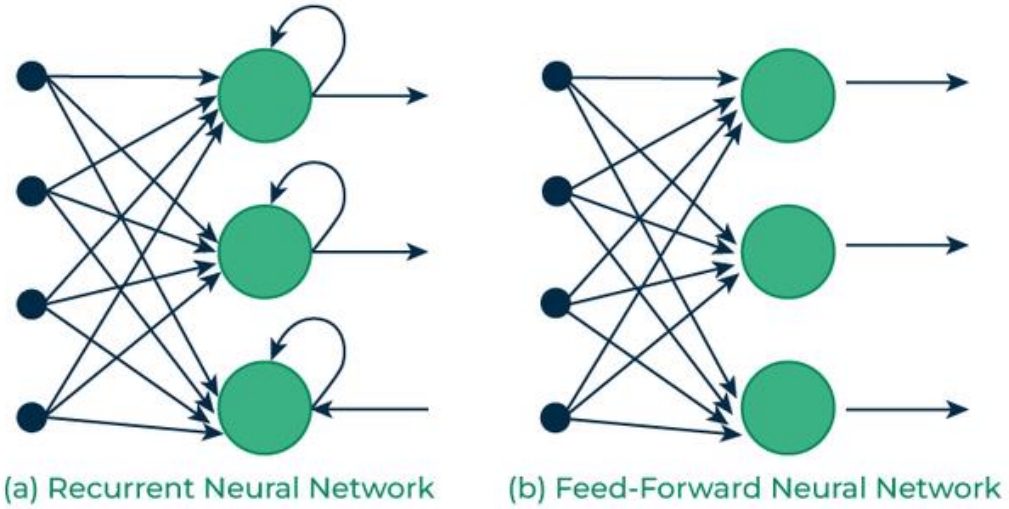


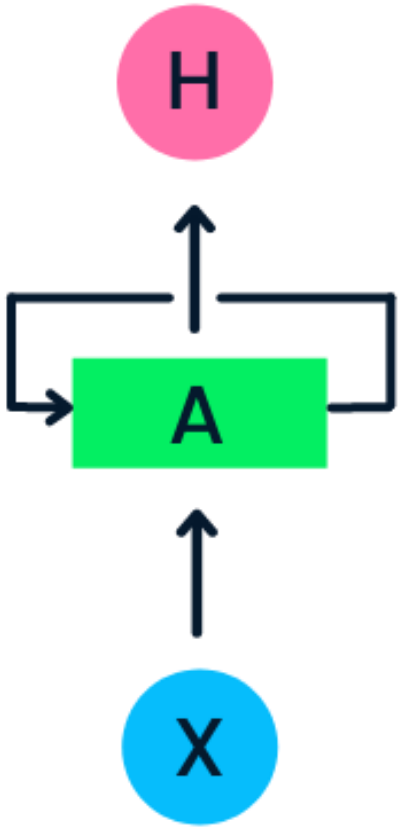
Text Data in Feedforward Neural Networks

- The order is not considered in the feedforward networks

Introduction to Recurrent Neural Networks

- A partial output of the activation functions of the hidden layer will be taken as an input for the next iteration



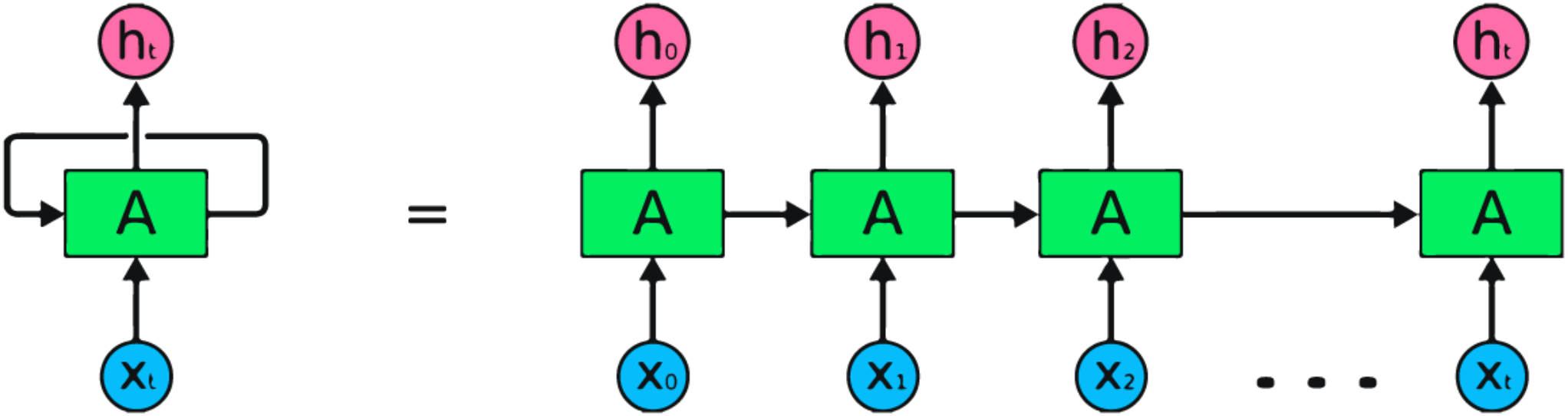


How Recurrent Neural Networks Work?

- In RNN, the information cycles through the loop, so the output is determined by the current input and previously received inputs.

How Recurrent Neural Networks Work?

- In a traditional neural network, you might have multiple hidden layers between the input and output layers, and each hidden layer would have its own separate parameters (like weights and biases).
- However, in a Recurrent Neural Network (RNN), things work a bit differently, especially when processing sequences of data (like time steps in a series or words in a sentence).



How Recurrent Neural Networks Work?

Single Input (X):

- Imagine that the input (X) is not just one piece of data but a sequence.
- For example, if you're processing a sentence, each word in the sentence is part of that sequence.
- At each time step in the sequence, the RNN processes one piece of input (like a word or a time step in a time series).

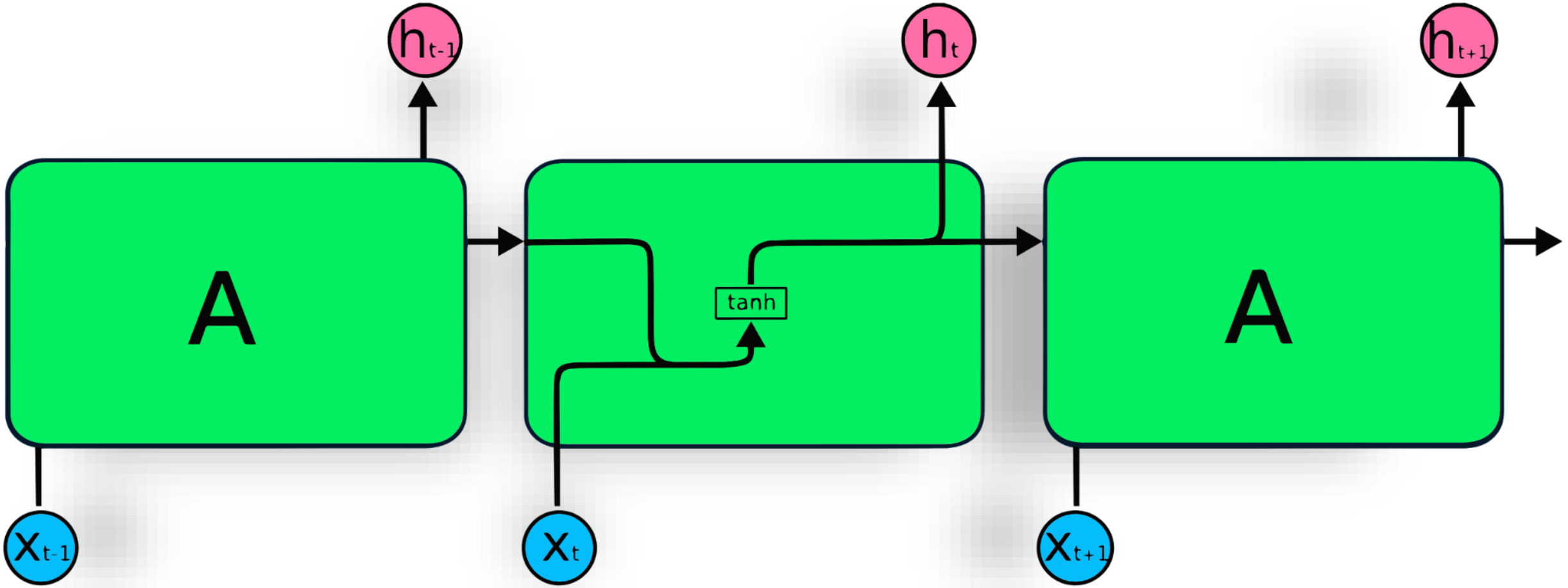
Middle Layer (A) – Looping for Each Time Step:

- Instead of creating a new hidden layer for each step in the sequence, the RNN uses the same hidden layer repeatedly.
- This means that for each time step in the input sequence, the same hidden layer (A) is used.
- This hidden layer shares the same parameters (weights, biases, and activation function) across all time steps.



How Recurrent Neural Networks Work?

- Ex-
- Suppose your sequence is made up of three words, and you pass the first word to the hidden layer (A).
- The RNN processes it, then moves to the next word and uses the same hidden layer (with the same weights, biases, etc.) to process that word.
- This repetition continues for the entire sequence (for each word).



How Recurrent Neural Networks Work?

How Recurrent Neural Networks Work?

- In an RNN, the hidden state a_t at time t can be expressed as:

$$a_t = f(W_a a_{t-1} + W_x x_t + b_a)$$

- W_a is the weight matrix for the previous hidden state a_{t-1}.
- Wx is the weight matrix for the current input x_t.
- b_h is the bias term.
- f is an activation function, often a non-linear function such as tanh or ReLU.

How Recurrent Neural Networks Work?

- Output in each x_t is,

$$h_t = f'(W_h a_t + b_h)$$

- Common activation functions (f') for different types of output include:
  - Softmax
  - Sigmoid
  - Linear Activation

Why Use the Same Hidden Layer Repeatedly?

The idea behind using the same hidden layer multiple times is to allow the RNN to remember what happened in previous time steps.

This is useful because it helps the network maintain a memory of past information while processing the current time step, which is important for sequences where the context matters (like sentences or time series data).

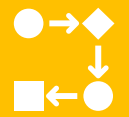
Backpropagation Through Time (BPTT)



Since the same hidden layer (A) is used for each time step, the model needs a way to adjust the parameters not just for each layer, but also across time steps.

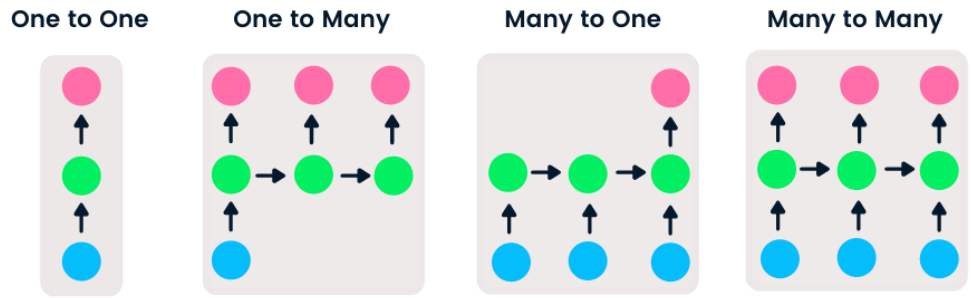


That’s where Backpropagation Through Time (BPTT) comes in.



BPTT calculates the errors across all time steps in the sequence and adjusts the shared parameters based on the accumulated errors from each step.

Types of RNN



- Feedforward networks have single input and output, while recurrent neural networks are flexible as the length of inputs and outputs can be changed.
- This flexibility allows RNNs to generate music, sentiment classification, and machine translation.
- There are four types of RNN based on different lengths of inputs and outputs.

Types of RNN



One-to-one (Vanilla RNN) is a simple neural network. It is commonly used for machine learning problems that have a single input and output.



One-to-many has a single input and multiple outputs. This is used for generating image captions.



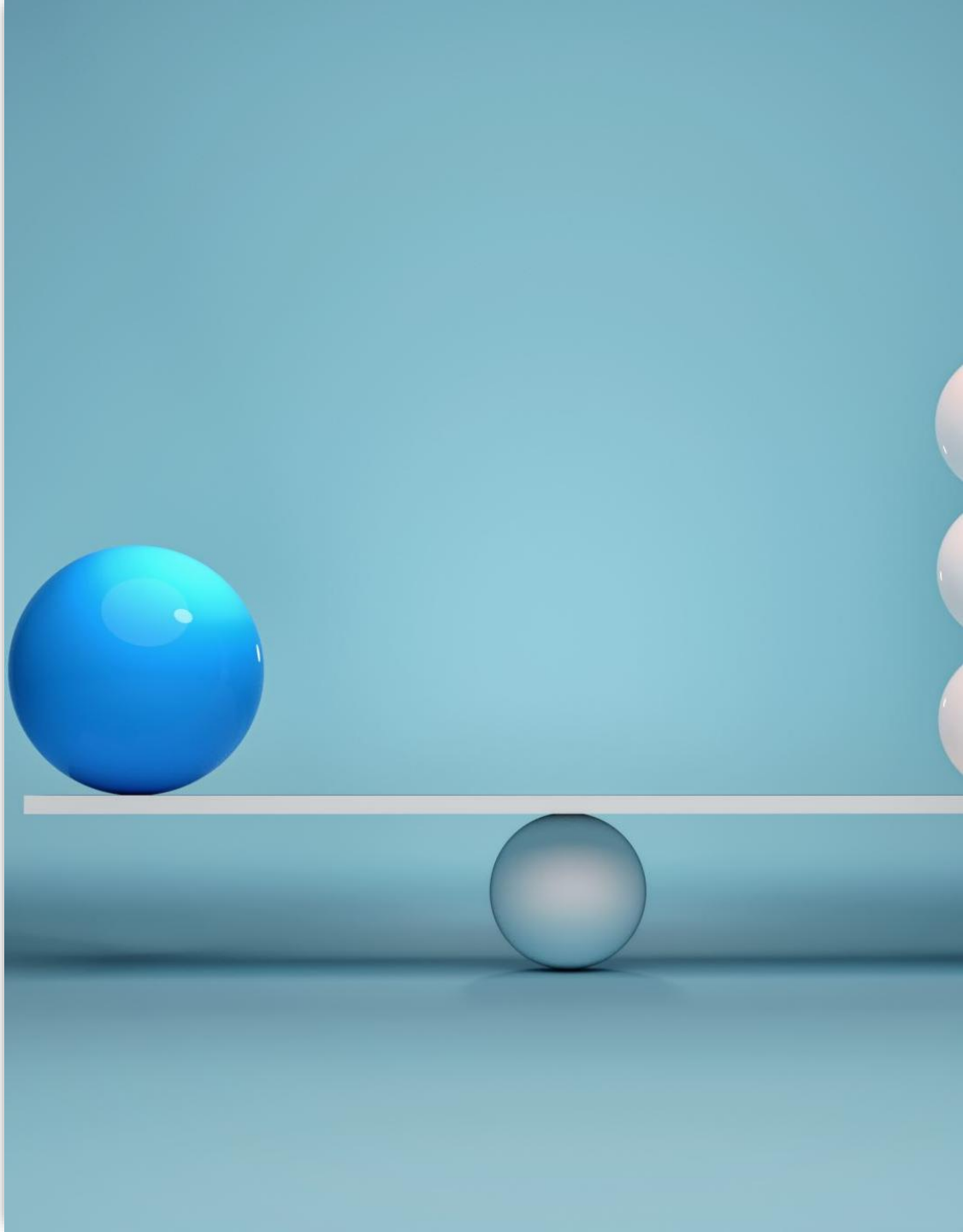
Many-to-one takes a sequence of multiple inputs and predicts a single output. It is popular in sentiment classification, where the input is text and the output is a category.



Many-to-many takes multiple inputs and outputs. The most common application is machine translation.

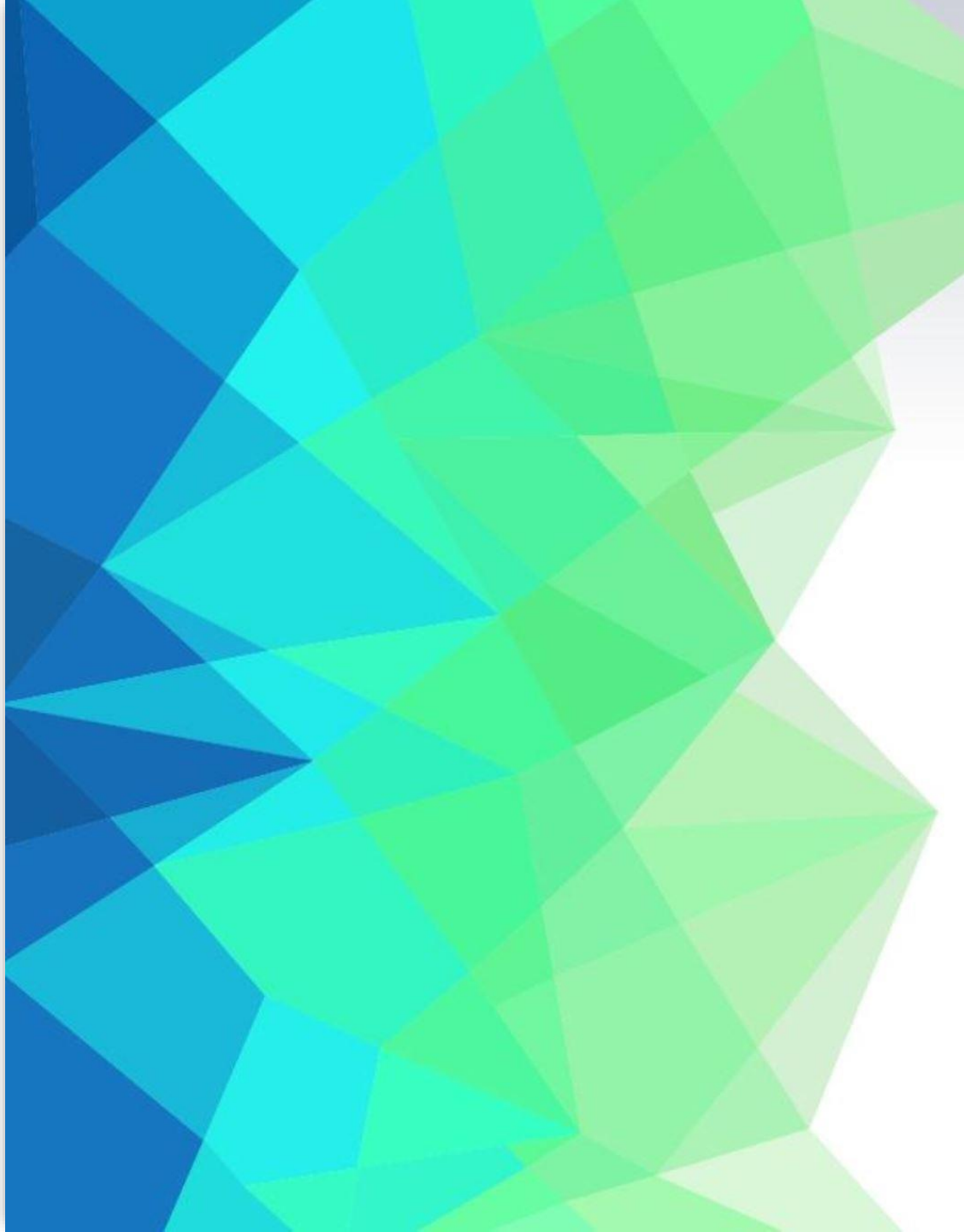
Vanishing Gradient Problem

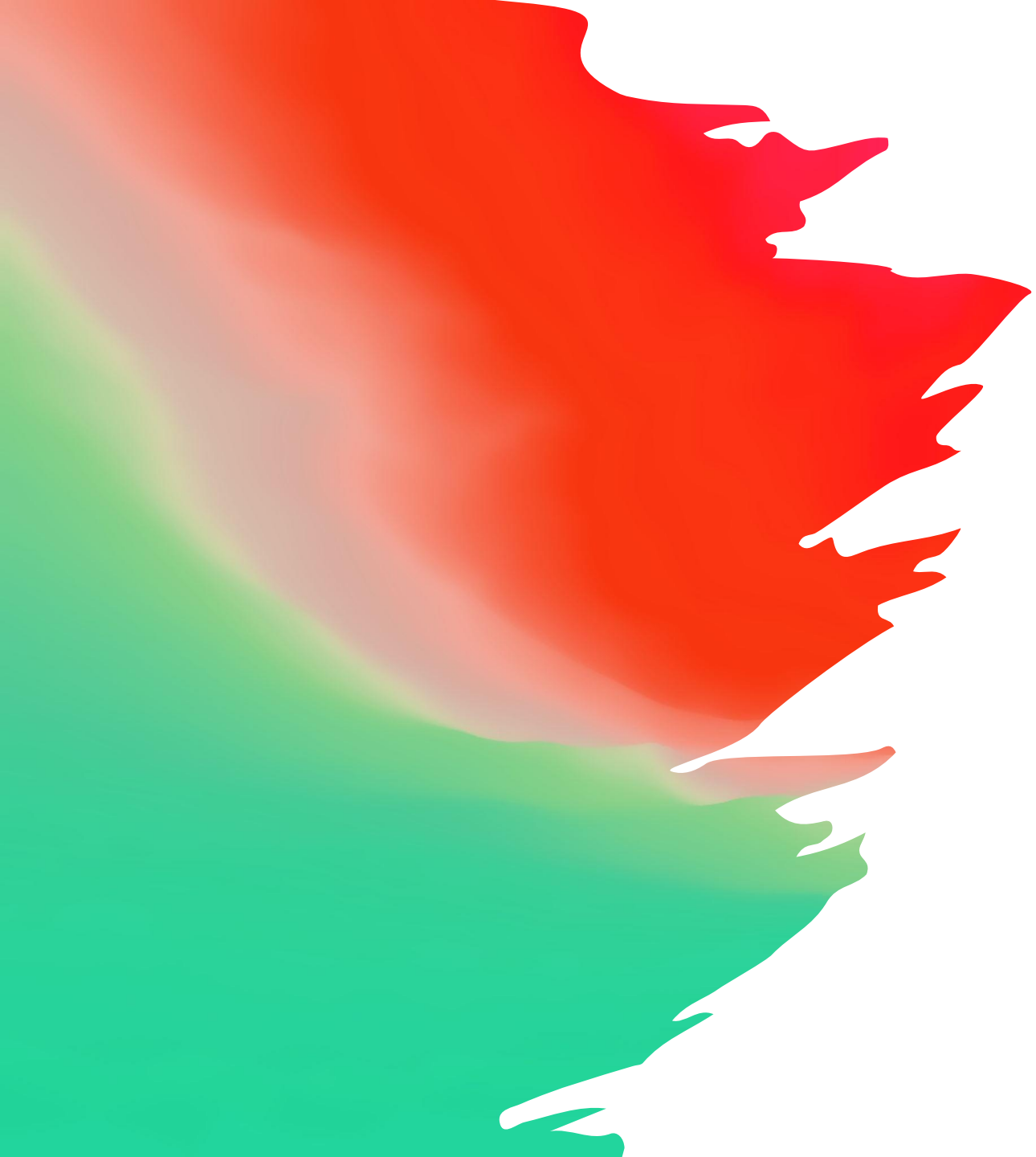
- In RNNs, when the model is training, it adjusts its weights using a process called backpropagation through time.
- In this process, the gradients (or error signals) are calculated and used to update the weights.
- However, when the network is very deep or has long sequences, the gradients can become very small (almost zero) as they are passed backward through the layers.
- This happens when the weights are less than 1.
- When the gradients are too small, the network struggles to update the weights, especially in earlier layers (or earlier time steps).
- This means that the RNN "forgets" important information from the beginning of the sequence.



Vanishing Gradient Problem

- Ex-
- Suppose an RNN is trying to learn dependencies between the first and the 100th word of a sentence.
- As the gradient is propagated back from the 100th to the 1st word, the gradients diminish, making it difficult for the RNN to learn long-term dependencies.
- The network can only focus on short-term patterns, and the gradient effectively becomes 0, preventing meaningful updates to the earlier layers.





Exploding Gradient Problem

- Gradients can also become **very large** during BPTT.
- Large gradients cause very large weight updates.
- Training can become unstable.
- The loss may suddenly increase.
- This is called the **Exploding Gradient Problem**.
- **Gradient clipping** is commonly used to control it.

Why Do We Need LSTM?

Basic RNN works well with **short-term dependencies**.

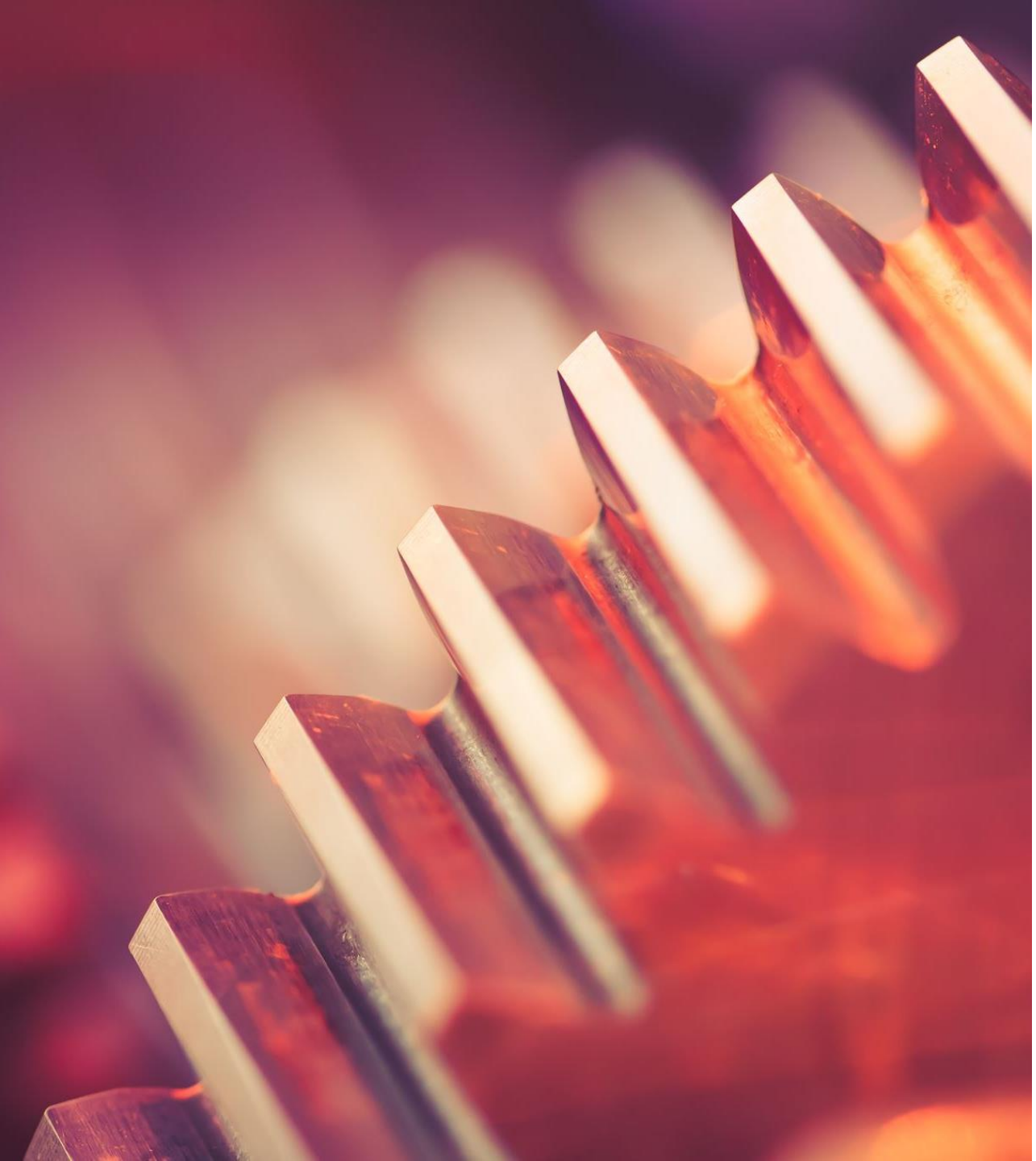
It may forget information from much earlier time steps.

Vanishing gradients make long-term learning difficult.

LSTM was designed to keep important information for longer.

It decides what to **remember, forget, and output**.

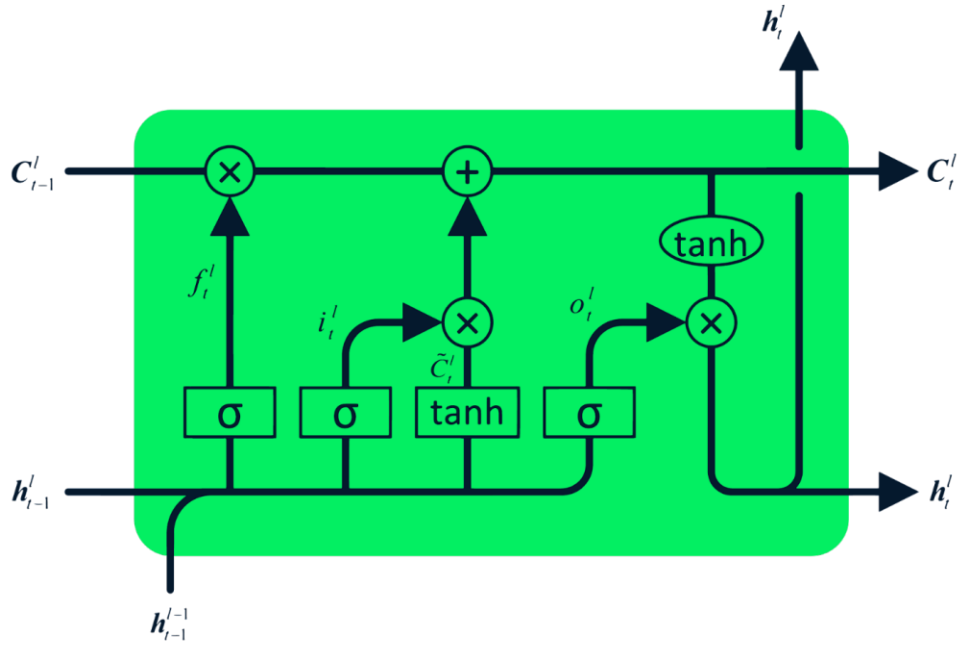
Therefore, LSTM is useful for long sequences.



Long Short-Term Memory (LSTM)

- The Long Short-Term Memory (LSTM) is the advanced type of RNN, which was designed to prevent both decaying and exploding gradient problems.
- Just like RNN, LSTM has repeating modules, but the structure is different. Instead of having a single layer of tanh, LSTM has four interacting layers that communicate with each other.
- This four-layered structure helps LSTM retain long-term memory and can be used in several sequential problems including machine translation, speech synthesis, speech recognition, and handwriting recognition.

Long Short-Term Memory (LSTM)



Main Parts of an LSTM Cell

- LSTM contains a **Cell State** and three main gates:
  - **Cell State** → carries long-term information.
  - **Forget Gate** → decides what old information to remove.
  - **Input Gate** → decides what new information to store.
  - **Output Gate** → decides what information to send forward.
- Gates normally use the **sigmoid activation function**.
- Values close to **0 block information**, while values close to **1 allow information**.



Gated Recurrent Unit (GRU)

- GRU is another improved version of RNN.
- It is similar to LSTM but has a **simpler structure**.
- It mainly uses an **Update Gate** and a **Reset Gate**.
- Reset Gate controls how much previous information is ignored.
- Update Gate controls how much information is carried forward.
- GRU normally has fewer parameters than LSTM.

RNN vs LSTM vs GRU

Feature	RNN	LSTM	GRU
Structure	Simple	More complex	Medium
Long-term memory	Weak	Strong	Strong
Gates	No	3 main gates	2 main gates
Training	Faster	Slower	Usually faster than LSTM
Parameters	Fewer	More	Fewer than LSTM
Good for	Short sequences	Long sequences	Long sequences

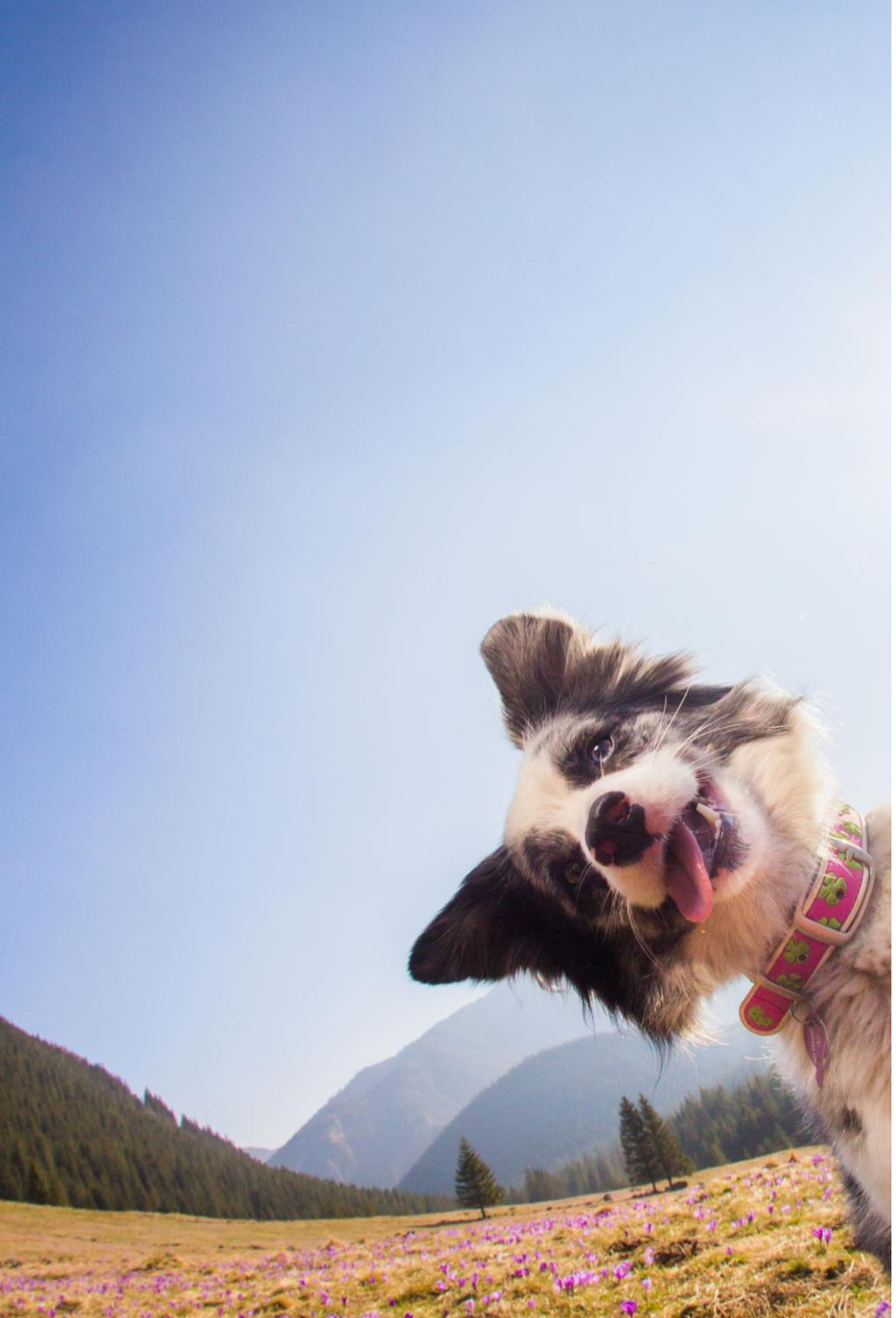
More Versions in RNN

- Bidirectional RNN
- Attention Mechanism
- Transformer Networks

Reinforcement Learning



H. M. Samadhi Chathuranga Rathnayake
MSc, MBA, PGDip, BSc (1st Class)



What is Reinforcement Learning?

- Think about the situation when you are training a dog. We can't direct the dog because he doesn't speak our language.
- We instead use a different approach.
- The dog tries to respond in a variety of ways as we imitate a situation (or a cue).
- We give the dog treats if their answer is what we were hoping for.
- Guess what, the dog performs the same action even more enthusiastically the following time it encounters the same circumstance in hopes of receiving more food.
- That's like learning "what to do" from favorable experiences.
- Dogs similarly have a tendency to pick up on bad behavior when faced with it.

What is Reinforcement Learning?



Our dog is a "**agent**" who is in contact with the outside world. The setting might be in your home, near you.



The circumstances they run into are comparable to a **state**. A state might be your dog standing as you speak in a particular tone in your living room.



Your dog moves from standing to sitting as a result of our agents' actions, which transfer it from one "**state**" to another.

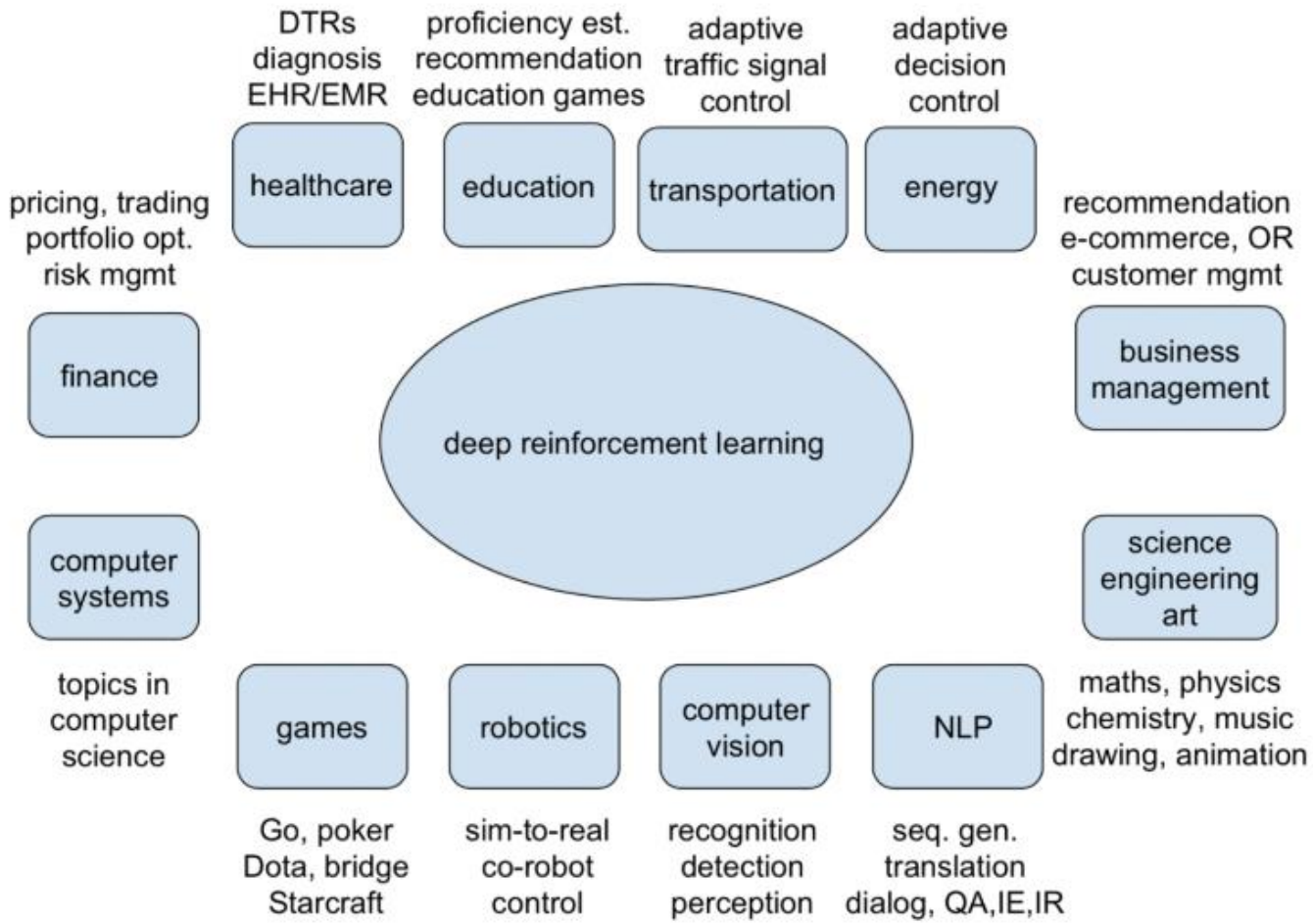


Following the shift, they might be given a **reward** or a **penalty**. They get a treat from you! Or a "No" as a penalty.



The method of selecting a course of action given a state in the hope of better results is known as a **policy**.

Applications of Reinforcement Learning



Yuxi Li, Deep Reinforcement Learning, arXiv, 2018

Reinforcement Learning Process





Agent

- The agent is the person who makes decisions based on rewards and penalties in reinforcement learning.
- Take a cricket batsman as an example. If he misses, he loses a point.
- He tries to hit the ball.
- He receives a reward if he hits the ball.
- He will learn how to play that specific ball as a result of these positive and bad experiences. The batsman in this instance is a representative.

Environment

- The Agent's environment in reinforcement learning is the setting in which it interacts and lives.
- The agent can interact with the environment by taking certain actions, but those actions cannot change the environment's dynamics or laws.
- This means that if humans were to influence the environment of the world, we would be constrained by the planet's physical laws.
- Our activities can influence the environment, but they cannot alter the physics of our world.

Deterministic vs Stochastic Environment

- Deterministic Environment
  - In a deterministic environment, the next state of the environment can always be determined based on the current state and the agent’s action.
  - For instance, if the agent steers left when operating a vehicle, the vehicle will only go in that direction.
  - In a perfect world, steering the car left will never cause it to move right; instead, it will always move left, according to determinism.
- Stochastic Environment
  - Consider the possibility that our agent's experience behind the wheel is not flawless.
  - There is a little chance that a break applied by an agent will not work and the car will not stop.
  - Similar to this, there is a slight risk that the car will just stop when the agent tries to accelerate it.
  - The next state of the car cannot always be predicted in this setting based on its current state and the agent's action.

Single Agent vs Multi-Agent Environnent

- Single Agent Environment
  - In a single agent environment, there is only one agent that exists and interacts with the environment.
  - For instance, it is a single agent environment if just one agent is operating the vehicle as it travels from point A to point B.
- Multi-Agent Environment
  - In a multi-agent environment, there are more than one agents that are interacting with the environment.
  - Therefore, in our example, a multi-agent environment exists if there are many cars being driven by various agents.

Discrete vs Continuous Environment

- Discrete Environment
  - In a discrete environment, the action space of the environment is discrete in nature.
  - For example, consider a grid world environment where the available action space is discrete like [left, right, up, down].
- Continuous Environment
  - In a continuous environment, the action space of the environment is continuous in nature.
  - A self-driving car world is an example of a continuous environment.
  - Here the action space of the environment consists of continuous action like car's speed, acceleration, steering rotation, etc.

Episodic vs Sequential Environment

- Episodic Environment
  - In an episodic environment, the agent's actions are confined to the particular episode only and not on any previous actions.
  - For example, in a game of cups and ball where the task is to guess the correct cup out of three cups having the ball, the agent's action is independent of its earlier attempts or episodes.
- Sequential Environment
  - In a sequential environment, the agent's actions are connected with the previous actions it took.
  - For example, in Chess, the current state of the game was reached due to all the game-playing actions the agent took.
  - And all the future actions in the game are also going to be dependent on its previous history of sequences.

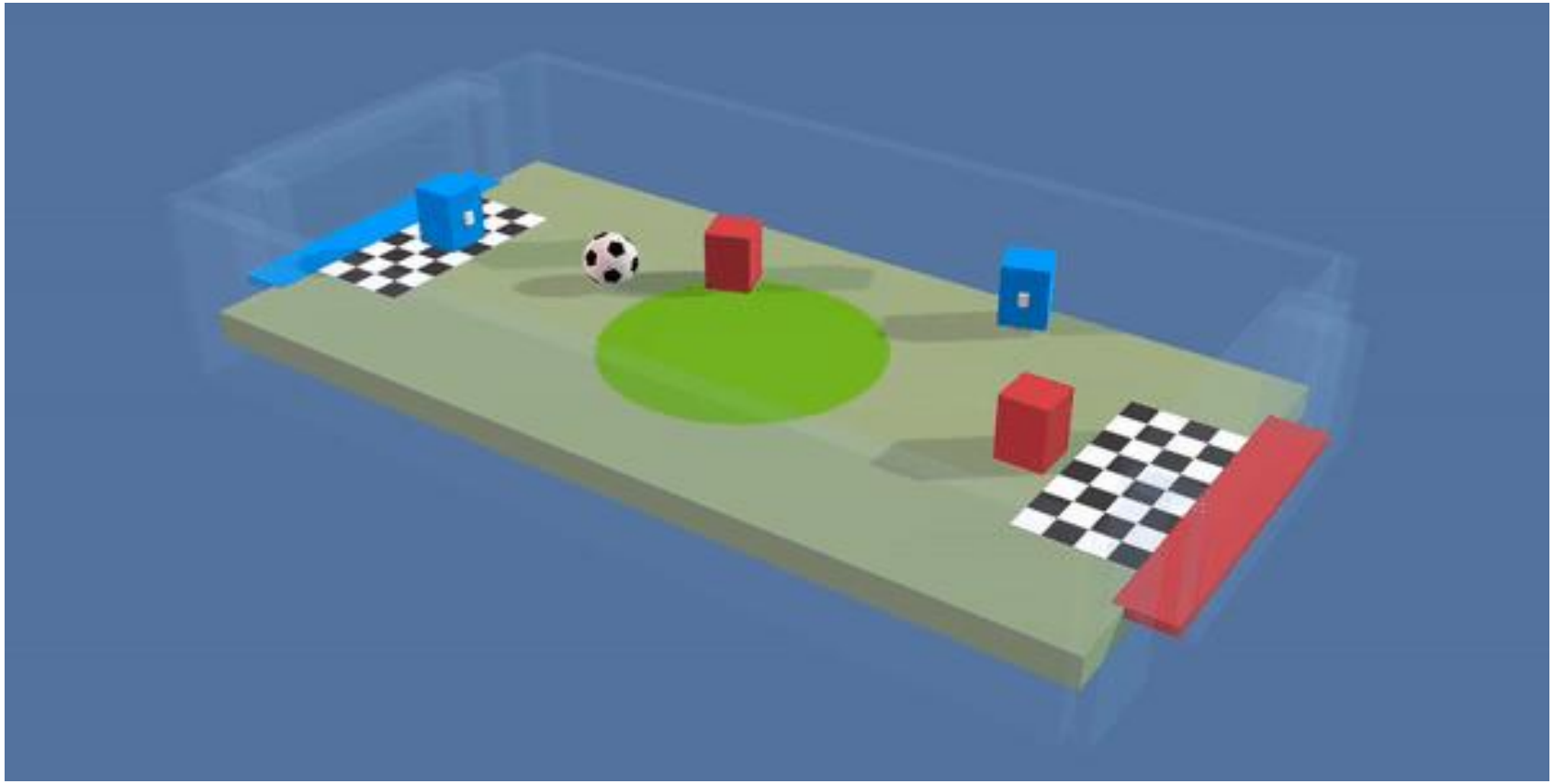
Fully Observable vs Partially Observable Environment

- Fully Observable Environment
  - In a fully observable environment, the agent is always aware of the complete state of the environment at any given point in time.
  - In the game of chess, the agent can always see the complete position of itself and its opponent on the board. Hence it is a fully observable environment.
- Partially Observable Environment
  - In a partially observable environment, the agent cannot always see the complete state of the environment at any given point in time.
  - In the game of poker, the agent cannot see the hands of the opponent for the most part of the game, and hence it is a partially observable environment.

State

- A state in reinforcement learning is a representation of the current environment that the agent is in.
- This state can be observed by the agent, and it includes all relevant information about the environment that the agent needs to know in order to make a decision.
- This state changes dynamically.

State of The Environment Changes



Reward

- The agent’s goal is specified by this component.
- In essence, we divide the entire process into time steps and assume that the agent does some action during each time step.
- Every action results in a change in the environment's state and a numerical reward that is given to the agent.
- This figure is influenced by the agent's behavior in time step t as well as the environment's condition in time interval t.
- Agent can thus affect reward in two different ways.
- It can improve its reward either directly through its behaviors or indirectly by altering the surroundings.
- In essence, rewards determine whether a behavior is good or poor, and the agent aims to maximize the reward over time.

Policy

- The fundamental component of reinforcement learning is the policy.
- It specifies the action the agent will take in a specific environment state.
- In essence, it links an agent's activities to environmental states.

Value Function

- The agent sees reward as an instant desire to be in a particular state.
- In other words, if we compare it to humans, the immediate objective is the reward.
- Unlike reward, value function defines a long-term goal for the agent.
- The value of the state represents the amount of reward agent can accumulate in the future, starting from that state.
- The value represents the prediction of rewards.
- In essence, the agent observes it as long-term desirability to be in a certain state.
- From the perspective of the agent, incentives are the main goal and values are secondary.
- This results from the environment's stochastic nature.
- However, judgments regarding the subsequent course of action are always made in light of the ideals.
- As a result, the agent is constantly attempting to enter the state with the highest value because doing so will ultimately yield greater rewards.

Markov Decision Process

- By taking action in each discrete time-step, the agent engages with the environment.
- The environment will alter its status in response to that action and provide some sort of monetary compensation.
- Utilizing the value function, the agent will strive to formulate a strategy that will maximize the payoff.
- Actually, reinforcement learning is attempting to address this issue.

Markov Decision Process

- We employ the **Markov Decision Processes (MDPs)** framework to represent this mathematically.
- Using this paradigm, nearly all reinforcement learning issues may be codified.

Markov Decision Process

- Markov Decision Process has four elements (S, A, Pa, Ra)
- *S* – Represents the **set of states**. At each time step *t*, the agent gets the environment's state – *St*, where $St \in S$.
- *A* – Represents the **set of actions** that the agent can foretake. At each time step *t*, based on the received state *St*, the agent makes the decision to perform an action – *At*, where $At \in A(St)$. *A(St)* represents a set of possible actions in the state *St*.
- *Pa* – Represents the **probability** that action in some state *s* will result in the time step *t*, will result in the state *s'* in the time step *t+1*.

$$P_a(s, s') = P(s' = S_{t+1} | S_t = s, A_t = a)$$

- *Ra* – Or more precisely *Ra(s, s')*, represents **expected reward** received after going from state *s* to the state *s'*, as a result of action *a*.

Markov Decision Process

- For easy understanding of equations, we will define some notations.
- We denote a set of states as $\boldsymbol{S}$, a set of actions as $\boldsymbol{A}$, and a set of rewards as $\boldsymbol{R}$.
- At each time step $\boldsymbol{t} = 0, 1, 2, \ldots$, some representation of the environment's state $\boldsymbol{St} \in \boldsymbol{S}$ is received by the agent.
- According to this state, the agent selects an action $\boldsymbol{At} \in \boldsymbol{A}$ which gives us the state-action pair $\boldsymbol{(St, At)}$.
- In the next time step $\boldsymbol{t+1}$, the transition of the environment happens and the new state $\boldsymbol{St+1} \in \boldsymbol{S}$ is achieved.
- At this time step $\boldsymbol{t+1}$, a reward $\boldsymbol{Rt+1} \in \boldsymbol{R}$ is received by the agent for the action $\boldsymbol{At}$ taken from state $\boldsymbol{St}$.

Markov Decision Process

- As we mentioned above that the goal of the agent is to maximize the cumulative rewards, we need to represent this cumulative reward in a formal way to use it in the calculations.
- We can call it as Expected Return and can be represented as,

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \cdots + R_T$$

Markov Decision Process

- The above equation works for episodic tasks.
- For continuing tasks, we need to update this equation as we don’t have a limit of time step T in continuing tasks.
- Discount factor γ is introduced here which forces the agent to focus on immediate rewards instead of future rewards.
- The value of γ remains between 0 and 1.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$
$$= \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Markov Decision Process

- Policy is a function that maps a given state to probabilities of selecting each possible action from the given state.
- If at time t, an agent follows policy π, then $\pi(a|s)$ becomes the probability that the action at time step t is $A_t=a$ if the state at time step t is $S_t=s$.
- The meaning of this is, the probability that an agent will take an action a in state s is $\pi(a|s)$ at time t with policy π.

Markov Decision Process

- The value functions are nothing more than a straightforward evaluation of how desirable it is for an agent to be in a particular state or to carry out a particular action in a particular state.
- Value functions come in two varieties, which are detailed below:

1. State Value Function
2. Action Value Function

Markov Decision Process

- The state-value function for policy π denoted as vπ determines the goodness of any given state for an agent who is following policy π.
- This function gives us the value which is the expected return starting from state s at time step t and following policy π afterward.

$$
\begin{aligned}
v_\pi(s) &= E_\pi[G_t \mid S_t = s] \\
&= E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s\right]
\end{aligned}
$$

Markov Decision Process

- The action-value function mentioned above determines the goodness of the action taken by the agent from a given state for policy π.
- This function gives the value which is the expected return starting from state s at time step t, with action a, and following policy π afterward.
- The output of this function is also called as Q-value where q stands for Quality. Note that in the state-value function, we did not consider the action taken by the agent.

$$q_\pi(s,a) = E_\pi[G_t \mid S_t = s, A_t = a]$$
$$= E_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right]$$

Markov Decision Process

- The policy π can be considered better than or same as the policy π′ if the expected return of policy π is greater than or equal to the expected return of policy π′ for all states.
- Such policy π is called Optimal Policy.

$$\pi \geq \pi' \text{ if and only if } v_{\pi}(s) \geq v_{\pi'}(s) \text{ for all } s \in \boldsymbol{S}.$$

Markov Decision Process

- The optimal policy should have an associated optimal state-value function or action-value function.

$$v_*(s) = \max_\pi v_\pi(s) \qquad q_*(s,a) = \max_\pi q_\pi(s,a)$$

- The optimal state-value function v∗ yields the largest expected return feasible for each of the state s with any policy π.
- And the optimal action-value function q∗ yields the largest expected return feasible for each state-action pair (s, a) with any policy π.

Bellman Optimality Equation

- The optimal action-value function q∗ satisfies the below-given equation. This is a fundamental property of the optimal action-value function.

$$q_*(s,a) = E\left[R_{t+1} + \gamma \max_{a'} q_*(s',a')\right]$$

Bellman Optimality Equation

- This equation tells that, at time t, for any state-action pair (s, a), the expected return from starting state s, taking action a, and with the optimal policy afterward will be equal to the expected reward Rt+1 we can get by selecting action a in state s, in addition with the maximum of “expected discounted return” that is achievable of any (s′, a′) where (s′, a′) is a potential next state-action pair.
- Considering the agent following an optimal policy, the latter state s′ will be the state from we can take the best possible next action a′ at time t+1.
- This seems somewhat difficult to understand.
- The essence is that this equation can be used to find optimal $q*$ in order to find optimal policy π and thus a reinforcement learning algorithm can find the action a that maximizes $q*(s, a)$.
- That is why this equation has its importance.

Bellman Optimality Equation

- The Optimal Value Function is recursively related to the Bellman Optimality Equation.
- The above property can be observed in the equation as we find $q*(s', a')$ which denotes the expected return after choosing an action a in state s which is then maximized to gain the optimal Q-value.

Q - Learning

- During the training of an agent in order to determine the optimal policy, we need to update the Q-value (Output of the Action-Value Function) iteratively.
- The Q-learning algorithm iteratively updates the Q-values for each state-action pair using the Bellman Optimality Equation until the Q-function converges to the optimal Q-function, $q*$.
- This process is called Value-Iteration.

Q - Learning

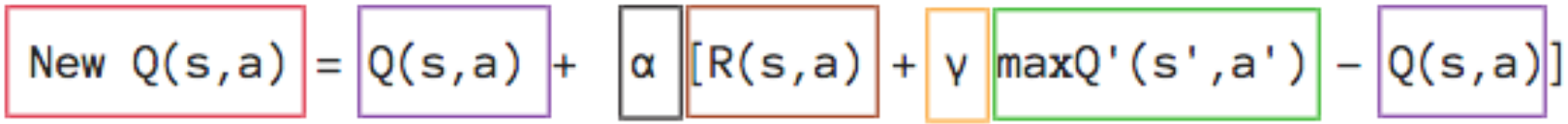
- To make the Q-value eventually converge to an optimal Q-value $q*$, what we have to do is for the given state-action pair, we have to make the Q-value as near as we can to the right-hand side of the Bellman Optimality Equation.
- For this, the loss between the Q-value and the optimal Q-value for the given state-action pair will be compared iteratively, and then each time we encounter the same state-action pair, we will update the Q-value, again and again, to reduce the loss.
- The loss can be given as $q*(s, a)-q(s, a)$.

$$q^{new}(s,a) = (1-\alpha)\underbrace{q(s,a)}_{\text{old value}} + \alpha\overbrace{\left(R_{t+1} + \gamma \max_{a'} q(s',a')\right)}^{\text{learned value}}$$

Q - Learning

- We can not overwrite the newly computed Q-value with the older value.
- Instead, we use a parameter called learning rate denoted by α, to determine how much information of the previously computed Q-value for the given state-action pair we retain over the newly computed Q-value calculated for the same state-action pair at a later time step.
- The higher the learning rate, the more quickly the agent will adopt the newly computed Q-value.
- Therefore we need to take care of the trade-off between new and old Q-value using the appropriate learning rate.

Q - Learning



- New Q Value for that state and the action
- Learning Rate
- Reward for taking that action at that state
- Current Q Values
- Maximum expected future reward given the new state (s') and all possible actions at that new state.
- Discount Rate

Q – Learning - Example

Q-Table		Actions				
		Action 1	Action 2	...	Action n-1	Action n
States	State 1	0.7334	0.65	...	0.4522	0.6452
	State 2	0.8444	0.4534	...	0.3322	0.7134
	...	...	...	...	...	...
	State n-1	0.7552	0.65	...	0.4522	0.6452
	State n	0.9234	0.4534	...	0.3322	0.7134

Q – Learning Steps

1. **Initialize** all *Q-Values* in the *Q-Table* arbitrary, and the Q value of terminal-state to 0:

$Q(s, a) = n, \ \forall s \in S, \ \forall a \in A(s)$

$Q(\text{terminal-state}, \cdot) = 0$

2. **Pick** the action *a*, from the set of actions defined for that state *A(s)* defined by the policy π.
3. **Perform** action *a*
4. **Observe** reward *R* and the next state *s'*
5. For all possible actions from the state *s'* select the one with the **highest** *Q-Value* – *a'*.
6. **Update** value for the state using the formula:

$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma Q(s', a') - Q(s, a)]$

7. **Repeat** steps 2-5 for each time step until the terminal state is reached
8. **Repeat** steps 2-6 for each episode

Q - Learning

- Q Learning algorithm can be represented as,

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big]$$

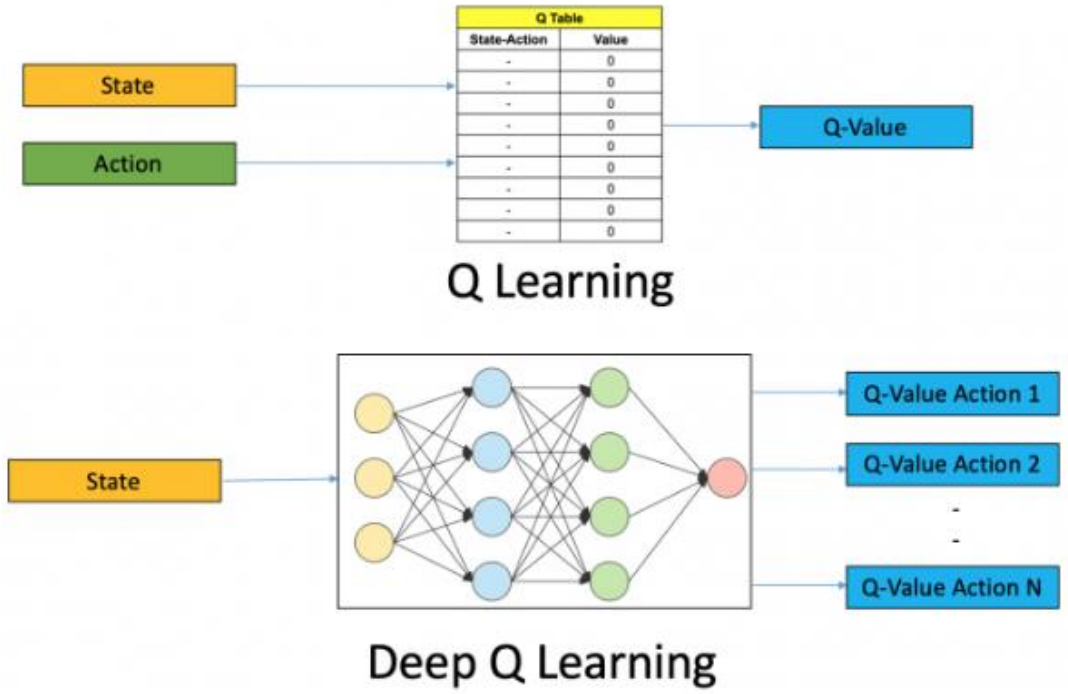
- Where alpha is the learning rate or step size. This simply determines to what extent newly acquired information overrides old information.

Why ‘Deep’ Q-Learning?

- Q-learning is a simple yet quite powerful algorithm.
- Imagine an environment with 10,000 states and 1,000 actions per state. This would create a table of 10 million cells. Things will quickly get out of control!
- It is pretty clear that we can’t infer the Q-value of new states from already explored states. This presents two problems:
  - First, the amount of memory required to save and update that table would increase as the number of states increases
  - Second, the amount of time required to explore each state to create the required Q-table would be unrealistic

Deep Q-Networks

- In deep Q-learning, we use a neural network to approximate the Q-value function. The state is given as the input and the Q-value of all possible actions is generated as the output.



In Deep Q

- All the past experience is stored by the user in memory
- The next action is determined by the maximum output of the Q-network
- The loss function here is mean squared error of the predicted Q-value and the target Q-value – Q*. This is basically a regression problem.
- However, we do not know the target or actual value here as we are dealing with a reinforcement learning problem.
- Going back to the Q-value update equation derived from the Bellman equation. we have:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big]$$

- The section in green represents the target.
- We can argue that it is predicting its own value, but since R is the unbiased true reward, the network is going to update its gradient using backpropagation to finally converge.

Pseudocode for Deep Q-Learning

Start with $Q_0(s, a)$ for all s, a.

Get initial state s

For k = 1, 2, ... till convergence

Sample action a, get next state s'

If s' is terminal:

$$\text{target} = R(s, a, s')$$

Sample new initial state s'

else:

$$\text{target} = R(s, a, s') + \gamma \max_{a'} Q_k(s', a')$$

Chasing a nonstationary target!

$$\theta_{k+1} \leftarrow \theta_k - \alpha \nabla_\theta \mathbb{E}_{s' \sim P(s'|s,a)} \left[(Q_\theta(s, a) - \text{target}(s'))^2 \right] \Big|_{\theta=\theta_k}$$

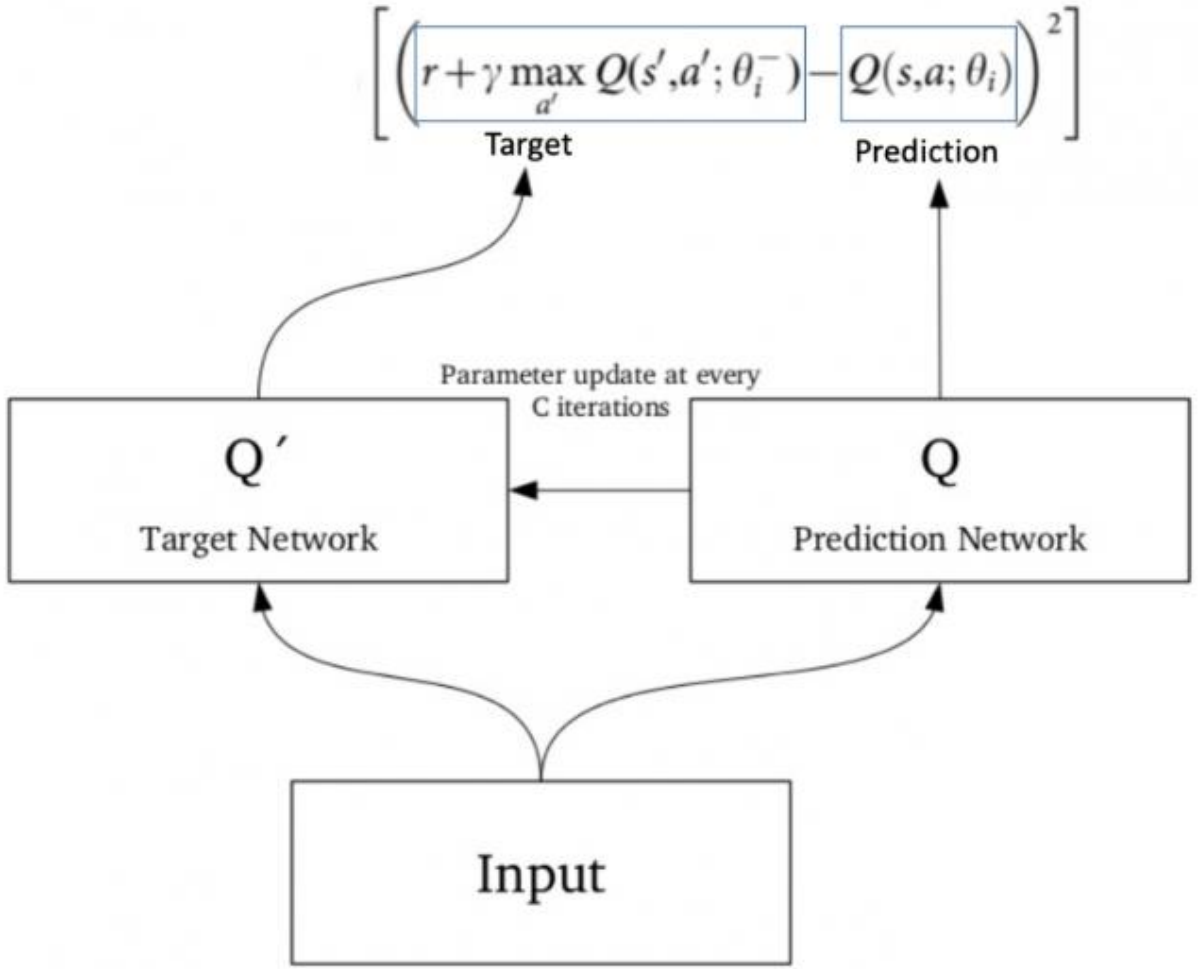
$$s \leftarrow s'$$

Updates are correlated within a trajectory!

Target Network

- Since the same network is calculating the predicted value and the target value, there could be a lot of divergence between these two.
- So, instead of using one neural network for learning, we can use two.
- We could use a separate network to estimate the target.
- This target network has the same architecture as the function approximator but with frozen parameters.
- For every C iterations (a hyperparameter), the parameters from the prediction network are copied to the target network.
- This leads to more stable training because it keeps the target function fixed (for a while).

Target Network



Experience Replay

- To perform experience replay, we store the agent's experiences – et=(st,at,rt,st+1)
- What does the above statement mean?
- Instead of running Q-learning on state/action pairs as they occur during simulation or the actual experience, the system stores the data discovered for [state, action, reward, next_state] – in a large table.
- Let's understand this using an example.
- Suppose we are trying to build a video game bot where each frame of the game represents a different state. During training, we could sample a random batch of 64 frames from the last 100,000 frames to train our network. This would get us a subset within which the correlation amongst the samples is low and will also provide better sampling efficiency.

Steps of Deep Q-Network

1. Preprocess and feed the game screen (state s) to our DQN, which will return the Q-values of all possible actions in the state
2. Select an action using the epsilon-greedy policy. With the probability epsilon, we select a random action *a* and with probability 1-epsilon, we select an action that has a maximum Q-value, such as a = argmax(Q(s,a,w))
3. Perform this action in a state *s* and move to a new state *s'* to receive a reward. This state s' is the preprocessed image of the next game screen. We store this transition in our replay buffer as <s,a,r,s'>
4. Next, sample some random batches of transitions from the replay buffer and calculate the loss
5. It is known that:

$$Loss = (r + \gamma max_{a'} Q(s', a'; \theta') - Q(s, a; \theta))^2$$

   which is just the squared difference between target Q and predicted Q
6. Perform gradient descent with respect to our actual network parameters in order to minimize this loss
7. After every C iterations, copy our actual network weights to the target network weights
8. Repeat these steps for *M* number of episodes

Linear Programming

Graphical Method



H. M. Samadhi Chathuranga Rathnayake

MSc, MBA, PGDip, BSc (1st Class)

Operations Research (OR)

- Operations research (OR) is an analytical method of problem-solving and decision-making that is useful in the management of organizations.
- In operations research, problems are broken down into basic components and then solved in defined steps by mathematical analysis.
- The concept of operations research arose during World War II by military planners.
- After the war, the techniques used in their operations research were applied to addressing problems in business, the government and society.

Operations Research (OR)

- The process of operations research can be broadly broken down into the following steps:

  - Identifying a problem that needs to be solved.
  - Constructing a model around the problem that resembles the real world and variables.
  - Using the model to derive solutions to the problem.
  - Testing each solution on the model and analyzing its success.
  - Implementing the solution to the actual problem.

Operations Research (OR)

- There are three primary characteristics of all operations research efforts:

- **Optimization-** The purpose of operations research is to achieve the best performance under the given circumstances. Optimization also involves comparing and narrowing down potential options.
- **Simulation-** This involves building models or replications in order to try out and test solutions before applying them.
- **Probability and statistics-** This includes using mathematical algorithms and data to uncover helpful insights and risks, make reliable predictions and test possible solutions.

Operations Research (OR)

- Operations research can be applied to a variety of use cases, including:

✓Scheduling and time management.
✓Urban and agricultural planning.
✓Enterprise resource planning (ERP) and supply chain management (SCM).
✓Inventory management.
✓Network optimization and engineering.
✓Packet routing optimization.
✓Risk management.

Linear Programming

- linear programming, mathematical modeling technique in which a linear function is maximized or minimized when subjected to various constraints.
- This technique has been useful for guiding quantitative decisions in business planning, in industrial engineering, and—to a lesser extent—in the social and physical sciences.
- Linear Programming Problems (LPP) provide the method of finding such an optimized function along with/or the values which would optimize the required function accordingly.
- Formulation of an LPP refers to translating the real-world problem into the form of mathematical equations which could be solved.
- It usually requires a thorough understanding of the problem.

Steps of Formulating Linear Programming

- **Step 1**: Identify the ‘n’ number of decision variables which govern the behaviour of the objective function (which needs to be optimized).
- **Step 2**: Identify the set of constraints on the decision variables and express them in the form of linear equations /inequations. This will set up our region in the n-dimensional space within which the objective function needs to be optimized. Don’t forget to impose the condition of non-negativity on the decision variables i.e. all of them must be positive since the problem might represent a physical scenario, and such variables can’t be negative.
- **Step 3**: Express the objective function in the form of a linear equation in the decision variables.
- **Step 4**: Optimize the objective function either graphically or mathematically.

Example – Formulating LP

- A calculator company produces a handheld calculator and a scientific calculator. Long-term projections indicate an expected demand of at least 150 scientific and 100 handheld calculators each day. Because of limitations on production capacity, no more than 250 scientific and 200 handheld calculators can be made daily. To satisfy a shipping contract, a minimum of 250 calculators must be shipped each day. If each scientific calculator sold, results in a 20 rupees loss, but each handheld calculator produces a 50 rupees profit; then how many of each type should be manufactured daily to maximize the net profit?



Example – Formulating LP

- **Step 1**: The decision variables: Since the question has asked for an optimum number of calculators, that's what our decision variables in this problem would be. Let,

  - x = number of scientific calculators produced
  - y = number of handheld calculators produced

- Therefore, we have 2 decision variables in this problem, namely 'x' and 'y'.

Example – Formulating LP

- **Step 2**: The constraints: Since the company can't produce a negative number of calculators in a day, a natural constraint would be:
  - $x \geq 0$
  - $y \geq 0$
- However, a lower bound for the company to sell calculators is already supplied in the problem. We can note it down as:
  - $x \geq 150$
  - $y \geq 100$

Example – Formulating LP

- We have also been given an upper bound for these variables, owing to the limitations on production by the company. We can write as follows:
  - $x \leq 250$
  - $y \leq 200$
- Besides, we also have a joint constraint on the values of ‘x’ and ‘y’ due to the minimum order on a shipping consignment; given as:
  - $x + y \geq 250$

Example – Formulating LP

- **Step 3**: Objective Function: Clearly, here we need to optimize the Net Profit function. The Net Profit Function is given as:

$$\mathbf{P = -20x + 50y}$$

Example – Formulating LP

- Step 4: Solving the problem: the system here –

$$\text{Maximization of } P = -20x + 50y,$$

subject to:

$$150 \leq x \leq 250$$

$$100 \leq y \leq 200$$

$$x + y \geq 250$$

Types of Linear Programming Problems

- **Diet Problems**
- As the name suggests in itself, such problems involve optimizing the intake of certain types of foods rich in certain nutrients that could help one follow a particular diet plan.
- More precisely, the goal of a diet problem is to select a set of foods that will satisfy a set of a daily nutritional requirement at a minimum cost.
- Constraints – The specified nutritional requirements, that could be a specific calorie intake or the amount of sugar or cholesterol in the diet.
- Objective function – The cost of the food intake.

Types of Linear Programming Problems

- Example: A kitchen manager at Nulife Hospital has to decide the food mix for the patients. Dietary instructions are that each patient must get at least:

- One gram of protein
- One gram of fat
- Three gms. of carbohydrates

Types of Linear Programming Problems

- **Manufacturing Problems**
- These problems involve optimizing the production rate or the net profits of the manufactured products, which could be a function of the available workspace, the number of labourers, machine hours, packaging material used, raw materials required, the market value of the product etc.
- These find application in the industrial sectors and hence the prediction of a company's possible capital increase over the years.

- Constraints – Variables like the work-hours, the cost of the packaging material used etc.
- Objective function – The production rate

Types of Linear Programming Problems

- Example: A company makes two products (X and Y) using two machines (A and B).
- Each unit of X that is produced requires 50 minutes processing time on machine A and 30 minutes processing time on machine B.
- Each unit of Y that is produced requires 24 minutes processing time on machine A and 33 minutes processing time on machine B.
- At the start of the current week, there are 30 units of X and 90 units of Y in stock. Available processing time on machine ‘A’ is forecast to be 40 hours and on machine ‘B’ is forecast to be 35 hours.
- The forecast of demand for X in the current week is 75 units and for Y is 95 units.
- Company policy is to maximize the combined sum of the units of X and the units of Y in stock at the end of the week.

Types of Linear Programming Problems

- **Transportation Problems**
- These problems are related to the study of the efficient transportation routes i.e. how efficiently the product from different sources of production is transported to the different markets, such as the total transportation cost is minimum.
- Analysis of such problems is very crucial for big companies with several production plants and a widespread area to cater to.

- Constraints – The specific supply and demand patterns.
- Objective function – The transportation cost

Types of Linear Programming Problems

- Example: A firm has 3 factories – Alpha, Tango, and Kingston.
- There are four major warehouses situated at locations PTR, CNB, DLS, and MMB.
- The average daily production of the product at Alpha, Tango, Kingston is 30, 40 and 50 units respectively.
- The average daily requirement of this product at PTR, CNB, DLS, and MMB is 35, 30, 32, 25 units respectively.
- The goal here is to minimize the total transportation costs.

Types of Linear Programming Problems

- **Optimal Assignment Problems**
- These problems are concerned with the completion of a particular task/assignment of a company by choosing a certain number of employees to complete the assignment within the required deadline, given that a single person works on only one job within the assignment.
- Such problems find their application in event planning/management in big companies etc.

- Constraints – The number of employees, the work-hours of each employee etc.
- Objective function – The total assignments done

Types of Linear Programming Problems

- Example: The Watson Toys and Watches Company has four men available for work on four separate jobs.
- Only one man can work on any one job.
- Find a suitable system of assigning men to the jobs in such a way that the total cost of the assignment is minimum.

Linear Programming – Graphical Method

- Owing to the importance of linear programming models in various industries, many types of algorithms have been developed over the years to solve them.
- Some famous mentions include the Simplex method, the Hungarian approach, and others.
- Here we are going to concentrate on one of the most basic methods to handle a linear programming problem i.e. the graphical method.
- In principle, this method works for almost all different types of problems but gets more and more difficult to solve when the number of decision variables and the constraints increases.
- Therefore, we'll illustrate it in a simple case i.e. for two variables only.

Steps of Graphical Method

- **Step 1: Formulate the LP (Linear programming) problem**
- We have already understood the mathematical formulation of an LP problem in a previous section.
- Note that this is the most crucial step as all the subsequent steps depend on our analysis here.

Steps of Graphical Method

- **Step 2: Construct a graph and plot the constraint lines**
- The graph must be constructed in ‘n’ dimensions, where ‘n’ is the number of decision variables.
- This should give you an idea about the complexity of this step if the number of decision variables increases.
- One must know that one cannot imagine more than 3-dimensions anyway.
- The constraint lines can be constructed by joining the horizontal and vertical intercepts found from each constraint equation.

Steps of Graphical Method

- **Step 3: Determine the valid side of each constraint line**
- This is used to determine the domain of the available space, which can result in a feasible solution.
- A simple method is to put the coordinates of the origin (0,0) in the problem and determine whether the objective function takes on a physical solution or not.
- If yes, then the side of the constraint lines on which the origin lies is the valid side.
- Otherwise it lies on the opposite one.

Steps of Graphical Method

- **Step 4: Identify the feasible solution region**
- The feasible solution region on the graph is the one which is satisfied by all the constraints.
- It could be viewed as the intersection of the valid regions of each constraint line as well.
- Choosing any point in this area would result in a valid solution for our objective function.

Steps of Graphical Method

- **Step 5: Plot the objective function on the graph**
- It will clearly be a straight line since we are dealing with linear equations here.
- One must be sure to draw it differently from the constraint lines to avoid confusion.
- Choose the constant value in the equation of the objective function randomly, just to make it clearly distinguishable.

Steps of Graphical Method

- **Step 6: Find the optimum point**
- An optimum point always lies on one of the corners of the feasible region.
- How to find it? Place a ruler on the graph sheet, parallel to the objective function.
- Be sure to keep the orientation of this ruler fixed in space.
- We only need the direction of the straight line of the objective function.
- Now begin from the far corner of the graph and tend to slide it towards the origin.

Steps of Graphical Method

- **Step 6: Find the optimum point**

- **If the goal is to minimize the objective function**, find the point of contact of the ruler with the feasible region, which is the closest to the origin.
- This is the optimum point for minimizing the function.
- **If the goal is to maximize the objective function**, find the point of contact of the ruler with the feasible region, which is the farthest from the origin.
- This is the optimum point for maximizing the function.

Steps of Graphical Method

- **Step 7: Calculate the coordinates of the optimum point**
- This is the last step of the process.
- Once you locate the optimum point, you'll need to find its coordinates.
- This can be done by drawing two perpendicular lines from the point onto the coordinate axes and noting down the coordinates.
- Otherwise, you may proceed algebraically also if the optimum point is at the intersection of two constraint lines and find it by solving a set of simultaneous linear equations.
- The Optimum Point gives you the values of the decision variables necessary to optimize the objective function.
- To find out the optimized objective function, one can simply put in the values of these parameters in the equation of the objective function.

Graphical Method Example

- A health-conscious family wants to have a very well controlled vitamin C-rich mixed fruit-breakfast which is a good source of dietary fibre as well; in the form of 5 fruit servings per day. They choose apples and bananas as their target fruits, which can be purchased from an online vendor in bulk at a reasonable price. Bananas cost 30 rupees per dozen (6 servings) and apples cost 80 rupees per kg (8 servings). Given: 1 banana contains 8.8 mg of Vitamin C and 100-125 g of apples i.e. 1 serving contains 5.2 mg of Vitamin C. Every person of the family would like to have at least 20 mg of Vitamin C daily but would like to keep the intake under 60 mg. How much fruit servings would the family have to consume on a daily basis per person to minimize their cost?



Graphical Method Example

- We begin step-wise with the formulation of the problem first. The constraint variables – ‘x’ = number of banana servings taken and ‘y’ = number of servings of apples taken.
- Let us find out the objective function now.
- Cost of a banana serving = 30/6 rupees = 5 rupees. Thus, the cost of ‘x’ banana servings = 5x rupees
- Cost of an apple serving = 80/8 rupees = 10 rupees. Thus the cost of ‘y’ apple servings = 10y rupees

$$Total\ Cost\ C\ =\ 5x\ +\ 10y$$

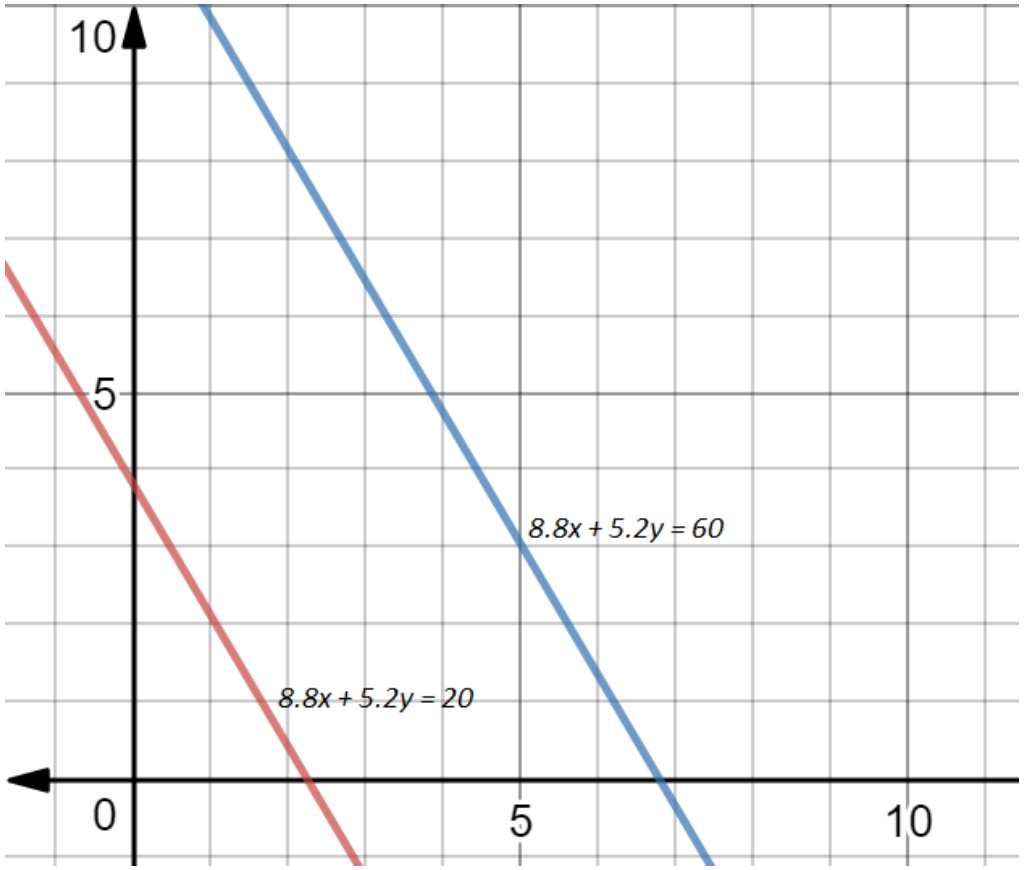
Constraints: $x\ \geq\ 0;\ y\ \geq\ 0$ (non-negative number of servings)

- Total Vitamin C intake:

$$8.8x\ +\ 5.2y\ \geq\ 20 \qquad (1)$$
$$8.8x\ +\ 5.2y\ \leq\ 60 \qquad (2)$$

Graphical Method Example

- Now let us plot a graph with the constraint equations

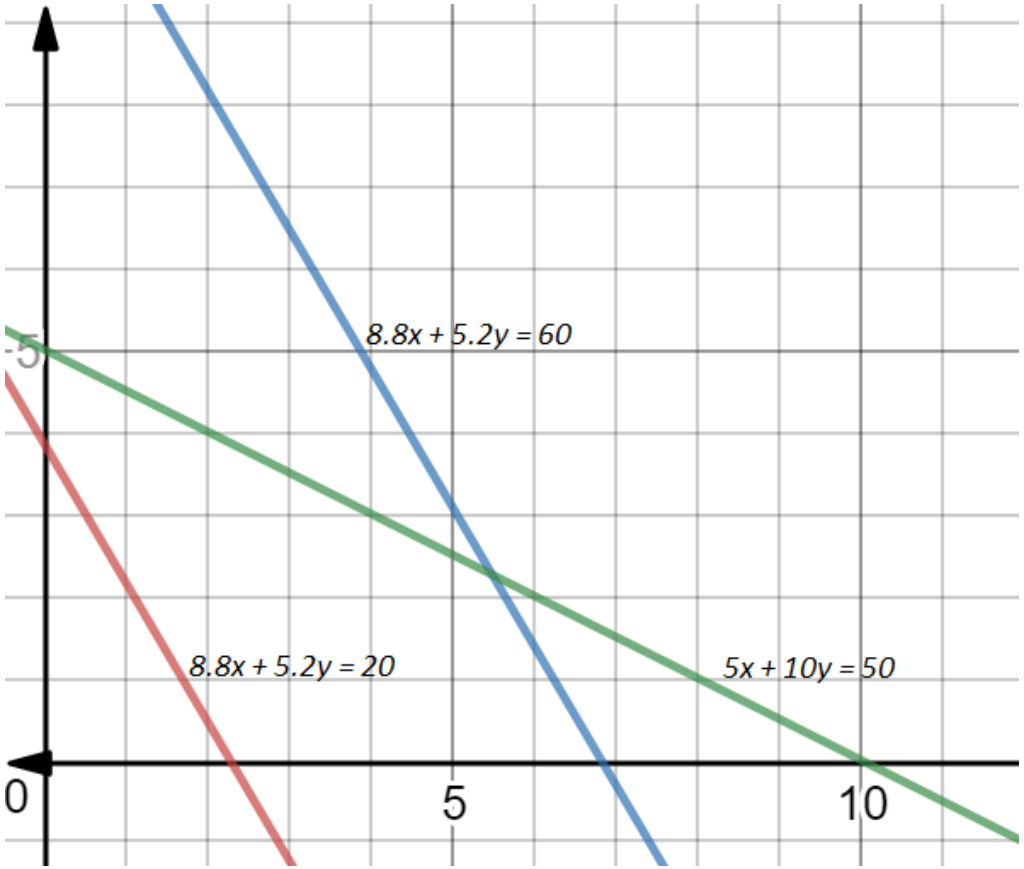


Graphical Method Example

- To check for the validity of the equations, put x=0, y=0 in (1).
- Clearly, it doesn’t satisfy the inequality.
- Therefore, we must choose the side opposite to the origin as our valid region.
- Similarly, the side towards origin is the valid region for equation 2)
- **Feasible Region**: As per the analysis above, the feasible region for this problem would be the one in between the red and blue lines in the graph.

Graphical Method Example

- For the direction of the objective function; let us plot 5x+10y = 50.



Graphical Method Example

- Now take a ruler and place it on the straight line of the objective function.
- Start sliding it from the left end of the graph.
- We want the minimum value of the cost i.e. the minimum value of the optimum function C.
- Thus we should slide the ruler in such a way that a point is reached, which:

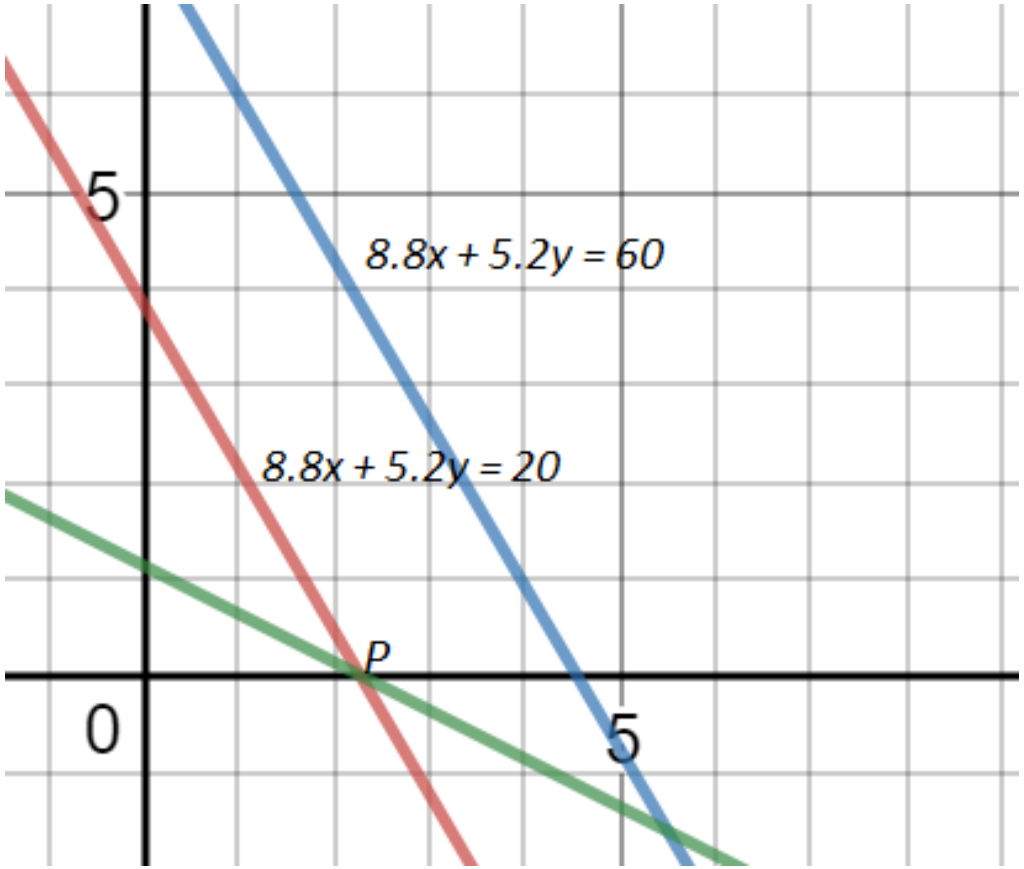
    - Lies in the feasible region
    - Is closer to the origin as compared to the other points

Graphical Method Example

- This would be our Optimum Point.
- It has been marked as P in the graph.
- It is the one which you will get at the extreme right side of the feasible region here.
- I’ve also shown the position in which your ruler needs to be to get this point by the line in green.

Graphical Method Example

- Check the graph.



Graphical Method Example

- Now coordinates of this point should be calculated.
- To do this, just solve the simultaneous pair of linear equations:

$$y = 0$$
$$8.8x + 5.2y = 20$$

- We'll get the coordinates of 'P' as (2.27, 0).
- This implies that the family must consume 2.27 bananas and 0 apples to minimize their cost and function according to their diet plan.

Python Implementation

- **PuLP** is a python library which can be used to solve linear programming problems.
- Consider the following example,

$$
\begin{aligned}
& min && Z = 60x_1 + 40x_3 \\
s.t. \quad & && 4x_1 + 4x_2 \geq 10 \\
& && 2x_1 + x_2 \geq 4 \\
& && 6x_1 + 2x_2 \leq 12 \\
& && x_1 \geq 0 \\
& && x_2 \geq 0
\end{aligned}
$$

Linear Programming

Simplex Method



H. M. Samadhi Chathuranga Rathnayake
MSc, MBA, PGDip, BSc (1st Class)

Introduction

- Approach to solving linear programming models by hand using slack variables, tableaus, and pivot variables as a means to finding the optimal solution of an optimization problem.
- Linear program is a method of achieving the best outcome given a maximum or minimum equation with linear constraints.

Steps

- To solve a linear programming model using the Simplex method the following steps are necessary:
  - Standard form
  - Introducing slack variables
  - Creating the tableau
  - Pivot variables
  - Creating a new tableau
  - Checking for optimality
  - Identify optimal values

Standard form

- Standard form is the baseline format for all linear programs before solving for the optimal solution and has three requirements:

1. Must be a maximization problem
2. All linear constraints must be in a less-than-or-equal-to inequality
3. All variables are non-negative.

Example

- Objective Function:

$$\text{Minimize: } -z = -8x_1 - 10x_2 - 7x_3$$

- Constraints:

$$x_1 + 3x_2 + 2x_3 \leq 10$$

$$-x_1 - 5x_2 - x_3 \geq -8$$

$$x_1, x_2, x_3 \geq 0$$

Must be a maximization problem

- Multiply both the left and the right sides of the objective function by -1.

$$-1 * \ -z = -8x_1 - 10x_2 - 7x_3$$

$$z = 8x_1 + 10x_2 + 7x_3$$

- Maximize : $z = 8x_1 + 10x_2 + 7x_3$

All linear constraints must be in a less than or equal to inequality

- multiply by -1 on both sides, the inequality can be changed to less-than-or-equal-to.

$$-1 * (-x_1 - 5x_2 - x_3 \geq -8)$$

$$x_1 + 5x_2 + x_3 \leq 8$$

Determine Slack Variables

- Slack variables are additional variables that are introduced into the linear constraints of a linear program to transform them from inequality constraints to equality constraints.

$$x1 + 3x2 + 2x3 + s1 = 10$$

$$x1 + 5x2 + x3 + s2 = 8$$

$$x1, x2, x3, s1, s2 \geq 0$$

Setting up the Tableau

- Simplex tableau is used to perform row operations on the linear programming model as well as to check a solution for optimality.
- The tableau consists of the coefficient corresponding to the linear constraint variables and the coefficients of the objective function
- In the tableau below, the bolded top row of the tableau states what each column represents.
- The following two rows represent the linear constraint variable coefficients from the linear programming model, and the last row represents the objective function variable coefficients.

Example

- Maximize : $z = 8x_1 + 10x_2 + 7x_3$

$$x_1 + 3x_2 + 2x_3 + s_1 = 10$$

$$x_1 + 5x_2 + x_3 + s_2 = 8$$

$\mathbf{x_1}$	$\mathbf{x_2}$	$\mathbf{x_3}$	$\mathbf{s_1}$	$\mathbf{s_2}$	$\mathbf{z}$	$\mathbf{b}$
1	3	2	1	0	0	10
1	5	1	0	1	0	8
-8	-10	-7	0	0	1	0

Full Process of Simplex Method

$$
\begin{aligned}
\textbf{Max} \quad & 7x_1 + 6x_2 \\
\textbf{s.t.} \quad & 2x_1 + 4x_2 \leq 16 \\
& 3x_1 + 2x_2 \leq 12 \\
& x_1, x_2 \geq 0
\end{aligned}
$$